

NOVUS

COMPREHENSIVE SOFTWARE TEST CASE SPECIFICATION

BOSS Magazine PH — NOVUS Editorial and Publishing Management System
IEEE-Aligned Black-and-White Format

Derived from: NOVUS Fully Dressed Use Case Specification

Prepared by: QA / Test Engineering
Document Type: Test Case Specification (Traceable to Use Case Basic, Alternate, and Exception Flows)

1. Document Overview

This document specifies comprehensive test cases for the NOVUS web-based editorial and publishing management system, derived directly from the NOVUS Fully Dressed Use Case Specification (BOSS Magazine PH, IEEE-Aligned Black-and-White Format). Each use case's Basic Flow, every documented Alternate Flow, and every documented Exception Flow has been converted into one traceable, individually executable test case. The document mirrors the structure and black-and-white, two-column, IEEE-style presentation of the source specification, organized by the same nine modules and use case IDs.

Scope: 9 modules, 44 use cases, 163 test cases (1 positive Basic Flow test per use case, plus one test per Alternate Flow and one test per Exception Flow, including the standard Authorization/Session Failure (E-AUTH) exception present on protected state-changing use cases).

1.1 Test Case Field Definitions

Field	Definition
Test Case ID	Unique identifier: TC-<Use Case Number>-<Flow Code>. BF = Basic Flow, A1/A2/... = Alternate Flow, E1/E2/... = Exception Flow, E-AUTH = Authorization/Session Failure.
Related Use Case	The fully dressed use case this test case validates.
Requirement Traceability	The requirement ID inherited from the source use case, for end-to-end traceability.
Test Objective	What the test is intended to prove or disprove about system behavior.
Priority	Inherited from the use case's documented priority (High / Medium / Low).
Test Type	Positive (Basic Flow), or Negative/Alternate, or Negative/Exception.
Preconditions	System and data state required before test execution, including the specific condition under test.
Test Data	Representative accounts, records, and input values needed to execute the test.
Test Steps	Numbered, executable actions and verification points, traceable to the use case flow steps.
Expected Result	The correct system behavior, derived from the use case flow and postconditions.
Postconditions (Verification)	System state that must hold after execution, inherited from the use case.
Actual Result / Status	Left blank for the test executor to complete during test-cycle execution.

1.2 Test Design Notes

- Every test case traces one-to-one to a specific numbered flow (Basic, Alternate, or Exception) in the source use case, so the two documents can be cross-referenced by ID.
- Where the source use case does not specify exact numeric boundary values (e.g., file size limits, character limits, date ranges), the corresponding test case notes that the value must be confirmed against the technical design specification before it is finalized for execution; testers should not assume undocumented limits.
- Authorization/Session Failure (E-AUTH) test cases apply to protected, state-changing use cases and verify that no partial state change is committed when a session or permission check fails.
- This specification covers functional correctness of each documented flow. It does not replace dedicated performance, load, accessibility, or full security penetration test plans, which should be scoped separately.

Table of Contents

(Right-click and choose "Update Field", or press F9, if the entries above do not appear when this document is first opened.)

2. Module 1: Manage Editorial Pipeline

UC-1.1: Submit Bundle — Test Suite

Use case purpose: Allow an Editor to formally submit a completed content bundle (article draft, images, metadata) into the editorial pipeline for tracking and review.

Actors: Editor **Requirement:** PR-EDT-01 **Priority:** High

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-1.1.

Test Case ID	Scenario	Type	Priority
TC-1.1-BF	Verify successful submit bundle (Basic Flow)	Positive	High
TC-1.1-A1	Verify system behavior for alternate flow: Incomplete Bundle (Submit Bundle)	Negative / Alternate	High
TC-1.1-E1	Verify system behavior for exception flow: Save Failure (Submit Bundle)	Negative / Exception	High
TC-1.1-E-AUTH	Verify access is denied when session/authorization is invalid (Submit Bundle)	Negative / Exception	High

TC-1.1-BF — Verify successful submit bundle (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.1-BF
Related Use Case	UC-1.1 — Submit Bundle
Requirement Traceability	PR-EDT-01
Test Objective	Confirm that Editor can successfully complete the "Submit Bundle" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow an Editor to formally submit a completed content bundle (article draft, images, metadata) into the editorial pipeline for tracking and review.
Priority	High
Test Type	Positive
Preconditions	• System is online • Editor is authenticated and logged in • A draft bundle exists and is marked ready by its Writer & Graphics Designer
Test Data	Valid, active Editor account with the required role/permission; a content bundle in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Editor navigates to the Editorial Pipeline module. 2. Editor selects "Submit Bundle." 3. System displays draft bundles ready for submission. 4. Editor selects the target bundle and reviews its contents. 5. Editor clicks "Submit." 6. System validates that all required components (text, assets, metadata) are present.

TEST CASE ELEMENT	SPECIFICATION
	7. System sets the bundle's workflow status to "Submitted" and timestamps the action. 8. System displays a confirmation notice to the Editor. 9. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Bundle status changes to "Submitted" - Bundle appears on the Pipeline Dashboard
Postconditions (Verification)	• Bundle status changes to "Submitted" • Bundle appears on the Pipeline Dashboard
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.1-A1 — Verify system behavior for alternate flow: Incomplete Bundle (Submit Bundle)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.1-A1
Related Use Case	UC-1.1 — Submit Bundle
Requirement Traceability	PR-EDT-01
Test Objective	Confirm the system correctly handles the alternate scenario "Incomplete Bundle" defined in Alternate Flow A1 of "Submit Bundle", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Editor is authenticated and logged in • A draft bundle exists and is marked ready by its Writer & Graphics Designer • Condition under test: The condition "Incomplete Bundle" occurs for the content bundle during the "Submit Bundle" workflow.
Test Data	Editor account (as required by the scenario); a content bundle configured so the "Incomplete Bundle" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a content bundle / environment state that will trigger the "Incomplete Bundle" condition. 2. As Editor, perform the relevant Basic Flow action(s) on the content bundle so that the "Incomplete Bundle" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules.

TEST CASE ELEMENT	SPECIFICATION
	5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Incomplete Bundle" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Bundle status changes to "Submitted" • Bundle appears on the Pipeline Dashboard
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.1-E1 — Verify system behavior for exception flow: Save Failure (Submit Bundle)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.1-E1
Related Use Case	UC-1.1 — Submit Bundle
Requirement Traceability	PR-EDT-01
Test Objective	Confirm the system correctly handles the exception scenario "Save Failure" defined in Exception Flow E1 of "Submit Bundle", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Editor is authenticated and logged in • A draft bundle exists and is marked ready by its Writer & Graphics Designer • Condition under test: The condition "Save Failure" occurs for the content bundle during the "Submit Bundle" workflow.
Test Data	Editor account (as required by the scenario); a content bundle configured so the "Save Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a content bundle / environment state that will trigger the "Save Failure" condition.

TEST CASE ELEMENT	SPECIFICATION
	2. As Editor, perform the relevant Basic Flow action(s) on the content bundle so that the "Save Failure" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Save Failure" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Bundle status changes to "Submitted" • Bundle appears on the Pipeline Dashboard
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.1-E-AUTH — Verify access is denied when session/authorization is invalid (Submit Bundle)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.1-E-AUTH
Related Use Case	UC-1.1 — Submit Bundle
Requirement Traceability	PR-EDT-01
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Submit Bundle", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Editor is authenticated and logged in • A draft bundle exists and is marked ready by its Writer & Graphics Designer • Condition under test: Editor's session has expired, or Editor lacks the role/ownership permission required to perform "Submit Bundle", immediately before the state-changing step.

TEST CASE ELEMENT	SPECIFICATION
Test Data	Editor account (as required by the scenario); a content bundle configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Editor. 2. Attempt to initiate the protected "Submit Bundle" action (content bundle). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the content bundle. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Bundle status changes to "Submitted" • Bundle appears on the Pipeline Dashboard
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-1.2: Draft Article — Test Suite

Use case purpose: Allow a Writer & Editor to create and save a new article draft within the pipeline.

Actors: Writer, Editor, AI (secondary, via include) **Requirement:** PR-EDT-02 **Priority:** High

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-1.2.

Test Case ID	Scenario	Type	Priority
TC-1.2-BF	Verify successful draft article (Basic Flow)	Positive	High
TC-1.2-A1	Verify system behavior for alternate flow: Save As Incomplete (Draft Article)	Negative / Alternate	High
TC-1.2-E1	Verify system behavior for exception flow: Evaluation Service Unavailable (Draft Article)	Negative / Exception	High

TC-1.2-BF — Verify successful draft article (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.2-BF

TEST CASE ELEMENT	SPECIFICATION
Related Use Case	UC-1.2 — Draft Article
Requirement Traceability	PR-EDT-02
Test Objective	Confirm that Writer / Editor / AI (secondary, via include) can successfully complete the "Draft Article" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow a Writer & Editor to create and save a new article draft within the pipeline.
Priority	High
Test Type	Positive
Preconditions	• System is online • Writer or Editor is authenticated and logged in • Writer has an assigned topic or pitch
Test Data	Valid, active Writer / Editor / AI (secondary, via include) account with the required role/permission; a article draft in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Writer or Editor selects "Draft Article" from the Editorial Pipeline module. 2. System opens a blank article editor. 3. Writer or Editor enters the headline, body content, and relevant metadata (tags, category). 4. Writer or Editor saves the draft. 5. System stores the draft and assigns it a unique bundle reference. 6. System includes Generate Article Evaluation (UC-1.11) to automatically analyze the draft. 7. System displays the evaluation summary alongside the draft. 8. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - A new article draft exists with status "Drafting" or "Awaiting Evaluation" - An AI evaluation has been generated for the draft
Postconditions (Verification)	• A new article draft exists with status "Drafting" or "Awaiting Evaluation" • An AI evaluation has been generated for the draft
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.2-A1 — Verify system behavior for alternate flow: Save As Incomplete (Draft Article)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.2-A1
Related Use Case	UC-1.2 — Draft Article

TEST CASE ELEMENT	SPECIFICATION
Requirement Traceability	PR-EDT-02
Test Objective	Confirm the system correctly handles the alternate scenario "Save As Incomplete" defined in Alternate Flow A1 of "Draft Article", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Writer or Editor is authenticated and logged in • Writer has an assigned topic or pitch • Condition under test: The condition "Save As Incomplete" occurs for the article draft during the "Draft Article" workflow.
Test Data	Writer / Editor / AI (secondary, via include) account (as required by the scenario); a article draft configured so the "Save As Incomplete" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a article draft / environment state that will trigger the "Save As Incomplete" condition. 2. As Writer / Editor / AI (secondary, via include), perform the relevant Basic Flow action(s) on the article draft so that the "Save As Incomplete" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Save As Incomplete" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• A new article draft exists with status "Drafting" or "Awaiting Evaluation" • An AI evaluation has been generated for the draft
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.2-E1 — Verify system behavior for exception flow: Evaluation Service Unavailable (Draft Article)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.2-E1
Related Use Case	UC-1.2 — Draft Article
Requirement Traceability	PR-EDT-02
Test Objective	Confirm the system correctly handles the exception scenario "Evaluation Service Unavailable" defined in Exception Flow E1 of "Draft Article", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Writer or Editor is authenticated and logged in • Writer has an assigned topic or pitch • Condition under test: The condition "Evaluation Service Unavailable" occurs for the article draft during the "Draft Article" workflow.
Test Data	Writer / Editor / AI (secondary, via include) account (as required by the scenario); a article draft configured so the "Evaluation Service Unavailable" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a article draft / environment state that will trigger the "Evaluation Service Unavailable" condition. 2. As Writer / Editor / AI (secondary, via include), perform the relevant Basic Flow action(s) on the article draft so that the "Evaluation Service Unavailable" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Evaluation Service Unavailable" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• A new article draft exists with status "Drafting" or "Awaiting Evaluation" • An AI evaluation has been generated for the draft
Actual Result	<i>To be recorded during test execution.</i>

TEST CASE ELEMENT	SPECIFICATION
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-1.3: Edit Article Content — Test Suite

Use case purpose: Allow an Editor or Writer to modify the text content of an article already in the pipeline.

Actors: Editor, Writer **Requirement:** PR-EDT-03 **Priority:** High

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-1.3.

Test Case ID	Scenario	Type	Priority
TC-1.3-BF	Verify successful edit article content (Basic Flow)	Positive	High
TC-1.3-A1	Verify system behavior for alternate flow: Discard Changes (Edit Article Content)	Negative / Alternate	High
TC-1.3-E1	Verify system behavior for exception flow: Concurrent Edit Conflict (Edit Article Content)	Negative / Exception	High

TC-1.3-BF — Verify successful edit article content (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.3-BF
Related Use Case	UC-1.3 — Edit Article Content
Requirement Traceability	PR-EDT-03
Test Objective	Confirm that Editor / Writer can successfully complete the "Edit Article Content" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow an Editor or Writer to modify the text content of an article already in the pipeline.
Priority	High
Test Type	Positive
Preconditions	• System is online • User is authenticated as Editor or Writer • Target article exists in the pipeline and is not locked/published
Test Data	Valid, active Editor / Writer account with the required role/permission; a article content in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. User opens the target article bundle from the pipeline. 2. User selects "Edit Content." 3. System loads the article into an editable text editor. 4. User modifies headline, body text, or metadata. 5. User clicks "Save Changes."

TEST CASE ELEMENT	SPECIFICATION
	6. System validates the content and stores the new version. 7. System logs the edit (author, timestamp) in the article's version history. 8. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Article content is updated - Edit history/version log is updated
Postconditions (Verification)	• Article content is updated • Edit history/version log is updated
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.3-A1 — Verify system behavior for alternate flow: Discard Changes (Edit Article Content)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.3-A1
Related Use Case	UC-1.3 — Edit Article Content
Requirement Traceability	PR-EDT-03
Test Objective	Confirm the system correctly handles the alternate scenario "Discard Changes" defined in Alternate Flow A1 of "Edit Article Content", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • User is authenticated as Editor or Writer • Target article exists in the pipeline and is not locked/published • Condition under test: The condition "Discard Changes" occurs for the article content during the "Edit Article Content" workflow.
Test Data	Editor / Writer account (as required by the scenario); a article content configured so the "Discard Changes" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a article content / environment state that will trigger the "Discard Changes" condition. 2. As Editor / Writer, perform the relevant Basic Flow action(s) on the article content so that the "Discard Changes" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules.

TEST CASE ELEMENT	SPECIFICATION
	5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Discard Changes" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Article content is updated • Edit history/version log is updated
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.3-E1 — Verify system behavior for exception flow: Concurrent Edit Conflict (Edit Article Content)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.3-E1
Related Use Case	UC-1.3 — Edit Article Content
Requirement Traceability	PR-EDT-03
Test Objective	Confirm the system correctly handles the exception scenario "Concurrent Edit Conflict" defined in Exception Flow E1 of "Edit Article Content", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • User is authenticated as Editor or Writer • Target article exists in the pipeline and is not locked/published • Condition under test: The condition "Concurrent Edit Conflict" occurs for the article content during the "Edit Article Content" workflow.
Test Data	Editor / Writer account (as required by the scenario); a article content configured so the "Concurrent Edit Conflict" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a article content / environment state that will trigger the "Concurrent Edit Conflict" condition.

TEST CASE ELEMENT	SPECIFICATION
	2. As Editor / Writer, perform the relevant Basic Flow action(s) on the article content so that the "Concurrent Edit Conflict" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Concurrent Edit Conflict" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Article content is updated • Edit history/version log is updated
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-1.4: Request Revision — Test Suite

Use case purpose: Allow an Editor to request revisions for either a Writer-submitted article or a magazine design uploaded by a Graphics Designer, based on editorial or design review.

Actors: Editor, Writer (for article revision requests), Graphics Designer (for magazine-design revision requests)

Requirement: PR-EDT-04 **Priority:** High

This suite contains 5 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-1.4.

Test Case ID	Scenario	Type	Priority
TC-1.4-BF	Verify successful request revision (Basic Flow)	Positive	High
TC-1.4-A1	Verify system behavior for alternate flow: No Ai Feedback Available (Request Revision)	Negative / Alternate	High
TC-1.4-A2	Verify system behavior for alternate flow: Magazine Design Revision (Request Revision)	Negative / Alternate	High
TC-1.4-E1	Verify system behavior for exception flow: Notification Failure (Request Revision)	Negative / Exception	High
TC-1.4-E2	Verify system behavior for exception flow: Target State Changed Concurrently (Request Revision)	Negative / Exception	High

TC-1.4-BF — Verify successful request revision (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.4-BF
Related Use Case	UC-1.4 — Request Revision
Requirement Traceability	PR-EDT-04
Test Objective	Confirm that Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests) can successfully complete the "Request Revision" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow an Editor to request revisions for either a Writer-submitted article or a magazine design uploaded by a Graphics Designer, based on editorial or design review.
Priority	High
Test Type	Positive
Preconditions	• System is online • Editor is authenticated and logged in • Target article or magazine design exists and is in a reviewable state • Editor has permission to request revision
Test Data	Valid, active Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests) account with the required role/permission; a revision request in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Editor opens the submitted article or uploaded magazine design requiring review. 2. System identifies the target type as either a Writer-submitted article or a Graphics Designer-uploaded magazine design. 3. System displays the latest version, responsible contributor, current workflow status, and revision history. 4. Editor selects "Request Revision." 5. System prompts the Editor for specific revision notes appropriate to the selected target type. 6. Editor enters actionable feedback. For articles, feedback may address content, structure, grammar, factual support, or editorial requirements. For magazine designs, feedback may address page layout, typography, imagery, branding, dimensions, or other design requirements. 7. Editor reviews and confirms the revision request. 8. System revalidates that the target remains reviewable and that the Editor still has permission. 9. System preserves the submitted version and records the revision request, Editor, timestamp, target type, and feedback. 10. System changes the target status to "Revision Requested." 11. System notifies the responsible Writer for an article or Graphics Designer for a magazine design. 12. System updates the pipeline/dashboard and makes the revision request visible to the responsible contributor. 13. Verify each postcondition listed below is met.

TEST CASE ELEMENT	SPECIFICATION
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Target status changes to "Revision Requested" - Current submitted version is preserved - Revision notes are recorded - Responsible Writer or Graphics Designer is notified
Postconditions (Verification)	• Target status changes to "Revision Requested" • Current submitted version is preserved • Revision notes are recorded • Responsible Writer or Graphics Designer is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.4-A1 — Verify system behavior for alternate flow: No Ai Feedback Available (Request Revision)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.4-A1
Related Use Case	UC-1.4 — Request Revision
Requirement Traceability	PR-EDT-04
Test Objective	Confirm the system correctly handles the alternate scenario "No Ai Feedback Available" defined in Alternate Flow A1 of "Request Revision", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Editor is authenticated and logged in • Target article or magazine design exists and is in a reviewable state • Editor has permission to request revision • Condition under test: The condition "No Ai Feedback Available" occurs for the revision request during the "Request Revision" workflow.
Test Data	Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests) account (as required by the scenario); a revision request configured so the "No Ai Feedback Available" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a revision request / environment state that will trigger the "No Ai Feedback Available" condition. 2. As Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests), perform the relevant Basic Flow action(s) on the revision request so that the "No Ai Feedback Available" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow.

TEST CASE ELEMENT	SPECIFICATION
	4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "No Ai Feedback Available" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Target status changes to "Revision Requested" • Current submitted version is preserved • Revision notes are recorded • Responsible Writer or Graphics Designer is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.4-A2 — Verify system behavior for alternate flow: Magazine Design Revision (Request Revision)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.4-A2
Related Use Case	UC-1.4 — Request Revision
Requirement Traceability	PR-EDT-04
Test Objective	Confirm the system correctly handles the alternate scenario "Magazine Design Revision" defined in Alternate Flow A2 of "Request Revision", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Editor is authenticated and logged in • Target article or magazine design exists and is in a reviewable state • Editor has permission to request revision • Condition under test: The condition "Magazine Design Revision" occurs for the revision request during the "Request Revision" workflow.

TEST CASE ELEMENT	SPECIFICATION
Test Data	Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests) account (as required by the scenario); a revision request configured so the "Magazine Design Revision" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a revision request / environment state that will trigger the "Magazine Design Revision" condition. 2. As Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests), perform the relevant Basic Flow action(s) on the revision request so that the "Magazine Design Revision" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A2: it detects and validates the "Magazine Design Revision" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Target status changes to "Revision Requested" • Current submitted version is preserved • Revision notes are recorded • Responsible Writer or Graphics Designer is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.4-E1 — Verify system behavior for exception flow: Notification Failure (Request Revision)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.4-E1
Related Use Case	UC-1.4 — Request Revision
Requirement Traceability	PR-EDT-04
Test Objective	Confirm the system correctly handles the exception scenario "Notification Failure" defined in Exception Flow E1 of "Request Revision", protecting data integrity and informing the user appropriately.

TEST CASE ELEMENT SPECIFICATION	
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Editor is authenticated and logged in • Target article or magazine design exists and is in a reviewable state • Editor has permission to request revision • Condition under test: The condition "Notification Failure" occurs for the revision request during the "Request Revision" workflow.
Test Data	Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests) account (as required by the scenario); a revision request configured so the "Notification Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a revision request / environment state that will trigger the "Notification Failure" condition. 2. As Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests), perform the relevant Basic Flow action(s) on the revision request so that the "Notification Failure" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Notification Failure" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Target status changes to "Revision Requested" • Current submitted version is preserved • Revision notes are recorded • Responsible Writer or Graphics Designer is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.4-E2 — Verify system behavior for exception flow: Target State Changed Concurrently (Request Revision)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.4-E2
Related Use Case	UC-1.4 — Request Revision
Requirement Traceability	PR-EDT-04
Test Objective	Confirm the system correctly handles the exception scenario "Target State Changed Concurrently" defined in Exception Flow E2 of "Request Revision", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Editor is authenticated and logged in • Target article or magazine design exists and is in a reviewable state • Editor has permission to request revision • Condition under test: The condition "Target State Changed Concurrently" occurs for the revision request during the "Request Revision" workflow.
Test Data	Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests) account (as required by the scenario); a revision request configured so the "Target State Changed Concurrently" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a revision request / environment state that will trigger the "Target State Changed Concurrently" condition. 2. As Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests), perform the relevant Basic Flow action(s) on the revision request so that the "Target State Changed Concurrently" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E2: it detects the "Target State Changed Concurrently" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Target status changes to "Revision Requested" • Current submitted version is preserved • Revision notes are recorded

TEST CASE ELEMENT	SPECIFICATION
	Responsible Writer or Graphics Designer is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-1.5: Override AI — Test Suite

Use case purpose: Allow an Editor to manually override the AI's evaluation verdict for an article, using editorial judgment.

Actors: Editor **Requirement:** PR-EDT-05 **Priority:** Medium

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-1.5.

Test Case ID	Scenario	Type	Priority
TC-1.5-BF	Verify successful override ai (Basic Flow)	Positive	Medium
TC-1.5-A1	Verify system behavior for alternate flow: Partial Override (Override AI)	Negative / Alternate	Medium
TC-1.5-E1	Verify system behavior for exception flow: Missing Justification (Override AI)	Negative / Exception	Medium

TC-1.5-BF — Verify successful override ai (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.5-BF
Related Use Case	UC-1.5 — Override AI
Requirement Traceability	PR-EDT-05
Test Objective	Confirm that Editor can successfully complete the "Override AI" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow an Editor to manually override the AI's evaluation verdict for an article, using editorial judgment.
Priority	Medium
Test Type	Positive
Preconditions	- System is online - Editor is authenticated and logged in - Generate Article Evaluation has produced a verdict the Editor disagrees with
Test Data	Valid, active Editor account with the required role/permission; a AI evaluation verdict in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Editor reviews the AI evaluation verdict for an article.

TEST CASE ELEMENT	SPECIFICATION
	2. Editor selects "Override AI." 3. System requires the Editor to enter a justification for the override. 4. Editor submits the justification. 5. System updates the article's evaluation record to reflect the manual override. 6. System allows the article to proceed in the pipeline despite the original AI verdict. 7. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Article's evaluation verdict is flagged as "Editor Override" - Override justification is recorded
Postconditions (Verification)	• Article's evaluation verdict is flagged as "Editor Override" • Override justification is recorded
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.5-A1 — Verify system behavior for alternate flow: Partial Override (Override AI)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.5-A1
Related Use Case	UC-1.5 — Override AI
Requirement Traceability	PR-EDT-05
Test Objective	Confirm the system correctly handles the alternate scenario "Partial Override" defined in Alternate Flow A1 of "Override AI", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	• System is online • Editor is authenticated and logged in • Generate Article Evaluation has produced a verdict the Editor disagrees with • Condition under test: The condition "Partial Override" occurs for the AI evaluation verdict during the "Override AI" workflow.
Test Data	Editor account (as required by the scenario); a AI evaluation verdict configured so the "Partial Override" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a AI evaluation verdict / environment state that will trigger the "Partial Override" condition. 2. As Editor, perform the relevant Basic Flow action(s) on the AI evaluation verdict so that the "Partial Override" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow.

TEST CASE ELEMENT	SPECIFICATION
	4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Partial Override" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Article's evaluation verdict is flagged as "Editor Override" • Override justification is recorded
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.5-E1 — Verify system behavior for exception flow: Missing Justification (Override AI)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.5-E1
Related Use Case	UC-1.5 — Override AI
Requirement Traceability	PR-EDT-05
Test Objective	Confirm the system correctly handles the exception scenario "Missing Justification" defined in Exception Flow E1 of "Override AI", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • Editor is authenticated and logged in • Generate Article Evaluation has produced a verdict the Editor disagrees with • Condition under test: The condition "Missing Justification" occurs for the AI evaluation verdict during the "Override AI" workflow.
Test Data	Editor account (as required by the scenario); a AI evaluation verdict configured so the "Missing Justification" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).

TEST CASE ELEMENT	SPECIFICATION
Test Steps	1. Prepare a AI evaluation verdict / environment state that will trigger the "Missing Justification" condition. 2. As Editor, perform the relevant Basic Flow action(s) on the AI evaluation verdict so that the "Missing Justification" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Missing Justification" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Article's evaluation verdict is flagged as "Editor Override" • Override justification is recorded
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-1.6: Update Article Workflow Status — Test Suite

Use case purpose: Allow an Editor to move an article through its editorial workflow stages (e.g., In Review, Approved, Ready for Publication).

Actors: Editor **Requirement:** PR-EDT-06 **Priority:** Medium

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-1.6.

Test Case ID	Scenario	Type	Priority
TC-1.6-BF	Verify successful update article workflow status (Basic Flow)	Positive	Medium
TC-1.6-A1	Verify system behavior for alternate flow: Revert Status (Update Article Workflow Status)	Negative / Alternate	Medium
TC-1.6-E1	Verify system behavior for exception flow: Invalid Transition (Update Article Workflow Status)	Negative / Exception	Medium
TC-1.6-E-AUTH	Verify access is denied when session/authorization is invalid (Update Article Workflow Status)	Negative / Exception	Medium

TC-1.6-BF — Verify successful update article workflow status (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.6-BF
Related Use Case	UC-1.6 — Update Article Workflow Status
Requirement Traceability	PR-EDT-06
Test Objective	Confirm that Editor can successfully complete the "Update Article Workflow Status" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow an Editor to move an article through its editorial workflow stages (e.g., In Review, Approved, Ready for Publication).
Priority	Medium
Test Type	Positive
Preconditions	• System is online • Editor is authenticated and logged in • Article exists in the pipeline
Test Data	Valid, active Editor account with the required role/permission; a article workflow status in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Editor opens the target article's pipeline entry. 2. Editor selects "Update Workflow Status." 3. System displays available next-stage options based on the current status. 4. Editor selects the new status. 5. Editor confirms the change. 6. System updates the article's status and records the transition with a timestamp. 7. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Article's workflow status is updated - Status change is logged in the article's history
Postconditions (Verification)	• Article's workflow status is updated • Status change is logged in the article's history
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.6-A1 — Verify system behavior for alternate flow: Revert Status (Update Article Workflow Status)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.6-A1
Related Use Case	UC-1.6 — Update Article Workflow Status

TEST CASE ELEMENT	SPECIFICATION
Requirement Traceability	PR-EDT-06
Test Objective	Confirm the system correctly handles the alternate scenario "Revert Status" defined in Alternate Flow A1 of "Update Article Workflow Status", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	• System is online • Editor is authenticated and logged in • Article exists in the pipeline • Condition under test: The condition "Revert Status" occurs for the article workflow status during the "Update Article Workflow Status" workflow.
Test Data	Editor account (as required by the scenario); a article workflow status configured so the "Revert Status" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a article workflow status / environment state that will trigger the "Revert Status" condition. 2. As Editor, perform the relevant Basic Flow action(s) on the article workflow status so that the "Revert Status" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Revert Status" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Article's workflow status is updated • Status change is logged in the article's history
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.6-E1 — Verify system behavior for exception flow: Invalid Transition (Update Article Workflow Status)

TEST CASE SPECIFICATION	
Test Case ID	TC-1.6-E1
Related Use Case	UC-1.6 — Update Article Workflow Status
Requirement Traceability	PR-EDT-06
Test Objective	Confirm the system correctly handles the exception scenario "Invalid Transition" defined in Exception Flow E1 of "Update Article Workflow Status", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • Editor is authenticated and logged in • Article exists in the pipeline • Condition under test: The condition "Invalid Transition" occurs for the article workflow status during the "Update Article Workflow Status" workflow.
Test Data	Editor account (as required by the scenario); a article workflow status configured so the "Invalid Transition" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a article workflow status / environment state that will trigger the "Invalid Transition" condition. 2. As Editor, perform the relevant Basic Flow action(s) on the article workflow status so that the "Invalid Transition" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Invalid Transition" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Article's workflow status is updated • Status change is logged in the article's history
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.6-E-AUTH — Verify access is denied when session/authorization is invalid (Update Article Workflow Status)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.6-E-AUTH
Related Use Case	UC-1.6 — Update Article Workflow Status
Requirement Traceability	PR-EDT-06
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Update Article Workflow Status", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • Editor is authenticated and logged in • Article exists in the pipeline • Condition under test: Editor's session has expired, or Editor lacks the role/ownership permission required to perform "Update Article Workflow Status", immediately before the state-changing step.
Test Data	Editor account (as required by the scenario); a article workflow status configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Editor. 2. Attempt to initiate the protected "Update Article Workflow Status" action (article workflow status). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the article workflow status. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Article's workflow status is updated • Status change is logged in the article's history
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-1.7: Withdraw Article — Test Suite

Use case purpose: Allow an Editor or Writer to remove an article from the active pipeline before it is published.

Actors: Editor, Writer **Requirement:** PR-EDT-07 **Priority:** Medium

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-1.7.

Test Case ID	Scenario	Type	Priority
TC-1.7-BF	Verify successful withdraw article (Basic Flow)	Positive	Medium
TC-1.7-A1	Verify system behavior for alternate flow: Restore Withdrawn Article (Withdraw Article)	Negative / Alternate	Medium
TC-1.7-E1	Verify system behavior for exception flow: Already Published (Withdraw Article)	Negative / Exception	Medium
TC-1.7-E-AUTH	Verify access is denied when session/authorization is invalid (Withdraw Article)	Negative / Exception	Medium

TC-1.7-BF — Verify successful withdraw article (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.7-BF
Related Use Case	UC-1.7 — Withdraw Article
Requirement Traceability	PR-EDT-07
Test Objective	Confirm that Editor / Writer can successfully complete the "Withdraw Article" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow an Editor or Writer to remove an article from the active pipeline before it is published.
Priority	Medium
Test Type	Positive
Preconditions	• System is online • User is authenticated as Editor or Writer • Article is not yet published
Test Data	Valid, active Editor / Writer account with the required role/permission; a article in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. User opens the target article's pipeline entry. 2. User selects "Withdraw Article." 3. System prompts for confirmation and a withdrawal reason. 4. User confirms. 5. System sets the article status to "Withdrawn" and logs the reason. 6. System removes the article from active pipeline dashboards. 7. Verify each postcondition listed below is met.

TEST CASE ELEMENT	SPECIFICATION
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Article status changes to "Withdrawn" - Article is removed from active pipeline views
Postconditions (Verification)	• Article status changes to "Withdrawn" • Article is removed from active pipeline views
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.7-A1 — Verify system behavior for alternate flow: Restore Withdrawn Article (Withdraw Article)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.7-A1
Related Use Case	UC-1.7 — Withdraw Article
Requirement Traceability	PR-EDT-07
Test Objective	Confirm the system correctly handles the alternate scenario "Restore Withdrawn Article" defined in Alternate Flow A1 of "Withdraw Article", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	• System is online • User is authenticated as Editor or Writer • Article is not yet published • Condition under test: The condition "Restore Withdrawn Article" occurs for the article during the "Withdraw Article" workflow.
Test Data	Editor / Writer account (as required by the scenario); a article configured so the "Restore Withdrawn Article" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a article / environment state that will trigger the "Restore Withdrawn Article" condition. 2. As Editor / Writer, perform the relevant Basic Flow action(s) on the article so that the "Restore Withdrawn Article" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state.

TEST CASE ELEMENT	SPECIFICATION
	6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Restore Withdrawn Article" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	- Article status changes to "Withdrawn" - Article is removed from active pipeline views
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.7-E1 — Verify system behavior for exception flow: Already Published (Withdraw Article)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.7-E1
Related Use Case	UC-1.7 — Withdraw Article
Requirement Traceability	PR-EDT-07
Test Objective	Confirm the system correctly handles the exception scenario "Already Published" defined in Exception Flow E1 of "Withdraw Article", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	- System is online - User is authenticated as Editor or Writer - Article is not yet published - Condition under test: The condition "Already Published" occurs for the article during the "Withdraw Article" workflow.
Test Data	Editor / Writer account (as required by the scenario); a article configured so the "Already Published" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	- Prepare a article / environment state that will trigger the "Already Published" condition. - As Editor / Writer, perform the relevant Basic Flow action(s) on the article so that the "Already Published" condition is encountered. - Verify the system detects the condition at the correct point in the Basic Flow.

TEST CASE ELEMENT	SPECIFICATION
	4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Already Published" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Article status changes to "Withdrawn" • Article is removed from active pipeline views
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.7-E-AUTH — Verify access is denied when session/authorization is invalid (Withdraw Article)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.7-E-AUTH
Related Use Case	UC-1.7 — Withdraw Article
Requirement Traceability	PR-EDT-07
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Withdraw Article", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • User is authenticated as Editor or Writer • Article is not yet published • Condition under test: Editor / Writer's session has expired, or Editor / Writer lacks the role/ownership permission required to perform "Withdraw Article", immediately before the state-changing step.

TEST CASE ELEMENT	SPECIFICATION
Test Data	Editor / Writer account (as required by the scenario); a article configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Editor / Writer. 2. Attempt to initiate the protected "Withdraw Article" action (article). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the article. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Article status changes to "Withdrawn" • Article is removed from active pipeline views
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-1.8: Upload Magazine Design — Test Suite

Use case purpose: Allow a Graphics Designer to upload a magazine design for an issue, create a versioned submission, and provide an Editor with a design that can be reviewed and returned for revision.

Actors: Graphics Designer, Editor (secondary reviewer) **Requirement:** PR-EDT-08 **Priority:** High

This suite contains 7 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-1.8.

Test Case ID	Scenario	Type	Priority
TC-1.8-BF	Verify successful upload magazine design (Basic Flow)	Positive	High
TC-1.8-A1	Verify system behavior for alternate flow: Version Replacement (Upload Magazine Design)	Negative / Alternate	High
TC-1.8-A2	Verify system behavior for alternate flow: Upload After Revision Request (Upload Magazine Design)	Negative / Alternate	High
TC-1.8-E1	Verify system behavior for exception flow: Unsupported Format Or Size Limit Exceeded (Upload Magazine Design)	Negative / Exception	High
TC-1.8-E2	Verify system behavior for exception flow: Upload Interruption (Upload Magazine Design)	Negative / Exception	High

Test Case ID	Scenario	Type	Priority
TC-1.8-E3	Verify system behavior for exception flow: Unauthorized Target Issue (Upload Magazine Design)	Negative / Exception	High
TC-1.8-E4	Verify system behavior for exception flow: Concurrent Design Submission (Upload Magazine Design)	Negative / Exception	High

TC-1.8-BF — Verify successful upload magazine design (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.8-BF
Related Use Case	UC-1.8 — Upload Magazine Design
Requirement Traceability	PR-EDT-08
Test Objective	Confirm that Graphics Designer / Editor (secondary reviewer) can successfully complete the "Upload Magazine Design" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow a Graphics Designer to upload a magazine design for an issue, create a versioned submission, and provide an Editor with a design that can be reviewed and returned for revision.
Priority	High
Test Type	Positive
Preconditions	• System is online • Graphics Designer is authenticated and logged in • Target issue exists • Designer is assigned or authorized to upload to the target issue • Design file meets configured publication requirements
Test Data	Valid, active Graphics Designer / Editor (secondary reviewer) account with the required role/permission; a magazine design file in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. User navigates to the specific issue or bundle record in the system. User selects "Upload Magazine Designs." System opens a secure file upload interface. User selects the compiled digital magazine file (e.g., PDF) from their local device. System validates the file format and ensures it is within the maximum file size limits. System securely uploads the file, attaches it to the issue record, and timestamps the action. System displays a success confirmation notice to the user. 2. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Design file is securely stored as a versioned asset - Design is linked to the target issue - New version is submitted for Editor review - Prior versions remain traceable - Editor is notified
Postconditions (Verification)	• Design file is securely stored as a versioned asset • Design is linked to the target issue • New version is submitted for Editor review

TEST CASE ELEMENT	SPECIFICATION
	• Prior versions remain traceable • Editor is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.8-A1 — Verify system behavior for alternate flow: Version Replacement (Upload Magazine Design)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.8-A1
Related Use Case	UC-1.8 — Upload Magazine Design
Requirement Traceability	PR-EDT-08
Test Objective	Confirm the system correctly handles the alternate scenario "Version Replacement" defined in Alternate Flow A1 of "Upload Magazine Design", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Graphics Designer is authenticated and logged in • Target issue exists • Designer is assigned or authorized to upload to the target issue • Design file meets configured publication requirements • Condition under test: The condition "Version Replacement" occurs for the magazine design file during the "Upload Magazine Design" workflow.
Test Data	Graphics Designer / Editor (secondary reviewer) account (as required by the scenario); a magazine design file configured so the "Version Replacement" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a magazine design file / environment state that will trigger the "Version Replacement" condition. 2. As Graphics Designer / Editor (secondary reviewer), perform the relevant Basic Flow action(s) on the magazine design file so that the "Version Replacement" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state.

TEST CASE ELEMENT	SPECIFICATION
	6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Version Replacement" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Design file is securely stored as a versioned asset • Design is linked to the target issue • New version is submitted for Editor review • Prior versions remain traceable • Editor is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.8-A2 — Verify system behavior for alternate flow: Upload After Revision Request (Upload Magazine Design)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.8-A2
Related Use Case	UC-1.8 — Upload Magazine Design
Requirement Traceability	PR-EDT-08
Test Objective	Confirm the system correctly handles the alternate scenario "Upload After Revision Request" defined in Alternate Flow A2 of "Upload Magazine Design", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Graphics Designer is authenticated and logged in • Target issue exists • Designer is assigned or authorized to upload to the target issue • Design file meets configured publication requirements • Condition under test: The condition "Upload After Revision Request" occurs for the magazine design file during the "Upload Magazine Design" workflow.
Test Data	Graphics Designer / Editor (secondary reviewer) account (as required by the scenario); a magazine design file configured so the "Upload After Revision Request" condition

TEST CASE ELEMENT	SPECIFICATION
	can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a magazine design file / environment state that will trigger the "Upload After Revision Request" condition. 2. As Graphics Designer / Editor (secondary reviewer), perform the relevant Basic Flow action(s) on the magazine design file so that the "Upload After Revision Request" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A2: it detects and validates the "Upload After Revision Request" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Design file is securely stored as a versioned asset • Design is linked to the target issue • New version is submitted for Editor review • Prior versions remain traceable • Editor is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.8-E1 — Verify system behavior for exception flow: Unsupported Format Or Size Limit Exceeded (Upload Magazine Design)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.8-E1
Related Use Case	UC-1.8 — Upload Magazine Design
Requirement Traceability	PR-EDT-08
Test Objective	Confirm the system correctly handles the exception scenario "Unsupported Format Or Size Limit Exceeded" defined in Exception Flow E1 of "Upload Magazine Design", protecting data integrity and informing the user appropriately.

TEST CASE ELEMENT	SPECIFICATION
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Graphics Designer is authenticated and logged in • Target issue exists • Designer is assigned or authorized to upload to the target issue • Design file meets configured publication requirements • Condition under test: The condition "Unsupported Format Or Size Limit Exceeded" occurs for the magazine design file during the "Upload Magazine Design" workflow.
Test Data	Graphics Designer / Editor (secondary reviewer) account (as required by the scenario); a magazine design file configured so the "Unsupported Format Or Size Limit Exceeded" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a magazine design file / environment state that will trigger the "Unsupported Format Or Size Limit Exceeded" condition. 2. As Graphics Designer / Editor (secondary reviewer), perform the relevant Basic Flow action(s) on the magazine design file so that the "Unsupported Format Or Size Limit Exceeded" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Unsupported Format Or Size Limit Exceeded" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Design file is securely stored as a versioned asset • Design is linked to the target issue • New version is submitted for Editor review • Prior versions remain traceable • Editor is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.8-E2 — Verify system behavior for exception flow: Upload Interruption (Upload Magazine Design)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.8-E2
Related Use Case	UC-1.8 — Upload Magazine Design
Requirement Traceability	PR-EDT-08
Test Objective	Confirm the system correctly handles the exception scenario "Upload Interruption" defined in Exception Flow E2 of "Upload Magazine Design", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Graphics Designer is authenticated and logged in • Target issue exists • Designer is assigned or authorized to upload to the target issue • Design file meets configured publication requirements • Condition under test: The condition "Upload Interruption" occurs for the magazine design file during the "Upload Magazine Design" workflow.
Test Data	Graphics Designer / Editor (secondary reviewer) account (as required by the scenario); a magazine design file configured so the "Upload Interruption" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a magazine design file / environment state that will trigger the "Upload Interruption" condition. 2. As Graphics Designer / Editor (secondary reviewer), perform the relevant Basic Flow action(s) on the magazine design file so that the "Upload Interruption" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E2: it detects the "Upload Interruption" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.

TEST CASE ELEMENT	SPECIFICATION
Postconditions (Verification)	• Design file is securely stored as a versioned asset • Design is linked to the target issue • New version is submitted for Editor review • Prior versions remain traceable • Editor is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.8-E3 — Verify system behavior for exception flow: Unauthorized Target Issue (Upload Magazine Design)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.8-E3
Related Use Case	UC-1.8 — Upload Magazine Design
Requirement Traceability	PR-EDT-08
Test Objective	Confirm the system correctly handles the exception scenario "Unauthorized Target Issue" defined in Exception Flow E3 of "Upload Magazine Design", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Graphics Designer is authenticated and logged in • Target issue exists • Designer is assigned or authorized to upload to the target issue • Design file meets configured publication requirements • Condition under test: The condition "Unauthorized Target Issue" occurs for the magazine design file during the "Upload Magazine Design" workflow.
Test Data	Graphics Designer / Editor (secondary reviewer) account (as required by the scenario); a magazine design file configured so the "Unauthorized Target Issue" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a magazine design file / environment state that will trigger the "Unauthorized Target Issue" condition. 2. As Graphics Designer / Editor (secondary reviewer), perform the relevant Basic Flow action(s) on the magazine design file so that the "Unauthorized Target Issue" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules.

TEST CASE ELEMENT	SPECIFICATION
	5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E3: it detects the "Unauthorized Target Issue" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Design file is securely stored as a versioned asset • Design is linked to the target issue • New version is submitted for Editor review • Prior versions remain traceable • Editor is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.8-E4 — Verify system behavior for exception flow: Concurrent Design Submission (Upload Magazine Design)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.8-E4
Related Use Case	UC-1.8 — Upload Magazine Design
Requirement Traceability	PR-EDT-08
Test Objective	Confirm the system correctly handles the exception scenario "Concurrent Design Submission" defined in Exception Flow E4 of "Upload Magazine Design", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Graphics Designer is authenticated and logged in • Target issue exists • Designer is assigned or authorized to upload to the target issue • Design file meets configured publication requirements • Condition under test: The condition "Concurrent Design Submission" occurs for the magazine design file during the "Upload Magazine Design" workflow.

TEST CASE ELEMENT	SPECIFICATION
Test Data	Graphics Designer / Editor (secondary reviewer) account (as required by the scenario); a magazine design file configured so the "Concurrent Design Submission" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a magazine design file / environment state that will trigger the "Concurrent Design Submission" condition. 2. As Graphics Designer / Editor (secondary reviewer), perform the relevant Basic Flow action(s) on the magazine design file so that the "Concurrent Design Submission" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E4: it detects the "Concurrent Design Submission" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Design file is securely stored as a versioned asset • Design is linked to the target issue • New version is submitted for Editor review • Prior versions remain traceable • Editor is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-1.9: View Pipeline Dashboard — Test Suite

Use case purpose: Allow Editorial Staff to view an overview of all bundles currently active in the editorial pipeline, including status and assignees.

Actors: Editorial Staff (Editor, Writer, Graphics Designer, Publisher) **Requirement:** PR-EDT-09 **Priority:** Medium

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-1.9.

Test Case ID	Scenario	Type	Priority
TC-1.9-BF	Verify successful view pipeline dashboard (Basic Flow)	Positive	Medium

Test Case ID	Scenario	Type	Priority
TC-1.9-A1	Verify system behavior for alternate flow: No Active Bundles (View Pipeline Dashboard)	Negative / Alternate	Medium
TC-1.9-E1	Verify system behavior for exception flow: Data Load Failure (View Pipeline Dashboard)	Negative / Exception	Medium

TC-1.9-BF — Verify successful view pipeline dashboard (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.9-BF
Related Use Case	UC-1.9 — View Pipeline Dashboard
Requirement Traceability	PR-EDT-09
Test Objective	Confirm that Editorial Staff (Editor, Writer, Graphics Designer, Publisher) can successfully complete the "View Pipeline Dashboard" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow Editorial Staff to view an overview of all bundles currently active in the editorial pipeline, including status and assignees.
Priority	Medium
Test Type	Positive
Preconditions	• System is online • User is authenticated as Editor, Writer, Publisher or Graphics Designer
Test Data	Valid, active Editorial Staff (Editor, Writer, Graphics Designer, Publisher) account with the required role/permission; a pipeline dashboard view in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Editorial Staff member navigates to the Editorial Pipeline module. 2. Editorial Staff selects "View Pipeline Dashboard." 3. System retrieves all active bundles and their current statuses. 4. System displays the dashboard with filters (by status, assignee, deadline). 5. Editorial Staff member may apply filters to narrow the view. 6. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Dashboard is displayed reflecting current pipeline state
Postconditions (Verification)	• Dashboard is displayed reflecting current pipeline state
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.9-A1 — Verify system behavior for alternate flow: No Active Bundles (View Pipeline Dashboard)

TEST CASE SPECIFICATION	
Test Case ID	TC-1.9-A1
Related Use Case	UC-1.9 — View Pipeline Dashboard
Requirement Traceability	PR-EDT-09
Test Objective	Confirm the system correctly handles the alternate scenario "No Active Bundles" defined in Alternate Flow A1 of "View Pipeline Dashboard", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	• System is online • User is authenticated as Editor, Writer, Publisher or Graphics Designer • Condition under test: The condition "No Active Bundles" occurs for the pipeline dashboard view during the "View Pipeline Dashboard" workflow.
Test Data	Editorial Staff (Editor, Writer, Graphics Designer, Publisher) account (as required by the scenario); a pipeline dashboard view configured so the "No Active Bundles" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a pipeline dashboard view / environment state that will trigger the "No Active Bundles" condition. 2. As Editorial Staff (Editor, Writer, Graphics Designer, Publisher), perform the relevant Basic Flow action(s) on the pipeline dashboard view so that the "No Active Bundles" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "No Active Bundles" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Dashboard is displayed reflecting current pipeline state
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.9-E1 — Verify system behavior for exception flow: Data Load Failure (View Pipeline Dashboard)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.9-E1
Related Use Case	UC-1.9 — View Pipeline Dashboard
Requirement Traceability	PR-EDT-09
Test Objective	Confirm the system correctly handles the exception scenario "Data Load Failure" defined in Exception Flow E1 of "View Pipeline Dashboard", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • User is authenticated as Editor, Writer, Publisher or Graphics Designer • Condition under test: The condition "Data Load Failure" occurs for the pipeline dashboard view during the "View Pipeline Dashboard" workflow.
Test Data	Editorial Staff (Editor, Writer, Graphics Designer, Publisher) account (as required by the scenario); a pipeline dashboard view configured so the "Data Load Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a pipeline dashboard view / environment state that will trigger the "Data Load Failure" condition. 2. As Editorial Staff (Editor, Writer, Graphics Designer, Publisher), perform the relevant Basic Flow action(s) on the pipeline dashboard view so that the "Data Load Failure" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Data Load Failure" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Dashboard is displayed reflecting current pipeline state
Actual Result	<i>To be recorded during test execution.</i>

TEST CASE ELEMENT	SPECIFICATION
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-1.10: View Bundle — Test Suite

Use case purpose: Allow Editorial Staff to view the full detail of a single content bundle, including its content, assets, status, and history.

Actors: Editorial Staff (Editor, Writer, Graphics Designer, Publisher) **Requirement:** PR-EDT-10 **Priority:** Medium

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-1.10.

Test Case ID	Scenario	Type	Priority
TC-1.10-BF	Verify successful view bundle (Basic Flow)	Positive	Medium
TC-1.10-A1	Verify system behavior for alternate flow: Navigate To Related Action (View Bundle)	Negative / Alternate	Medium
TC-1.10-E1	Verify system behavior for exception flow: Bundle Not Found (View Bundle)	Negative / Exception	Medium

TC-1.10-BF — Verify successful view bundle (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.10-BF
Related Use Case	UC-1.10 — View Bundle
Requirement Traceability	PR-EDT-10
Test Objective	Confirm that Editorial Staff (Editor, Writer, Graphics Designer, Publisher) can successfully complete the "View Bundle" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow Editorial Staff to view the full detail of a single content bundle, including its content, assets, status, and history.
Priority	Medium
Test Type	Positive
Preconditions	• System is online • User is authenticated as Editor, Writer, Graphics Designer, or Publisher • Target bundle exists
Test Data	Valid, active Editorial Staff (Editor, Writer, Graphics Designer, Publisher) account with the required role/permission; a bundle record in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Editorial Staff member selects a bundle from the Pipeline Dashboard or search. 2. System retrieves the bundle's content, assets, status, and revision history. 3. System displays the full bundle detail view.

TEST CASE ELEMENT	SPECIFICATION
	4. Editorial Staff member reviews the information. 5. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Bundle detail view is displayed
Postconditions (Verification)	• Bundle detail view is displayed
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.10-A1 — Verify system behavior for alternate flow: Navigate To Related Action (View Bundle)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.10-A1
Related Use Case	UC-1.10 — View Bundle
Requirement Traceability	PR-EDT-10
Test Objective	Confirm the system correctly handles the alternate scenario "Navigate To Related Action" defined in Alternate Flow A1 of "View Bundle", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	• System is online • User is authenticated as Editor, Writer, Graphics Designer, or Publisher • Target bundle exists • Condition under test: The condition "Navigate To Related Action" occurs for the bundle record during the "View Bundle" workflow.
Test Data	Editorial Staff (Editor, Writer, Graphics Designer, Publisher) account (as required by the scenario); a bundle record configured so the "Navigate To Related Action" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a bundle record / environment state that will trigger the "Navigate To Related Action" condition. 2. As Editorial Staff (Editor, Writer, Graphics Designer, Publisher), perform the relevant Basic Flow action(s) on the bundle record so that the "Navigate To Related Action" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules.

TEST CASE ELEMENT	SPECIFICATION
	5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Navigate To Related Action" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Bundle detail view is displayed
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.10-E1 — Verify system behavior for exception flow: Bundle Not Found (View Bundle)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.10-E1
Related Use Case	UC-1.10 — View Bundle
Requirement Traceability	PR-EDT-10
Test Objective	Confirm the system correctly handles the exception scenario "Bundle Not Found" defined in Exception Flow E1 of "View Bundle", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • User is authenticated as Editor, Writer, Graphics Designer, or Publisher • Target bundle exists • Condition under test: The condition "Bundle Not Found" occurs for the bundle record during the "View Bundle" workflow.
Test Data	Editorial Staff (Editor, Writer, Graphics Designer, Publisher) account (as required by the scenario); a bundle record configured so the "Bundle Not Found" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a bundle record / environment state that will trigger the "Bundle Not Found" condition.

TEST CASE ELEMENT	SPECIFICATION
	2. As Editorial Staff (Editor, Writer, Graphics Designer, Publisher), perform the relevant Basic Flow action(s) on the bundle record so that the "Bundle Not Found" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Bundle Not Found" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Bundle detail view is displayed
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-1.11: Generate Article Evaluation — Test Suite

Use case purpose: Allow an Editor to request revisions for either a Writer-submitted article or a magazine design uploaded by a Graphics Designer, based on editorial or design review.

Actors: Editor, Writer (for article revision requests), Graphics Designer (for magazine-design revision requests)

Requirement: PR-EDT-11 **Priority:** High

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-1.11.

Test Case ID	Scenario	Type	Priority
TC-1.11-BF	Verify successful generate article evaluation (Basic Flow)	Positive	High
TC-1.11-A1	Verify system behavior for alternate flow: High-Confidence Approval (Generate Article Evaluation)	Negative / Alternate	High
TC-1.11-E1	Verify system behavior for exception flow: Ai Service Timeout (Generate Article Evaluation)	Negative / Exception	High

TC-1.11-BF — Verify successful generate article evaluation (Basic Flow)

TEST CASE SPECIFICATION	
Test Case ID	TC-1.11-BF
Related Use Case	UC-1.11 — Generate Article Evaluation
Requirement Traceability	PR-EDT-11
Test Objective	Confirm that Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests) can successfully complete the "Generate Article Evaluation" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow an Editor to request revisions for either a Writer-submitted article or a magazine design uploaded by a Graphics Designer, based on editorial or design review.
Priority	High
Test Type	Positive
Preconditions	• System is online • Editor is authenticated and logged in • Target article or magazine design exists and is in a reviewable state • Editor has permission to request revision
Test Data	Valid, active Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests) account with the required role/permission; a article evaluation in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. System sends the article's content to the AI evaluation service (triggered via include from Draft Article). 2. AI analyzes the article for grammar, and readability. 3. AI returns a quality score and a recommendation (Approve / Reject). 4. System attaches the evaluation result to the article record. 5. System exposes extension point "RevisionRequested" to the Editor if the recommendation is "Reject." 6. System exposes extension point "OverrideRequested" to the Editor for any evaluation result. 7. System notifies the Editor that evaluation is complete. 8. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Target status changes to "Revision Requested" - Current submitted version is preserved - Revision notes are recorded - Responsible Writer or Graphics Designer is notified
Postconditions (Verification)	• Target status changes to "Revision Requested" • Current submitted version is preserved • Revision notes are recorded • Responsible Writer or Graphics Designer is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.11-A1 — Verify system behavior for alternate flow: High-Confidence Approval (Generate Article Evaluation)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.11-A1
Related Use Case	UC-1.11 — Generate Article Evaluation
Requirement Traceability	PR-EDT-11
Test Objective	Confirm the system correctly handles the alternate scenario "High-Confidence Approval" defined in Alternate Flow A1 of "Generate Article Evaluation", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Editor is authenticated and logged in • Target article or magazine design exists and is in a reviewable state • Editor has permission to request revision • Condition under test: The condition "High-Confidence Approval" occurs for the article evaluation during the "Generate Article Evaluation" workflow.
Test Data	Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests) account (as required by the scenario); a article evaluation configured so the "High-Confidence Approval" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a article evaluation / environment state that will trigger the "High-Confidence Approval" condition. 2. As Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests), perform the relevant Basic Flow action(s) on the article evaluation so that the "High-Confidence Approval" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "High-Confidence Approval" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.

TEST CASE ELEMENT	SPECIFICATION
Postconditions (Verification)	• Target status changes to "Revision Requested" • Current submitted version is preserved • Revision notes are recorded • Responsible Writer or Graphics Designer is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-1.11-E1 — Verify system behavior for exception flow: Ai Service Timeout (Generate Article Evaluation)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-1.11-E1
Related Use Case	UC-1.11 — Generate Article Evaluation
Requirement Traceability	PR-EDT-11
Test Objective	Confirm the system correctly handles the exception scenario "Ai Service Timeout" defined in Exception Flow E1 of "Generate Article Evaluation", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Editor is authenticated and logged in • Target article or magazine design exists and is in a reviewable state • Editor has permission to request revision • Condition under test: The condition "Ai Service Timeout" occurs for the article evaluation during the "Generate Article Evaluation" workflow.
Test Data	Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests) account (as required by the scenario); a article evaluation configured so the "Ai Service Timeout" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a article evaluation / environment state that will trigger the "Ai Service Timeout" condition. 2. As Editor / Writer (for article revision requests) / Graphics Designer (for magazine-design revision requests), perform the relevant Basic Flow action(s) on the article evaluation so that the "Ai Service Timeout" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules.

TEST CASE ELEMENT	SPECIFICATION
	5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Ai Service Timeout" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Target status changes to "Revision Requested" • Current submitted version is preserved • Revision notes are recorded • Responsible Writer or Graphics Designer is notified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

3. Module 2: Manage Publication

UC-2.1: Configure Issue Formats — Test Suite

Use case purpose: Allow the Publisher to define and configure the output formats (digital layout templates, PDF/e-reader formats) available for a publication issue.

Actors: Publisher **Requirement:** PR-PUB-01 **Priority:** Medium

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-2.1.

Test Case ID	Scenario	Type	Priority
TC-2.1-BF	Verify successful configure issue formats (Basic Flow)	Positive	Medium
TC-2.1-A1	Verify system behavior for alternate flow: Duplicate Format (Configure Issue Formats)	Negative / Alternate	Medium
TC-2.1-E1	Verify system behavior for exception flow: Invalid Parameters (Configure Issue Formats)	Negative / Exception	Medium

TC-2.1-BF — Verify successful configure issue formats (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.1-BF
Related Use Case	UC-2.1 — Configure Issue Formats
Requirement Traceability	PR-PUB-01
Test Objective	Confirm that Publisher can successfully complete the "Configure Issue Formats" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow the Publisher to define and configure the output formats (digital layout templates, PDF/e-reader formats) available for a publication issue.
Priority	Medium
Test Type	Positive
Preconditions	• System is online • Publisher is authenticated and logged in
Test Data	Valid, active Publisher account with the required role/permission; a issue format configuration in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Publisher navigates to the Manage Publication module. 2. Publisher selects “Configure Issue Formats.” 3. System displays existing format templates (e.g., Web, PDF, ePub). 4. Publisher creates a new format or edits an existing one (dimensions, cover template, file type). 5. Publisher saves the configuration. 6. System validates and stores the format settings. 7. Verify each postcondition listed below is met.

TEST CASE ELEMENT	SPECIFICATION
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Issue format configuration is saved and available for scheduling
Postconditions (Verification)	• Issue format configuration is saved and available for scheduling
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.1-A1 — Verify system behavior for alternate flow: Duplicate Format (Configure Issue Formats)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.1-A1
Related Use Case	UC-2.1 — Configure Issue Formats
Requirement Traceability	PR-PUB-01
Test Objective	Confirm the system correctly handles the alternate scenario "Duplicate Format" defined in Alternate Flow A1 of "Configure Issue Formats", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	• System is online • Publisher is authenticated and logged in • Condition under test: The condition "Duplicate Format" occurs for the issue format configuration during the "Configure Issue Formats" workflow.
Test Data	Publisher account (as required by the scenario); a issue format configuration configured so the "Duplicate Format" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a issue format configuration / environment state that will trigger the "Duplicate Format" condition. 2. As Publisher, perform the relevant Basic Flow action(s) on the issue format configuration so that the "Duplicate Format" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it.

TEST CASE ELEMENT	SPECIFICATION
	8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Duplicate Format" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Issue format configuration is saved and available for scheduling
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.1-E1 — Verify system behavior for exception flow: Invalid Parameters (Configure Issue Formats)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.1-E1
Related Use Case	UC-2.1 — Configure Issue Formats
Requirement Traceability	PR-PUB-01
Test Objective	Confirm the system correctly handles the exception scenario "Invalid Parameters" defined in Exception Flow E1 of "Configure Issue Formats", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • Publisher is authenticated and logged in • Condition under test: The condition "Invalid Parameters" occurs for the issue format configuration during the "Configure Issue Formats" workflow.
Test Data	Publisher account (as required by the scenario); a issue format configuration configured so the "Invalid Parameters" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a issue format configuration / environment state that will trigger the "Invalid Parameters" condition. 2. As Publisher, perform the relevant Basic Flow action(s) on the issue format configuration so that the "Invalid Parameters" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state.

TEST CASE ELEMENT	SPECIFICATION
	6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Invalid Parameters" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	Issue format configuration is saved and available for scheduling
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-2.2: Schedule Issue Release — Test Suite

Use case purpose: Allow the Publisher to set a future release date and time for an upcoming issue.

Actors: Publisher **Requirement:** PR-PUB-02 **Priority:** High

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-2.2.

Test Case ID	Scenario	Type	Priority
TC-2.2-BF	Verify successful schedule issue release (Basic Flow)	Positive	High
TC-2.2-A1	Verify system behavior for alternate flow: Reschedule (Schedule Issue Release)	Negative / Alternate	High
TC-2.2-E1	Verify system behavior for exception flow: Invalid Date (Schedule Issue Release)	Negative / Exception	High
TC-2.2-E-AUTH	Verify access is denied when session/authorization is invalid (Schedule Issue Release)	Negative / Exception	High

TC-2.2-BF — Verify successful schedule issue release (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.2-BF
Related Use Case	UC-2.2 — Schedule Issue Release
Requirement Traceability	PR-PUB-02
Test Objective	Confirm that Publisher can successfully complete the "Schedule Issue Release" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow the Publisher to set a future release date and time for an upcoming issue.

TEST CASE ELEMENT	SPECIFICATION
Priority	High
Test Type	Positive
Preconditions	• System is online • Issue content (approved bundles) is ready • Issue format is configured
Test Data	Valid, active Publisher account with the required role/permission; a issue release schedule in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Publisher selects an issue that is ready for release. 2. Publisher selects “Schedule Issue Release.” 3. System displays a calendar/time picker. 4. Publisher selects the release date and time. 5. Publisher confirms. 6. System sets the issue status to “Scheduled” and stores the release timestamp. 7. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Issue is queued with a scheduled release date/time
Postconditions (Verification)	• Issue is queued with a scheduled release date/time
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.2-A1 — Verify system behavior for alternate flow: Reschedule (Schedule Issue Release)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.2-A1
Related Use Case	UC-2.2 — Schedule Issue Release
Requirement Traceability	PR-PUB-02
Test Objective	Confirm the system correctly handles the alternate scenario "Reschedule" defined in Alternate Flow A1 of "Schedule Issue Release", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Issue content (approved bundles) is ready • Issue format is configured

TEST CASE ELEMENT	SPECIFICATION
	Condition under test: The condition "Reschedule" occurs for the issue release schedule during the "Schedule Issue Release" workflow.
Test Data	Publisher account (as required by the scenario); a issue release schedule configured so the "Reschedule" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a issue release schedule / environment state that will trigger the "Reschedule" condition. 2. As Publisher, perform the relevant Basic Flow action(s) on the issue release schedule so that the "Reschedule" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Reschedule" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	Issue is queued with a scheduled release date/time
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.2-E1 — Verify system behavior for exception flow: Invalid Date (Schedule Issue Release)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.2-E1
Related Use Case	UC-2.2 — Schedule Issue Release
Requirement Traceability	PR-PUB-02
Test Objective	Confirm the system correctly handles the exception scenario "Invalid Date" defined in Exception Flow E1 of "Schedule Issue Release", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception

TEST CASE ELEMENT	SPECIFICATION
Preconditions	• System is online • Issue content (approved bundles) is ready • Issue format is configured • Condition under test: The condition "Invalid Date" occurs for the issue release schedule during the "Schedule Issue Release" workflow.
Test Data	Publisher account (as required by the scenario); a issue release schedule configured so the "Invalid Date" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a issue release schedule / environment state that will trigger the "Invalid Date" condition. 2. As Publisher, perform the relevant Basic Flow action(s) on the issue release schedule so that the "Invalid Date" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Invalid Date" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Issue is queued with a scheduled release date/time
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.2-E-AUTH — Verify access is denied when session/authorization is invalid (Schedule Issue Release)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.2-E-AUTH
Related Use Case	UC-2.2 — Schedule Issue Release
Requirement Traceability	PR-PUB-02

TEST CASE ELEMENT	SPECIFICATION
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Schedule Issue Release", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Issue content (approved bundles) is ready • Issue format is configured • Condition under test: Publisher's session has expired, or Publisher lacks the role/ownership permission required to perform "Schedule Issue Release", immediately before the state-changing step.
Test Data	Publisher account (as required by the scenario); a issue release schedule configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Publisher. 2. Attempt to initiate the protected "Schedule Issue Release" action (issue release schedule). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the issue release schedule. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Issue is queued with a scheduled release date/time
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-2.3: Execute Live Publishing — Test Suite

Use case purpose: Allow the Publisher to manually trigger immediate publishing of an issue, bypassing or overriding a schedule.

Actors: Publisher **Requirement:** PR-PUB-03 **Priority:** High

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-2.3.

Test Case ID	Scenario	Type	Priority
TC-2.3-BF	Verify successful execute live publishing (Basic Flow)	Positive	High
TC-2.3-A1	Verify system behavior for alternate flow: Alternate Path (Execute Live Publishing)	Negative / Alternate	High
TC-2.3-E1	Verify system behavior for exception flow: Publishing Failure (Execute Live Publishing)	Negative / Exception	High
TC-2.3-E-AUTH	Verify access is denied when session/authorization is invalid (Execute Live Publishing)	Negative / Exception	High

TC-2.3-BF — Verify successful execute live publishing (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.3-BF
Related Use Case	UC-2.3 — Execute Live Publishing
Requirement Traceability	PR-PUB-03
Test Objective	Confirm that Publisher can successfully complete the "Execute Live Publishing" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow the Publisher to manually trigger immediate publishing of an issue, bypassing or overriding a schedule.
Priority	High
Test Type	Positive
Preconditions	• System is online • Issue is fully approved and ready
Test Data	Valid, active Publisher account with the required role/permission; a live publishing action in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Publisher opens a ready issue. 2. Publisher selects "Execute Live Publishing." 3. System prompts for confirmation. 4. Publisher confirms. 5. System publishes the issue immediately, updates status to "Published," and makes it live on the platform. At this point, the optional Trigger Subscriber Notifications extension (UC-2.3.1) may fire. 6. System logs the publishing event. 7. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Issue is published live and accessible per its access tier
Postconditions (Verification)	• Issue is published live and accessible per its access tier
Actual Result	<i>To be recorded during test execution.</i>

TEST CASE ELEMENT	SPECIFICATION
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.3-A1 — Verify system behavior for alternate flow: Alternate Path (Execute Live Publishing)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.3-A1
Related Use Case	UC-2.3 — Execute Live Publishing
Requirement Traceability	PR-PUB-03
Test Objective	Confirm the system correctly handles the alternate scenario "Alternate Path" defined in Alternate Flow A1 of "Execute Live Publishing", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Issue is fully approved and ready • Condition under test: The condition "Alternate Path" occurs for the live publishing action during the "Execute Live Publishing" workflow.
Test Data	Publisher account (as required by the scenario); a live publishing action configured so the "Alternate Path" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a live publishing action / environment state that will trigger the "Alternate Path" condition. 2. As Publisher, perform the relevant Basic Flow action(s) on the live publishing action so that the "Alternate Path" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Alternate Path" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Issue is published live and accessible per its access tier

TEST CASE ELEMENT	SPECIFICATION
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.3-E1 — Verify system behavior for exception flow: Publishing Failure (Execute Live Publishing)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.3-E1
Related Use Case	UC-2.3 — Execute Live Publishing
Requirement Traceability	PR-PUB-03
Test Objective	Confirm the system correctly handles the exception scenario "Publishing Failure" defined in Exception Flow E1 of "Execute Live Publishing", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Issue is fully approved and ready • Condition under test: The condition "Publishing Failure" occurs for the live publishing action during the "Execute Live Publishing" workflow.
Test Data	Publisher account (as required by the scenario); a live publishing action configured so the "Publishing Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a live publishing action / environment state that will trigger the "Publishing Failure" condition. 2. As Publisher, perform the relevant Basic Flow action(s) on the live publishing action so that the "Publishing Failure" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Publishing Failure" condition, prevents any invalid or partial state change, clearly communicates the

TEST CASE ELEMENT	SPECIFICATION
	failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Issue is published live and accessible per its access tier
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.3-E-AUTH — Verify access is denied when session/authorization is invalid (Execute Live Publishing)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.3-E-AUTH
Related Use Case	UC-2.3 — Execute Live Publishing
Requirement Traceability	PR-PUB-03
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Execute Live Publishing", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Issue is fully approved and ready • Condition under test: Publisher's session has expired, or Publisher lacks the role/ownership permission required to perform "Execute Live Publishing", immediately before the state-changing step.
Test Data	Publisher account (as required by the scenario); a live publishing action configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Publisher. 2. Attempt to initiate the protected "Execute Live Publishing" action (live publishing action). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the live publishing action. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message.

TEST CASE ELEMENT	SPECIFICATION
	7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Issue is published live and accessible per its access tier
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-2.3.1: Trigger Subscriber Notifications — Test Suite

Use case purpose: Allow the Publisher to send notifications to subscribers and readers about a new issue or article, as an optional extension of the live publishing process.

Actors: Publisher **Requirement:** PR-PUB-04 **Priority:** Medium

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-2.3.1.

Test Case ID	Scenario	Type	Priority
TC-2.3.1-BF	Verify successful trigger subscriber notifications (Basic Flow)	Positive	Medium
TC-2.3.1-A1	Verify system behavior for alternate flow: Schedule Notification (Trigger Subscriber Notifications)	Negative / Alternate	Medium
TC-2.3.1-E1	Verify system behavior for exception flow: Delivery Failure (Trigger Subscriber Notifications)	Negative / Exception	Medium
TC-2.3.1-E2	Verify system behavior for exception flow: Extension Declined (Trigger Subscriber Notifications)	Negative / Exception	Medium

TC-2.3.1-BF — Verify successful trigger subscriber notifications (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.3.1-BF
Related Use Case	UC-2.3.1 — Trigger Subscriber Notifications
Requirement Traceability	PR-PUB-04
Test Objective	Confirm that Publisher can successfully complete the "Trigger Subscriber Notifications" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow the Publisher to send notifications to subscribers and readers about a new issue or article, as an optional extension of the live publishing process.
Priority	Medium

TEST CASE ELEMENT	SPECIFICATION
Test Type	Positive
Preconditions	• System is online • Execute Live Publishing (UC-2.3) has reached the point where the issue is published or is about to be published
Test Data	Valid, active Publisher account with the required role/permission; a subscriber notification in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Immediately after Execute Live Publishing (UC-2.3) publishes the issue, System offers the Publisher the option to trigger subscriber notifications. 2. Publisher elects to proceed with the extension. 3. System displays notification channel options (email, push, in-app). 4. Publisher customizes the notification message and target segment. 5. Publisher confirms send. 6. System dispatches the notifications, logs delivery statistics, and returns control to Execute Live Publishing (UC-2.3). 7. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Notification is dispatched to the relevant subscriber segment
Postconditions (Verification)	• Notification is dispatched to the relevant subscriber segment
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.3.1-A1 — Verify system behavior for alternate flow: Schedule Notification (Trigger Subscriber Notifications)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.3.1-A1
Related Use Case	UC-2.3.1 — Trigger Subscriber Notifications
Requirement Traceability	PR-PUB-04
Test Objective	Confirm the system correctly handles the alternate scenario "Schedule Notification" defined in Alternate Flow A1 of "Trigger Subscriber Notifications", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	• System is online • Execute Live Publishing (UC-2.3) has reached the point where the issue is published or is about to be published

TEST CASE ELEMENT	SPECIFICATION
	Condition under test: The condition "Schedule Notification" occurs for the subscriber notification during the "Trigger Subscriber Notifications" workflow.
Test Data	Publisher account (as required by the scenario); a subscriber notification configured so the "Schedule Notification" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a subscriber notification / environment state that will trigger the "Schedule Notification" condition. 2. As Publisher, perform the relevant Basic Flow action(s) on the subscriber notification so that the "Schedule Notification" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Schedule Notification" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	Notification is dispatched to the relevant subscriber segment
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.3.1-E1 — Verify system behavior for exception flow: Delivery Failure (Trigger Subscriber Notifications)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.3.1-E1
Related Use Case	UC-2.3.1 — Trigger Subscriber Notifications
Requirement Traceability	PR-PUB-04
Test Objective	Confirm the system correctly handles the exception scenario "Delivery Failure" defined in Exception Flow E1 of "Trigger Subscriber Notifications", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception

TEST CASE ELEMENT	SPECIFICATION
Preconditions	• System is online • Execute Live Publishing (UC-2.3) has reached the point where the issue is published or is about to be published • Condition under test: The condition "Delivery Failure" occurs for the subscriber notification during the "Trigger Subscriber Notifications" workflow.
Test Data	Publisher account (as required by the scenario); a subscriber notification configured so the "Delivery Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a subscriber notification / environment state that will trigger the "Delivery Failure" condition. 2. As Publisher, perform the relevant Basic Flow action(s) on the subscriber notification so that the "Delivery Failure" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Delivery Failure" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Notification is dispatched to the relevant subscriber segment
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.3.1-E2 — Verify system behavior for exception flow: Extension Declined (Trigger Subscriber Notifications)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.3.1-E2
Related Use Case	UC-2.3.1 — Trigger Subscriber Notifications
Requirement Traceability	PR-PUB-04

TEST CASE ELEMENT	SPECIFICATION
Test Objective	Confirm the system correctly handles the exception scenario "Extension Declined" defined in Exception Flow E2 of "Trigger Subscriber Notifications", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • Execute Live Publishing (UC-2.3) has reached the point where the issue is published or is about to be published • Condition under test: The condition "Extension Declined" occurs for the subscriber notification during the "Trigger Subscriber Notifications" workflow.
Test Data	Publisher account (as required by the scenario); a subscriber notification configured so the "Extension Declined" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a subscriber notification / environment state that will trigger the "Extension Declined" condition. 2. As Publisher, perform the relevant Basic Flow action(s) on the subscriber notification so that the "Extension Declined" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E2: it detects the "Extension Declined" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Notification is dispatched to the relevant subscriber segment
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-2.5: Archive Previous Issue — Test Suite

Use case purpose: Allow the Publisher to move an older, already-published issue into the archive for historical access.

Actors: Publisher **Requirement:** PR-PUB-05 **Priority:** Low

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-2.5.

Test Case ID	Scenario	Type	Priority
TC-2.5-BF	Verify successful archive previous issue (Basic Flow)	Positive	Low
TC-2.5-A1	Verify system behavior for alternate flow: Bulk Archive (Archive Previous Issue)	Negative / Alternate	Low
TC-2.5-E1	Verify system behavior for exception flow: Active Issue Protection (Archive Previous Issue)	Negative / Exception	Low
TC-2.5-E-AUTH	Verify access is denied when session/authorization is invalid (Archive Previous Issue)	Negative / Exception	Low

TC-2.5-BF — Verify successful archive previous issue (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.5-BF
Related Use Case	UC-2.5 — Archive Previous Issue
Requirement Traceability	PR-PUB-05
Test Objective	Confirm that Publisher can successfully complete the "Archive Previous Issue" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow the Publisher to move an older, already-published issue into the archive for historical access.
Priority	Low
Test Type	Positive
Preconditions	• System is online • Issue has been published and is no longer the current issue
Test Data	Valid, active Publisher account with the required role/permission; a archived issue in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Publisher navigates to the issue list. 2. Publisher selects an old issue. 3. Publisher selects “Archive Previous Issue.” 4. System prompts for confirmation. 5. Publisher confirms. 6. System updates the issue status to “Archived” and relocates it to the archive section. 7. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Issue is flagged as archived and relocated to the archive listing
Postconditions (Verification)	• Issue is flagged as archived and relocated to the archive listing
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.5-A1 — Verify system behavior for alternate flow: Bulk Archive (Archive Previous Issue)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.5-A1
Related Use Case	UC-2.5 — Archive Previous Issue
Requirement Traceability	PR-PUB-05
Test Objective	Confirm the system correctly handles the alternate scenario "Bulk Archive" defined in Alternate Flow A1 of "Archive Previous Issue", protecting data integrity and informing the user appropriately.
Priority	Low
Test Type	Negative / Alternate
Preconditions	• System is online • Issue has been published and is no longer the current issue • Condition under test: The condition "Bulk Archive" occurs for the archived issue during the "Archive Previous Issue" workflow.
Test Data	Publisher account (as required by the scenario); a archived issue configured so the "Bulk Archive" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a archived issue / environment state that will trigger the "Bulk Archive" condition. 2. As Publisher, perform the relevant Basic Flow action(s) on the archived issue so that the "Bulk Archive" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Bulk Archive" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Issue is flagged as archived and relocated to the archive listing
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.5-E1 — Verify system behavior for exception flow: Active Issue Protection (Archive Previous Issue)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.5-E1
Related Use Case	UC-2.5 — Archive Previous Issue
Requirement Traceability	PR-PUB-05
Test Objective	Confirm the system correctly handles the exception scenario "Active Issue Protection" defined in Exception Flow E1 of "Archive Previous Issue", protecting data integrity and informing the user appropriately.
Priority	Low
Test Type	Negative / Exception
Preconditions	• System is online • Issue has been published and is no longer the current issue • Condition under test: The condition "Active Issue Protection" occurs for the archived issue during the "Archive Previous Issue" workflow.
Test Data	Publisher account (as required by the scenario); a archived issue configured so the "Active Issue Protection" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a archived issue / environment state that will trigger the "Active Issue Protection" condition. 2. As Publisher, perform the relevant Basic Flow action(s) on the archived issue so that the "Active Issue Protection" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Active Issue Protection" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Issue is flagged as archived and relocated to the archive listing
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-2.5-E-AUTH — Verify access is denied when session/authorization is invalid (Archive Previous Issue)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-2.5-E-AUTH
Related Use Case	UC-2.5 — Archive Previous Issue
Requirement Traceability	PR-PUB-05
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Archive Previous Issue", protecting data integrity and informing the user appropriately.
Priority	Low
Test Type	Negative / Exception
Preconditions	• System is online • Issue has been published and is no longer the current issue • Condition under test: Publisher's session has expired, or Publisher lacks the role/ownership permission required to perform "Archive Previous Issue", immediately before the state-changing step.
Test Data	Publisher account (as required by the scenario); a archived issue configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Publisher. 2. Attempt to initiate the protected "Archive Previous Issue" action (archived issue). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the archived issue. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Issue is flagged as archived and relocated to the archive listing
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

4. Module 3: Manage Reports

UC-3.1: Generate Revenue Report — Test Suite

Use case purpose: Allow Management Staff to generate a report summarizing subscription and payment revenue over a selected period.

Actors: Management Staff (Editor, Admin, Publisher) **Requirement:** PR-RPT-01 **Priority:** High

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-3.1.

Test Case ID	Scenario	Type	Priority
TC-3.1-BF	Verify successful generate revenue report (Basic Flow)	Positive	High
TC-3.1-A1	Verify system behavior for alternate flow: Recurring Report (Generate Revenue Report)	Negative / Alternate	High
TC-3.1-E1	Verify system behavior for exception flow: No Data Found (Generate Revenue Report)	Negative / Exception	High

TC-3.1-BF — Verify successful generate revenue report (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-3.1-BF
Related Use Case	UC-3.1 — Generate Revenue Report
Requirement Traceability	PR-RPT-01
Test Objective	Confirm that Management Staff (Editor, Admin, Publisher) can successfully complete the "Generate Revenue Report" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow Management Staff to generate a report summarizing subscription and payment revenue over a selected period.
Priority	High
Test Type	Positive
Preconditions	• System is online • User is authenticated as Editor, Admin, or Publisher • Payment data exists for the requested period
Test Data	Valid, active Management Staff (Editor, Admin, Publisher) account with the required role/permission; a revenue report in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Management Staff navigates to the Manage Reports module. 2. Management Staff selects "Generate Subscription Revenue Report." 3. System prompts for a date range and filters (plan type, region). 4. User submits the parameters. 5. System compiles revenue data from payment records. 6. System displays the report on screen; Management Staff may optionally trigger the Export Report Data extension (UC-3.1.1) to download the report as PDF or CSV.

TEST CASE ELEMENT	SPECIFICATION
	7. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Report is generated and available for viewing/export
Postconditions (Verification)	• Report is generated and available for viewing/export
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-3.1-A1 — Verify system behavior for alternate flow: Recurring Report (Generate Revenue Report)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-3.1-A1
Related Use Case	UC-3.1 — Generate Revenue Report
Requirement Traceability	PR-RPT-01
Test Objective	Confirm the system correctly handles the alternate scenario "Recurring Report" defined in Alternate Flow A1 of "Generate Revenue Report", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • User is authenticated as Editor, Admin, or Publisher • Payment data exists for the requested period • Condition under test: The condition "Recurring Report" occurs for the revenue report during the "Generate Revenue Report" workflow.
Test Data	Management Staff (Editor, Admin, Publisher) account (as required by the scenario); a revenue report configured so the "Recurring Report" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a revenue report / environment state that will trigger the "Recurring Report" condition. 2. As Management Staff (Editor, Admin, Publisher), perform the relevant Basic Flow action(s) on the revenue report so that the "Recurring Report" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules.

TEST CASE ELEMENT	SPECIFICATION
	5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Recurring Report" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Report is generated and available for viewing/export
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-3.1-E1 — Verify system behavior for exception flow: No Data Found (Generate Revenue Report)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-3.1-E1
Related Use Case	UC-3.1 — Generate Revenue Report
Requirement Traceability	PR-RPT-01
Test Objective	Confirm the system correctly handles the exception scenario "No Data Found" defined in Exception Flow E1 of "Generate Revenue Report", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • User is authenticated as Editor, Admin, or Publisher • Payment data exists for the requested period • Condition under test: The condition "No Data Found" occurs for the revenue report during the "Generate Revenue Report" workflow.
Test Data	Management Staff (Editor, Admin, Publisher) account (as required by the scenario); a revenue report configured so the "No Data Found" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a revenue report / environment state that will trigger the "No Data Found" condition.

TEST CASE ELEMENT	SPECIFICATION
	2. As Management Staff (Editor, Admin, Publisher), perform the relevant Basic Flow action(s) on the revenue report so that the "No Data Found" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "No Data Found" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	Report is generated and available for viewing/export
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-3.1.1: Export Report Data — Test Suite

Use case purpose: Serve as a shared extension that converts the currently displayed report's on-screen data into a downloadable PDF or CSV file.

Actors: Management Staff (Editor, Admin, Publisher) **Requirement:** PR-RPT-04 **Priority:** Medium

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-3.1.1.

Test Case ID	Scenario	Type	Priority
TC-3.1.1-BF	Verify successful export report data (Basic Flow)	Positive	Medium
TC-3.1.1-A1	Verify system behavior for alternate flow: Default Format Download (Export Report Data)	Negative / Alternate	Medium
TC-3.1.1-E1	Verify system behavior for exception flow: Export Failure (Export Report Data)	Negative / Exception	Medium

TC-3.1.1-BF — Verify successful export report data (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-3.1.1-BF
Related Use Case	UC-3.1.1 — Export Report Data

TEST CASE ELEMENT	SPECIFICATION
Requirement Traceability	PR-RPT-04
Test Objective	Confirm that Management Staff (Editor, Admin, Publisher) can successfully complete the "Export Report Data" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Serve as a shared extension that converts the currently displayed report's on-screen data into a downloadable PDF or CSV file.
Priority	Medium
Test Type	Positive
Preconditions	• System is online • A report has been generated and is currently displayed by UC-3.1, UC-3.2, or UC-3.3
Test Data	Valid, active Management Staff (Editor, Admin, Publisher) account with the required role/permission; a exported report file in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. While viewing a generated report, Management Staff selects "Export." 2. System displays the available export formats (PDF, CSV). 3. Management Staff selects a format. 4. System converts the on-screen report data into the selected file format. 5. System generates the downloadable file and returns control to the originating report use case (UC-3.1, UC-3.2, or UC-3.3). 6. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Report data is exported and downloaded in the selected format (PDF or CSV)
Postconditions (Verification)	• Report data is exported and downloaded in the selected format (PDF or CSV)
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-3.1.1-A1 — Verify system behavior for alternate flow: Default Format Download (Export Report Data)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-3.1.1-A1
Related Use Case	UC-3.1.1 — Export Report Data
Requirement Traceability	PR-RPT-04

TEST CASE ELEMENT	SPECIFICATION
Test Objective	Confirm the system correctly handles the alternate scenario "Default Format Download" defined in Alternate Flow A1 of "Export Report Data", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	• System is online • A report has been generated and is currently displayed by UC-3.1, UC-3.2, or UC-3.3 • Condition under test: The condition "Default Format Download" occurs for the exported report file during the "Export Report Data" workflow.
Test Data	Management Staff (Editor, Admin, Publisher) account (as required by the scenario); a exported report file configured so the "Default Format Download" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a exported report file / environment state that will trigger the "Default Format Download" condition. 2. As Management Staff (Editor, Admin, Publisher), perform the relevant Basic Flow action(s) on the exported report file so that the "Default Format Download" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Default Format Download" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Report data is exported and downloaded in the selected format (PDF or CSV)
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-3.1.1-E1 — Verify system behavior for exception flow: Export Failure (Export Report Data)

TEST CASE SPECIFICATION	
Test Case ID	TC-3.1.1-E1
Related Use Case	UC-3.1.1 — Export Report Data
Requirement Traceability	PR-RPT-04
Test Objective	Confirm the system correctly handles the exception scenario "Export Failure" defined in Exception Flow E1 of "Export Report Data", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • A report has been generated and is currently displayed by UC-3.1, UC-3.2, or UC-3.3 • Condition under test: The condition "Export Failure" occurs for the exported report file during the "Export Report Data" workflow.
Test Data	Management Staff (Editor, Admin, Publisher) account (as required by the scenario); a exported report file configured so the "Export Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a exported report file / environment state that will trigger the "Export Failure" condition. 2. As Management Staff (Editor, Admin, Publisher), perform the relevant Basic Flow action(s) on the exported report file so that the "Export Failure" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Export Failure" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Report data is exported and downloaded in the selected format (PDF or CSV)
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-3.2: Generate Editorial Pipeline Report — Test Suite

Use case purpose: Allow Management Staff to generate a report on editorial pipeline throughput, such as articles drafted, submitted, published, and average turnaround time.

Actors: Management Staff (Editor, Admin, Publisher) **Requirement:** PR-RPT-02 **Priority:** Medium

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-3.2.

Test Case ID	Scenario	Type	Priority
TC-3.2-BF	Verify successful generate editorial pipeline report (Basic Flow)	Positive	Medium
TC-3.2-A1	Verify system behavior for alternate flow: Drill-Down (Generate Editorial Pipeline Report)	Negative / Alternate	Medium
TC-3.2-E1	Verify system behavior for exception flow: Generation Timeout (Generate Editorial Pipeline Report)	Negative / Exception	Medium

TC-3.2-BF — Verify successful generate editorial pipeline report (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-3.2-BF
Related Use Case	UC-3.2 — Generate Editorial Pipeline Report
Requirement Traceability	PR-RPT-02
Test Objective	Confirm that Management Staff (Editor, Admin, Publisher) can successfully complete the "Generate Editorial Pipeline Report" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow Management Staff to generate a report on editorial pipeline throughput, such as articles drafted, submitted, published, and average turnaround time.
Priority	Medium
Test Type	Positive
Preconditions	• System is online • Pipeline activity data exists
Test Data	Valid, active Management Staff (Editor, Admin, Publisher) account with the required role/permission; a editorial pipeline report in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Management Staff selects “Generate Editorial Pipeline Report.” 2. System prompts for a date range and scope (team or author). 3. User submits the parameters. 4. System compiles pipeline metrics. 5. System displays the report on screen; Management Staff may optionally trigger the Export Report Data extension (UC-3.1.1) to download the report. 6. Verify each postcondition listed below is met.

TEST CASE ELEMENT	SPECIFICATION
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Report is generated and available for viewing/export
Postconditions (Verification)	• Report is generated and available for viewing/export
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-3.2-A1 — Verify system behavior for alternate flow: Drill-Down (Generate Editorial Pipeline Report)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-3.2-A1
Related Use Case	UC-3.2 — Generate Editorial Pipeline Report
Requirement Traceability	PR-RPT-02
Test Objective	Confirm the system correctly handles the alternate scenario "Drill-Down" defined in Alternate Flow A1 of "Generate Editorial Pipeline Report", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	• System is online • Pipeline activity data exists • Condition under test: The condition "Drill-Down" occurs for the editorial pipeline report during the "Generate Editorial Pipeline Report" workflow.
Test Data	Management Staff (Editor, Admin, Publisher) account (as required by the scenario); a editorial pipeline report configured so the "Drill-Down" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a editorial pipeline report / environment state that will trigger the "Drill-Down" condition. 2. As Management Staff (Editor, Admin, Publisher), perform the relevant Basic Flow action(s) on the editorial pipeline report so that the "Drill-Down" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user.

TEST CASE ELEMENT	SPECIFICATION
	7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Drill-Down" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Report is generated and available for viewing/export
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-3.2-E1 — Verify system behavior for exception flow: Generation Timeout (Generate Editorial Pipeline Report)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-3.2-E1
Related Use Case	UC-3.2 — Generate Editorial Pipeline Report
Requirement Traceability	PR-RPT-02
Test Objective	Confirm the system correctly handles the exception scenario "Generation Timeout" defined in Exception Flow E1 of "Generate Editorial Pipeline Report", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • Pipeline activity data exists • Condition under test: The condition "Generation Timeout" occurs for the editorial pipeline report during the "Generate Editorial Pipeline Report" workflow.
Test Data	Management Staff (Editor, Admin, Publisher) account (as required by the scenario); a editorial pipeline report configured so the "Generation Timeout" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a editorial pipeline report / environment state that will trigger the "Generation Timeout" condition. 2. As Management Staff (Editor, Admin, Publisher), perform the relevant Basic Flow action(s) on the editorial pipeline report so that the "Generation Timeout" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules.

TEST CASE ELEMENT	SPECIFICATION
	5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Generation Timeout" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	Report is generated and available for viewing/export
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-3.3: Generate AI Evaluation Summary — Test Suite

Use case purpose: Allow Management Staff to generate a summary report of AI article evaluations, including scores, override rate, and revision-request rate.

Actors: Management Staff (Editor, Admin, Publisher) **Requirement:** PR-RPT-03 **Priority:** Medium

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-3.3.

Test Case ID	Scenario	Type	Priority
TC-3.3-BF	Verify successful generate ai evaluation summary (Basic Flow)	Positive	Medium
TC-3.3-A1	Verify system behavior for alternate flow: Filter By Staff (Generate AI Evaluation Summary)	Negative / Alternate	Medium
TC-3.3-E1	Verify system behavior for exception flow: Insufficient Data (Generate AI Evaluation Summary)	Negative / Exception	Medium

TC-3.3-BF — Verify successful generate ai evaluation summary (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-3.3-BF
Related Use Case	UC-3.3 — Generate AI Evaluation Summary
Requirement Traceability	PR-RPT-03
Test Objective	Confirm that Management Staff (Editor, Admin, Publisher) can successfully complete the "Generate AI Evaluation Summary" use case end-to-end via the documented Basic

TEST CASE ELEMENT	SPECIFICATION
	Flow, and that the system reaches all specified postconditions. Allow Management Staff to generate a summary report of AI article evaluations, including scores, override rate, and revision-request rate.
Priority	Medium
Test Type	Positive
Preconditions	• System is online • AI evaluations exist for the requested period
Test Data	Valid, active Management Staff (Editor, Admin, Publisher) account with the required role/permission; a AI evaluation summary in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Management Staff selects “Generate AI Evaluation Summary.” 2. System prompts for a date range. 3. User submits the parameters. 4. System aggregates evaluation scores, override counts, and revision-request counts. 5. System displays the summary report on screen; Management Staff may optionally trigger the Export Report Data extension (UC-3.1.1) to download the report. 6. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Report is generated and available for viewing/export
Postconditions (Verification)	• Report is generated and available for viewing/export
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-3.3-A1 — Verify system behavior for alternate flow: Filter By Staff (Generate AI Evaluation Summary)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-3.3-A1
Related Use Case	UC-3.3 — Generate AI Evaluation Summary
Requirement Traceability	PR-RPT-03
Test Objective	Confirm the system correctly handles the alternate scenario "Filter By Staff" defined in Alternate Flow A1 of "Generate AI Evaluation Summary", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate

TEST CASE ELEMENT	SPECIFICATION
Preconditions	• System is online • AI evaluations exist for the requested period • Condition under test: The condition "Filter By Staff" occurs for the AI evaluation summary during the "Generate AI Evaluation Summary" workflow.
Test Data	Management Staff (Editor, Admin, Publisher) account (as required by the scenario); a AI evaluation summary configured so the "Filter By Staff" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a AI evaluation summary / environment state that will trigger the "Filter By Staff" condition. 2. As Management Staff (Editor, Admin, Publisher), perform the relevant Basic Flow action(s) on the AI evaluation summary so that the "Filter By Staff" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Filter By Staff" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Report is generated and available for viewing/export
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-3.3-E1 — Verify system behavior for exception flow: Insufficient Data (Generate AI Evaluation Summary)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-3.3-E1
Related Use Case	UC-3.3 — Generate AI Evaluation Summary
Requirement Traceability	PR-RPT-03

TEST CASE ELEMENT	SPECIFICATION
Test Objective	Confirm the system correctly handles the exception scenario "Insufficient Data" defined in Exception Flow E1 of "Generate AI Evaluation Summary", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • AI evaluations exist for the requested period • Condition under test: The condition "Insufficient Data" occurs for the AI evaluation summary during the "Generate AI Evaluation Summary" workflow.
Test Data	Management Staff (Editor, Admin, Publisher) account (as required by the scenario); a AI evaluation summary configured so the "Insufficient Data" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a AI evaluation summary / environment state that will trigger the "Insufficient Data" condition. 2. As Management Staff (Editor, Admin, Publisher), perform the relevant Basic Flow action(s) on the AI evaluation summary so that the "Insufficient Data" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Insufficient Data" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Report is generated and available for viewing/export
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

5. Module 4: Manage Platform Settings

UC-4.1: Update Site Theme and Layout — Test Suite

Use case purpose: Allow the Admin to customize the platform's visual theme and homepage layout.

Actors: Admin **Requirement:** PR-SET-01 **Priority:** Low

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-4.1.

Test Case ID	Scenario	Type	Priority
TC-4.1-BF	Verify successful update site theme and layout (Basic Flow)	Positive	Low
TC-4.1-A1	Verify system behavior for alternate flow: Revert To Default (Update Site Theme and Layout)	Negative / Alternate	Low
TC-4.1-E1	Verify system behavior for exception flow: Invalid Configuration (Update Site Theme and Layout)	Negative / Exception	Low
TC-4.1-E-AUTH	Verify access is denied when session/authorization is invalid (Update Site Theme and Layout)	Negative / Exception	Low

TC-4.1-BF — Verify successful update site theme and layout (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-4.1-BF
Related Use Case	UC-4.1 — Update Site Theme and Layout
Requirement Traceability	PR-SET-01
Test Objective	Confirm that Admin can successfully complete the "Update Site Theme and Layout" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow the Admin to customize the platform's visual theme and homepage layout.
Priority	Low
Test Type	Positive
Preconditions	• System is online • Admin is authenticated and logged in
Test Data	Valid, active Admin account with the required role/permission; a site theme and layout configuration in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Admin navigates to the Manage Platform Settings module. 2. Admin selects "Update Site Theme and Layout." 3. System displays current theme and layout options. 4. Admin adjusts colors, and section ordering. 5. Admin previews the changes. 6. Admin saves. 7. System applies the changes site-wide.

TEST CASE ELEMENT	SPECIFICATION
	8. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Theme and layout changes are applied site-wide
Postconditions (Verification)	• Theme and layout changes are applied site-wide
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-4.1-A1 — Verify system behavior for alternate flow: Revert To Default (Update Site Theme and Layout)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-4.1-A1
Related Use Case	UC-4.1 — Update Site Theme and Layout
Requirement Traceability	PR-SET-01
Test Objective	Confirm the system correctly handles the alternate scenario "Revert To Default" defined in Alternate Flow A1 of "Update Site Theme and Layout", protecting data integrity and informing the user appropriately.
Priority	Low
Test Type	Negative / Alternate
Preconditions	• System is online • Admin is authenticated and logged in • Condition under test: The condition "Revert To Default" occurs for the site theme and layout configuration during the "Update Site Theme and Layout" workflow.
Test Data	Admin account (as required by the scenario); a site theme and layout configuration configured so the "Revert To Default" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a site theme and layout configuration / environment state that will trigger the "Revert To Default" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the site theme and layout configuration so that the "Revert To Default" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user.

TEST CASE ELEMENT	SPECIFICATION
	7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Revert To Default" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Theme and layout changes are applied site-wide
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-4.1-E1 — Verify system behavior for exception flow: Invalid Configuration (Update Site Theme and Layout)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-4.1-E1
Related Use Case	UC-4.1 — Update Site Theme and Layout
Requirement Traceability	PR-SET-01
Test Objective	Confirm the system correctly handles the exception scenario "Invalid Configuration" defined in Exception Flow E1 of "Update Site Theme and Layout", protecting data integrity and informing the user appropriately.
Priority	Low
Test Type	Negative / Exception
Preconditions	• System is online • Admin is authenticated and logged in • Condition under test: The condition "Invalid Configuration" occurs for the site theme and layout configuration during the "Update Site Theme and Layout" workflow.
Test Data	Admin account (as required by the scenario); a site theme and layout configuration configured so the "Invalid Configuration" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a site theme and layout configuration / environment state that will trigger the "Invalid Configuration" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the site theme and layout configuration so that the "Invalid Configuration" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules.

TEST CASE ELEMENT	SPECIFICATION
	5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Invalid Configuration" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Theme and layout changes are applied site-wide
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-4.1-E-AUTH — Verify access is denied when session/authorization is invalid (Update Site Theme and Layout)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-4.1-E-AUTH
Related Use Case	UC-4.1 — Update Site Theme and Layout
Requirement Traceability	PR-SET-01
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Update Site Theme and Layout", protecting data integrity and informing the user appropriately.
Priority	Low
Test Type	Negative / Exception
Preconditions	• System is online • Admin is authenticated and logged in • Condition under test: Admin's session has expired, or Admin lacks the role/ownership permission required to perform "Update Site Theme and Layout", immediately before the state-changing step.
Test Data	Admin account (as required by the scenario); a site theme and layout configuration configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).

TEST CASE ELEMENT	SPECIFICATION
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Admin. 2. Attempt to initiate the protected "Update Site Theme and Layout" action (site theme and layout configuration). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the site theme and layout configuration. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Theme and layout changes are applied site-wide
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-4.2: Configure System Integrations — Test Suite

Use case purpose: Allow the Admin to manage third-party integrations such as the payment gateway, and the email service.

Actors: Admin **Requirement:** PR-SET-02 **Priority:** High

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-4.2.

Test Case ID	Scenario	Type	Priority
TC-4.2-BF	Verify successful configure system integrations (Basic Flow)	Positive	High
TC-4.2-A1	Verify system behavior for alternate flow: Disable Integration (Configure System Integrations)	Negative / Alternate	High
TC-4.2-E1	Verify system behavior for exception flow: Connection Test Failure (Configure System Integrations)	Negative / Exception	High
TC-4.2-E-AUTH	Verify access is denied when session/authorization is invalid (Configure System Integrations)	Negative / Exception	High

TC-4.2-BF — Verify successful configure system integrations (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-4.2-BF
Related Use Case	UC-4.2 — Configure System Integrations
Requirement Traceability	PR-SET-02
Test Objective	Confirm that Admin can successfully complete the "Configure System Integrations" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow the Admin to manage third-party integrations such as the payment gateway, and the email service.
Priority	High
Test Type	Positive
Preconditions	• System is online • Admin is authenticated and logged in
Test Data	Valid, active Admin account with the required role/permission; a system integration configuration in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Admin selects "Configure System Integrations." 2. System lists available integrations and their connection status. 3. Admin selects an integration to configure. 4. Admin enters or updates the API credentials or settings. 5. Admin tests the connection. 6. Admin saves. 7. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Integration configuration is updated and connection verified
Postconditions (Verification)	• Integration configuration is updated and connection verified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-4.2-A1 — Verify system behavior for alternate flow: Disable Integration (Configure System Integrations)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-4.2-A1
Related Use Case	UC-4.2 — Configure System Integrations
Requirement Traceability	PR-SET-02

TEST CASE ELEMENT	SPECIFICATION
Test Objective	Confirm the system correctly handles the alternate scenario "Disable Integration" defined in Alternate Flow A1 of "Configure System Integrations", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Admin is authenticated and logged in • Condition under test: The condition "Disable Integration" occurs for the system integration configuration during the "Configure System Integrations" workflow.
Test Data	Admin account (as required by the scenario); a system integration configuration configured so the "Disable Integration" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a system integration configuration / environment state that will trigger the "Disable Integration" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the system integration configuration so that the "Disable Integration" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Disable Integration" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Integration configuration is updated and connection verified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-4.2-E1 — Verify system behavior for exception flow: Connection Test Failure (Configure System Integrations)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-4.2-E1
Related Use Case	UC-4.2 — Configure System Integrations

TEST CASE ELEMENT	SPECIFICATION
Requirement Traceability	PR-SET-02
Test Objective	Confirm the system correctly handles the exception scenario "Connection Test Failure" defined in Exception Flow E1 of "Configure System Integrations", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Admin is authenticated and logged in • Condition under test: The condition "Connection Test Failure" occurs for the system integration configuration during the "Configure System Integrations" workflow.
Test Data	Admin account (as required by the scenario); a system integration configuration configured so the "Connection Test Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a system integration configuration / environment state that will trigger the "Connection Test Failure" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the system integration configuration so that the "Connection Test Failure" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Connection Test Failure" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Integration configuration is updated and connection verified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-4.2-E-AUTH — Verify access is denied when session/authorization is invalid (Configure System Integrations)

TEST CASE ELEMENT SPECIFICATION	
Test Case ID	TC-4.2-E-AUTH
Related Use Case	UC-4.2 — Configure System Integrations
Requirement Traceability	PR-SET-02
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Configure System Integrations", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Admin is authenticated and logged in • Condition under test: Admin's session has expired, or Admin lacks the role/ownership permission required to perform "Configure System Integrations", immediately before the state-changing step.
Test Data	Admin account (as required by the scenario); a system integration configuration configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Admin. 2. Attempt to initiate the protected "Configure System Integrations" action (system integration configuration). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the system integration configuration. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Integration configuration is updated and connection verified
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-4.3: Toggle Maintenance Mode — Test Suite

Use case purpose: Allow the Admin to place the platform into maintenance mode, restricting reader access during updates.

Actors: Admin **Requirement:** PR-SET-03 **Priority:** Medium

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-4.3.

Test Case ID	Scenario	Type	Priority
TC-4.3-BF	Verify successful toggle maintenance mode (Basic Flow)	Positive	Medium
TC-4.3-A1	Verify system behavior for alternate flow: Scheduled Maintenance (Toggle Maintenance Mode)	Negative / Alternate	Medium
TC-4.3-E1	Verify system behavior for exception flow: Publishing Conflict (Toggle Maintenance Mode)	Negative / Exception	Medium
TC-4.3-E-AUTH	Verify access is denied when session/authorization is invalid (Toggle Maintenance Mode)	Negative / Exception	Medium

TC-4.3-BF — Verify successful toggle maintenance mode (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-4.3-BF
Related Use Case	UC-4.3 — Toggle Maintenance Mode
Requirement Traceability	PR-SET-03
Test Objective	Confirm that Admin can successfully complete the "Toggle Maintenance Mode" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow the Admin to place the platform into maintenance mode, restricting reader access during updates.
Priority	Medium
Test Type	Positive
Preconditions	• System is online • Admin is authenticated and logged in
Test Data	Valid, active Admin account with the required role/permission; a maintenance mode toggle in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Admin selects "Toggle Maintenance Mode." 2. System prompts for confirmation and an optional custom maintenance message. 3. Admin confirms. 4. System enables maintenance mode, displaying the message to all non-Admin visitors. 5. Admin later disables maintenance mode to restore normal access. 6. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Platform maintenance state is toggled and reflected to visitors

TEST CASE ELEMENT	SPECIFICATION
Postconditions (Verification)	Platform maintenance state is toggled and reflected to visitors
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-4.3-A1 — Verify system behavior for alternate flow: Scheduled Maintenance (Toggle Maintenance Mode)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-4.3-A1
Related Use Case	UC-4.3 — Toggle Maintenance Mode
Requirement Traceability	PR-SET-03
Test Objective	Confirm the system correctly handles the alternate scenario "Scheduled Maintenance" defined in Alternate Flow A1 of "Toggle Maintenance Mode", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	- System is online - Admin is authenticated and logged in - Condition under test: The condition "Scheduled Maintenance" occurs for the maintenance mode toggle during the "Toggle Maintenance Mode" workflow.
Test Data	Admin account (as required by the scenario); a maintenance mode toggle configured so the "Scheduled Maintenance" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a maintenance mode toggle / environment state that will trigger the "Scheduled Maintenance" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the maintenance mode toggle so that the "Scheduled Maintenance" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.

TEST CASE ELEMENT	SPECIFICATION
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Scheduled Maintenance" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	Platform maintenance state is toggled and reflected to visitors
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-4.3-E1 — Verify system behavior for exception flow: Publishing Conflict (Toggle Maintenance Mode)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-4.3-E1
Related Use Case	UC-4.3 — Toggle Maintenance Mode
Requirement Traceability	PR-SET-03
Test Objective	Confirm the system correctly handles the exception scenario "Publishing Conflict" defined in Exception Flow E1 of "Toggle Maintenance Mode", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	- System is online - Admin is authenticated and logged in - Condition under test: The condition "Publishing Conflict" occurs for the maintenance mode toggle during the "Toggle Maintenance Mode" workflow.
Test Data	Admin account (as required by the scenario); a maintenance mode toggle configured so the "Publishing Conflict" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a maintenance mode toggle / environment state that will trigger the "Publishing Conflict" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the maintenance mode toggle so that the "Publishing Conflict" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user.

TEST CASE ELEMENT	SPECIFICATION
	7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Publishing Conflict" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	Platform maintenance state is toggled and reflected to visitors
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-4.3-E-AUTH — Verify access is denied when session/authorization is invalid (Toggle Maintenance Mode)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-4.3-E-AUTH
Related Use Case	UC-4.3 — Toggle Maintenance Mode
Requirement Traceability	PR-SET-03
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Toggle Maintenance Mode", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	- System is online - Admin is authenticated and logged in - Condition under test: Admin's session has expired, or Admin lacks the role/ownership permission required to perform "Toggle Maintenance Mode", immediately before the state-changing step.
Test Data	Admin account (as required by the scenario); a maintenance mode toggle configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Admin. 2. Attempt to initiate the protected "Toggle Maintenance Mode" action (maintenance mode toggle). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes.

TEST CASE ELEMENT	SPECIFICATION
	4. Verify the system blocks the action and does NOT partially commit any change to the maintenance mode toggle. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	Platform maintenance state is toggled and reflected to visitors
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

6. Module 5: Manage User Privileges

UC-5.1: Add New User — Test Suite

Use case purpose: Allow the Admin to manually create a new staff or reader account with an assigned role.

Actors: Admin **Requirement:** PR-USR-01 **Priority:** High

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-5.1.

Test Case ID	Scenario	Type	Priority
TC-5.1-BF	Verify successful add new user (Basic Flow)	Positive	High
TC-5.1-A1	Verify system behavior for alternate flow: Bulk Import (Add New User)	Negative / Alternate	High
TC-5.1-E1	Verify system behavior for exception flow: Duplicate Email (Add New User)	Negative / Exception	High
TC-5.1-E-AUTH	Verify access is denied when session/authorization is invalid (Add New User)	Negative / Exception	High

TC-5.1-BF — Verify successful add new user (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.1-BF
Related Use Case	UC-5.1 — Add New User
Requirement Traceability	PR-USR-01
Test Objective	Confirm that Admin can successfully complete the "Add New User" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow the Admin to manually create a new staff or reader account with an assigned role.
Priority	High
Test Type	Positive
Preconditions	• System is online • Admin is authenticated and logged in
Test Data	Valid, active Admin account with the required role/permission; a new user account in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Admin selects “Add New Staff.” 2. System displays a new-user form. 3. Admin enters the name, email, and assigns a role (Editor, Writer, Publisher, Admin, etc.). 4. Admin submits the form. 5. System creates the account and sends setup credentials to the user’s email. 6. Verify each postcondition listed below is met.

TEST CASE ELEMENT	SPECIFICATION
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - New user account is created with the assigned role
Postconditions (Verification)	• New user account is created with the assigned role
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-5.1-A1 — Verify system behavior for alternate flow: Bulk Import (Add New User)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.1-A1
Related Use Case	UC-5.1 — Add New User
Requirement Traceability	PR-USR-01
Test Objective	Confirm the system correctly handles the alternate scenario "Bulk Import" defined in Alternate Flow A1 of "Add New User", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Admin is authenticated and logged in • Condition under test: The condition "Bulk Import" occurs for the new user account during the "Add New User" workflow.
Test Data	Admin account (as required by the scenario); a new user account configured so the "Bulk Import" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a new user account / environment state that will trigger the "Bulk Import" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the new user account so that the "Bulk Import" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it.

TEST CASE ELEMENT	SPECIFICATION
	8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Bulk Import" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• New user account is created with the assigned role
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-5.1-E1 — Verify system behavior for exception flow: Duplicate Email (Add New User)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.1-E1
Related Use Case	UC-5.1 — Add New User
Requirement Traceability	PR-USR-01
Test Objective	Confirm the system correctly handles the exception scenario "Duplicate Email" defined in Exception Flow E1 of "Add New User", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Admin is authenticated and logged in • Condition under test: The condition "Duplicate Email" occurs for the new user account during the "Add New User" workflow.
Test Data	Admin account (as required by the scenario); a new user account configured so the "Duplicate Email" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a new user account / environment state that will trigger the "Duplicate Email" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the new user account so that the "Duplicate Email" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state.

TEST CASE ELEMENT	SPECIFICATION
	6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Duplicate Email" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• New user account is created with the assigned role
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-5.1-E-AUTH — Verify access is denied when session/authorization is invalid (Add New User)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.1-E-AUTH
Related Use Case	UC-5.1 — Add New User
Requirement Traceability	PR-USR-01
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Add New User", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Admin is authenticated and logged in • Condition under test: Admin's session has expired, or Admin lacks the role/ownership permission required to perform "Add New User", immediately before the state-changing step.
Test Data	Admin account (as required by the scenario); a new user account configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Admin. 2. Attempt to initiate the protected "Add New User" action (new user account). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes.

TEST CASE ELEMENT	SPECIFICATION
	4. Verify the system blocks the action and does NOT partially commit any change to the new user account. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	New user account is created with the assigned role
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-5.2: View User Directory — Test Suite

Use case purpose: Allow the Admin to view a searchable list of all system users along with their roles and status.

Actors: Admin **Requirement:** PR-USR-02 **Priority:** Medium

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-5.2.

Test Case ID	Scenario	Type	Priority
TC-5.2-BF	Verify successful view user directory (Basic Flow)	Positive	Medium
TC-5.2-A1	Verify system behavior for alternate flow: Export Directory (View User Directory)	Negative / Alternate	Medium
TC-5.2-E1	Verify system behavior for exception flow: Load Failure (View User Directory)	Negative / Exception	Medium

TC-5.2-BF — Verify successful view user directory (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.2-BF
Related Use Case	UC-5.2 — View User Directory
Requirement Traceability	PR-USR-02
Test Objective	Confirm that Admin can successfully complete the "View User Directory" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow the Admin to view a searchable list of all system users along with their roles and status.

TEST CASE ELEMENT	SPECIFICATION
Priority	Medium
Test Type	Positive
Preconditions	• System is online • Admin is authenticated and logged in
Test Data	Valid, active Admin account with the required role/permission; a user directory listing in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Admin selects “View User Directory.” 2. System retrieves and displays all user accounts. 3. Admin searches or filters by name, role, or status. From this listing, Admin may select a user record and optionally trigger Update User Role (UC-5.2.1), Suspend User Account (UC-5.2.2), or Force Password Reset (UC-5.2.3). 4. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - User directory is displayed
Postconditions (Verification)	• User directory is displayed
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-5.2-A1 — Verify system behavior for alternate flow: Export Directory (View User Directory)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.2-A1
Related Use Case	UC-5.2 — View User Directory
Requirement Traceability	PR-USR-02
Test Objective	Confirm the system correctly handles the alternate scenario "Export Directory" defined in Alternate Flow A1 of "View User Directory", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	• System is online • Admin is authenticated and logged in • Condition under test: The condition "Export Directory" occurs for the user directory listing during the "View User Directory" workflow.

TEST CASE ELEMENT	SPECIFICATION
Test Data	Admin account (as required by the scenario); a user directory listing configured so the "Export Directory" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a user directory listing / environment state that will trigger the "Export Directory" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the user directory listing so that the "Export Directory" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Export Directory" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• User directory is displayed
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-5.2-E1 — Verify system behavior for exception flow: Load Failure (View User Directory)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.2-E1
Related Use Case	UC-5.2 — View User Directory
Requirement Traceability	PR-USR-02
Test Objective	Confirm the system correctly handles the exception scenario "Load Failure" defined in Exception Flow E1 of "View User Directory", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • Admin is authenticated and logged in

TEST CASE ELEMENT	SPECIFICATION
	Condition under test: The condition "Load Failure" occurs for the user directory listing during the "View User Directory" workflow.
Test Data	Admin account (as required by the scenario); a user directory listing configured so the "Load Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a user directory listing / environment state that will trigger the "Load Failure" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the user directory listing so that the "Load Failure" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Load Failure" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	User directory is displayed
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-5.2.1: Update User Role — Test Suite

Use case purpose: Allow the Admin to change a user's assigned role and permission level.

Actors: Admin **Requirement:** PR-USR-03 **Priority:** High

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-5.2.1.

Test Case ID	Scenario	Type	Priority
TC-5.2.1-BF	Verify successful update user role (Basic Flow)	Positive	High
TC-5.2.1-A1	Verify system behavior for alternate flow: Multiple Roles (Update User Role)	Negative / Alternate	High
TC-5.2.1-E1	Verify system behavior for exception flow: Last Admin Protection (Update User Role)	Negative / Exception	High

TC-5.2.1-BF — Verify successful update user role (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.2.1-BF
Related Use Case	UC-5.2.1 — Update User Role
Requirement Traceability	PR-USR-03
Test Objective	Confirm that Admin can successfully complete the "Update User Role" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow the Admin to change a user's assigned role and permission level.
Priority	High
Test Type	Positive
Preconditions	• System is online • Admin is authenticated and logged in, and is currently viewing the User Directory (UC-5.2) • Target user account exists in the directory listing
Test Data	Valid, active Admin account with the required role/permission; a user role assignment in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. While viewing the User Directory (UC-5.2), Admin selects a target user record and triggers "Update User Role." 2. System displays available roles. 3. Admin selects the new role. 4. Admin confirms. 5. System updates the user's role, applies the permission changes immediately, and returns control to the User Directory (UC-5.2). 6. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - User's role and permissions are updated
Postconditions (Verification)	• User's role and permissions are updated
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-5.2.1-A1 — Verify system behavior for alternate flow: Multiple Roles (Update User Role)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.2.1-A1
Related Use Case	UC-5.2.1 — Update User Role

TEST CASE ELEMENT	SPECIFICATION
Requirement Traceability	PR-USR-03
Test Objective	Confirm the system correctly handles the alternate scenario "Multiple Roles" defined in Alternate Flow A1 of "Update User Role", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Admin is authenticated and logged in, and is currently viewing the User Directory (UC-5.2) • Target user account exists in the directory listing • Condition under test: The condition "Multiple Roles" occurs for the user role assignment during the "Update User Role" workflow.
Test Data	Admin account (as required by the scenario); a user role assignment configured so the "Multiple Roles" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a user role assignment / environment state that will trigger the "Multiple Roles" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the user role assignment so that the "Multiple Roles" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Multiple Roles" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• User's role and permissions are updated
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-5.2.1-E1 — Verify system behavior for exception flow: Last Admin Protection (Update User Role)

TEST CASE SPECIFICATION	
Test Case ID	TC-5.2.1-E1
Related Use Case	UC-5.2.1 — Update User Role
Requirement Traceability	PR-USR-03
Test Objective	Confirm the system correctly handles the exception scenario "Last Admin Protection" defined in Exception Flow E1 of "Update User Role", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Admin is authenticated and logged in, and is currently viewing the User Directory (UC-5.2) • Target user account exists in the directory listing • Condition under test: The condition "Last Admin Protection" occurs for the user role assignment during the "Update User Role" workflow.
Test Data	Admin account (as required by the scenario); a user role assignment configured so the "Last Admin Protection" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a user role assignment / environment state that will trigger the "Last Admin Protection" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the user role assignment so that the "Last Admin Protection" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Last Admin Protection" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• User's role and permissions are updated
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-5.2.2: Suspend User Account — Test Suite

Use case purpose: Allow the Admin to temporarily disable a user's access to the platform.

Actors: Admin **Requirement:** PR-USR-04 **Priority:** High

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-5.2.2.

Test Case ID	Scenario	Type	Priority
TC-5.2.2-BF	Verify successful suspend user account (Basic Flow)	Positive	High
TC-5.2.2-A1	Verify system behavior for alternate flow: Reinstate Account (Suspend User Account)	Negative / Alternate	High
TC-5.2.2-E1	Verify system behavior for exception flow: Self-Suspension Protection (Suspend User Account)	Negative / Exception	High

TC-5.2.2-BF — Verify successful suspend user account (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.2.2-BF
Related Use Case	UC-5.2.2 — Suspend User Account
Requirement Traceability	PR-USR-04
Test Objective	Confirm that Admin can successfully complete the "Suspend User Account" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow the Admin to temporarily disable a user's access to the platform.
Priority	High
Test Type	Positive
Preconditions	• System is online • Admin is authenticated and logged in, and is currently viewing the User Directory (UC-5.2) • Target user account exists in the directory listing
Test Data	Valid, active Admin account with the required role/permission; a user account suspension in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. While viewing the User Directory (UC-5.2), Admin selects a target user record and triggers "Suspend User Account." 2. System prompts for a reason and, optionally, a duration. 3. Admin confirms. 4. System suspends the account, revokes active sessions, and returns control to the User Directory (UC-5.2). 5. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - User account is suspended and active sessions are revoked
Postconditions (Verification)	• User account is suspended and active sessions are revoked

TEST CASE ELEMENT	SPECIFICATION
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-5.2.2-A1 — Verify system behavior for alternate flow: Reinstate Account (Suspend User Account)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.2.2-A1
Related Use Case	UC-5.2.2 — Suspend User Account
Requirement Traceability	PR-USR-04
Test Objective	Confirm the system correctly handles the alternate scenario "Reinstate Account" defined in Alternate Flow A1 of "Suspend User Account", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Admin is authenticated and logged in, and is currently viewing the User Directory (UC-5.2) • Target user account exists in the directory listing • Condition under test: The condition "Reinstate Account" occurs for the user account suspension during the "Suspend User Account" workflow.
Test Data	Admin account (as required by the scenario); a user account suspension configured so the "Reinstate Account" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a user account suspension / environment state that will trigger the "Reinstate Account" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the user account suspension so that the "Reinstate Account" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.

TEST CASE ELEMENT	SPECIFICATION
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Reinstate Account" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• User account is suspended and active sessions are revoked
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-5.2.2-E1 — Verify system behavior for exception flow: Self-Suspension Protection (Suspend User Account)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.2.2-E1
Related Use Case	UC-5.2.2 — Suspend User Account
Requirement Traceability	PR-USR-04
Test Objective	Confirm the system correctly handles the exception scenario "Self-Suspension Protection" defined in Exception Flow E1 of "Suspend User Account", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Admin is authenticated and logged in, and is currently viewing the User Directory (UC-5.2) • Target user account exists in the directory listing • Condition under test: The condition "Self-Suspension Protection" occurs for the user account suspension during the "Suspend User Account" workflow.
Test Data	Admin account (as required by the scenario); a user account suspension configured so the "Self-Suspension Protection" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a user account suspension / environment state that will trigger the "Self-Suspension Protection" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the user account suspension so that the "Self-Suspension Protection" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state.

TEST CASE ELEMENT	SPECIFICATION
	6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Self-Suspension Protection" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	User account is suspended and active sessions are revoked
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-5.2.3: Force Password Reset — Test Suite

Use case purpose: Allow the Admin to require a user to reset their password on next login, for example after a security concern.

Actors: Admin **Requirement:** PR-USR-05 **Priority:** Medium

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-5.2.3.

Test Case ID	Scenario	Type	Priority
TC-5.2.3-BF	Verify successful force password reset (Basic Flow)	Positive	Medium
TC-5.2.3-A1	Verify system behavior for alternate flow: Bulk Reset (Force Password Reset)	Negative / Alternate	Medium
TC-5.2.3-E1	Verify system behavior for exception flow: Undeliverable Email (Force Password Reset)	Negative / Exception	Medium

TC-5.2.3-BF — Verify successful force password reset (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.2.3-BF
Related Use Case	UC-5.2.3 — Force Password Reset
Requirement Traceability	PR-USR-05
Test Objective	Confirm that Admin can successfully complete the "Force Password Reset" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow the Admin to require a user to reset their password on next login, for example after a security concern.
Priority	Medium

TEST CASE ELEMENT	SPECIFICATION
Test Type	Positive
Preconditions	• System is online • Admin is authenticated and logged in, and is currently viewing the User Directory (UC-5.2) • Target user account exists in the directory listing
Test Data	Valid, active Admin account with the required role/permission; a password reset request in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. While viewing the User Directory (UC-5.2), Admin selects a target user record and triggers "Force Password Reset." 2. System asks the Admin to confirm the action. 3. System invalidates the current password and flags the account for mandatory reset. 4. System emails the user a reset link and returns control to the User Directory (UC-5.2). 5. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Account is flagged for mandatory password reset
Postconditions (Verification)	• Account is flagged for mandatory password reset
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-5.2.3-A1 — Verify system behavior for alternate flow: Bulk Reset (Force Password Reset)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.2.3-A1
Related Use Case	UC-5.2.3 — Force Password Reset
Requirement Traceability	PR-USR-05
Test Objective	Confirm the system correctly handles the alternate scenario "Bulk Reset" defined in Alternate Flow A1 of "Force Password Reset", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	• System is online • Admin is authenticated and logged in, and is currently viewing the User Directory (UC-5.2) • Target user account exists in the directory listing

TEST CASE ELEMENT	SPECIFICATION
	Condition under test: The condition "Bulk Reset" occurs for the password reset request during the "Force Password Reset" workflow.
Test Data	Admin account (as required by the scenario); a password reset request configured so the "Bulk Reset" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a password reset request / environment state that will trigger the "Bulk Reset" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the password reset request so that the "Bulk Reset" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Bulk Reset" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	Account is flagged for mandatory password reset
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-5.2.3-E1 — Verify system behavior for exception flow: Undeliverable Email (Force Password Reset)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-5.2.3-E1
Related Use Case	UC-5.2.3 — Force Password Reset
Requirement Traceability	PR-USR-05
Test Objective	Confirm the system correctly handles the exception scenario "Undeliverable Email" defined in Exception Flow E1 of "Force Password Reset", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception

TEST CASE ELEMENT	SPECIFICATION
Preconditions	• System is online • Admin is authenticated and logged in, and is currently viewing the User Directory (UC-5.2) • Target user account exists in the directory listing • Condition under test: The condition "Undeliverable Email" occurs for the password reset request during the "Force Password Reset" workflow.
Test Data	Admin account (as required by the scenario); a password reset request configured so the "Undeliverable Email" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a password reset request / environment state that will trigger the "Undeliverable Email" condition. 2. As Admin, perform the relevant Basic Flow action(s) on the password reset request so that the "Undeliverable Email" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Undeliverable Email" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Account is flagged for mandatory password reset
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

7. Module 6: Manage Account

UC-6.1: Register Account — Test Suite

Use case purpose: Allow an Unregistered Reader to create a new reader account.

Actors: Unregistered Reader **Requirement:** PR-ACC-01 **Priority:** High

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-6.1.

Test Case ID	Scenario	Type	Priority
TC-6.1-BF	Verify successful register account (Basic Flow)	Positive	High
TC-6.1-A1	Verify system behavior for alternate flow: Sso Registration (Register Account)	Negative / Alternate	High
TC-6.1-E1	Verify system behavior for exception flow: Duplicate Email (Register Account)	Negative / Exception	High

TC-6.1-BF — Verify successful register account (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.1-BF
Related Use Case	UC-6.1 — Register Account
Requirement Traceability	PR-ACC-01
Test Objective	Confirm that Unregistered Reader can successfully complete the "Register Account" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow an Unregistered Reader to create a new reader account.
Priority	High
Test Type	Positive
Preconditions	• System is online • Unregistered Reader has navigated to the application's URL
Test Data	Valid, active Unregistered Reader account with the required role/permission; a account registration in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Unregistered Reader clicks "Log In or Sign Up." 2. System displays the registration form. 3. Reader enters their name, email, and password. 4. Reader submits the form. 5. System validates uniqueness of the email and creates the account. 6. System sends a verification email. 7. Reader verifies the email to activate the account. 8. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - A new Registered Reader account is created and able to log in

TEST CASE ELEMENT	SPECIFICATION
Postconditions (Verification)	A new Registered Reader account is created and able to log in
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.1-A1 — Verify system behavior for alternate flow: Sso Registration (Register Account)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.1-A1
Related Use Case	UC-6.1 — Register Account
Requirement Traceability	PR-ACC-01
Test Objective	Confirm the system correctly handles the alternate scenario "Sso Registration" defined in Alternate Flow A1 of "Register Account", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	- System is online - Unregistered Reader has navigated to the application's URL - Condition under test: The condition "Sso Registration" occurs for the account registration during the "Register Account" workflow.
Test Data	Unregistered Reader account (as required by the scenario); a account registration configured so the "Sso Registration" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a account registration / environment state that will trigger the "Sso Registration" condition. 2. As Unregistered Reader, perform the relevant Basic Flow action(s) on the account registration so that the "Sso Registration" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.

TEST CASE ELEMENT	SPECIFICATION
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Sso Registration" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• A new Registered Reader account is created and able to log in
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.1-E1 — Verify system behavior for exception flow: Duplicate Email (Register Account)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.1-E1
Related Use Case	UC-6.1 — Register Account
Requirement Traceability	PR-ACC-01
Test Objective	Confirm the system correctly handles the exception scenario "Duplicate Email" defined in Exception Flow E1 of "Register Account", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Unregistered Reader has navigated to the application's URL • Condition under test: The condition "Duplicate Email" occurs for the account registration during the "Register Account" workflow.
Test Data	Unregistered Reader account (as required by the scenario); a account registration configured so the "Duplicate Email" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a account registration / environment state that will trigger the "Duplicate Email" condition. 2. As Unregistered Reader, perform the relevant Basic Flow action(s) on the account registration so that the "Duplicate Email" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it.

TEST CASE ELEMENT	SPECIFICATION
	8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Duplicate Email" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	A new Registered Reader account is created and able to log in
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-6.2: Authenticate User (Log In) — Test Suite

Use case purpose: Allow a registered User, whether a Reader or Staff member, to log in to their account.

Actors: User (Registered Reader, Staff) **Requirement:** PR-ACC-02 **Priority:** High

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-6.2.

Test Case ID	Scenario	Type	Priority
TC-6.2-BF	Verify successful authenticate user (log in) (Basic Flow)	Positive	High
TC-6.2-A1	Verify system behavior for alternate flow: Sso Login (Authenticate User (Log In))	Negative / Alternate	High
TC-6.2-E1	Verify system behavior for exception flow: Invalid Credentials (Authenticate User (Log In))	Negative / Exception	High
TC-6.2-E2	Verify system behavior for exception flow: Suspended Account (Authenticate User (Log In))	Negative / Exception	High

TC-6.2-BF — Verify successful authenticate user (log in) (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.2-BF
Related Use Case	UC-6.2 — Authenticate User (Log In)
Requirement Traceability	PR-ACC-02
Test Objective	Confirm that User (Registered Reader, Staff) can successfully complete the "Authenticate User (Log In)" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow a registered User, whether a Reader or Staff member, to log in to their account.
Priority	High
Test Type	Positive

TEST CASE ELEMENT	SPECIFICATION
Preconditions	• System is online • User has a registered, active account
Test Data	Valid, active User (Registered Reader, Staff) account with the required role/permission; a login session in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. User navigates to the login page. 2. User enters their email/username and password. 3. User submits the form. 4. System validates the credentials. 5. System creates an authenticated session and redirects the user to their dashboard. 6. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - An authenticated session is created for the User
Postconditions (Verification)	• An authenticated session is created for the User
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.2-A1 — Verify system behavior for alternate flow: Sso Login (Authenticate User (Log In))

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.2-A1
Related Use Case	UC-6.2 — Authenticate User (Log In)
Requirement Traceability	PR-ACC-02
Test Objective	Confirm the system correctly handles the alternate scenario "Sso Login" defined in Alternate Flow A1 of "Authenticate User (Log In)", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • User has a registered, active account • Condition under test: The condition "Sso Login" occurs for the login session during the "Authenticate User (Log In)" workflow.
Test Data	User (Registered Reader, Staff) account (as required by the scenario); a login session configured so the "Sso Login" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).

TEST CASE ELEMENT	SPECIFICATION
Test Steps	1. Prepare a login session / environment state that will trigger the "Sso Login" condition. 2. As User (Registered Reader, Staff), perform the relevant Basic Flow action(s) on the login session so that the "Sso Login" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Sso Login" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• An authenticated session is created for the User
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.2-E1 — Verify system behavior for exception flow: Invalid Credentials (Authenticate User (Log In))

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.2-E1
Related Use Case	UC-6.2 — Authenticate User (Log In)
Requirement Traceability	PR-ACC-02
Test Objective	Confirm the system correctly handles the exception scenario "Invalid Credentials" defined in Exception Flow E1 of "Authenticate User (Log In)", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • User has a registered, active account • Condition under test: The condition "Invalid Credentials" occurs for the login session during the "Authenticate User (Log In)" workflow.

TEST CASE ELEMENT	SPECIFICATION
Test Data	User (Registered Reader, Staff) account (as required by the scenario); a login session configured so the "Invalid Credentials" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a login session / environment state that will trigger the "Invalid Credentials" condition. 2. As User (Registered Reader, Staff), perform the relevant Basic Flow action(s) on the login session so that the "Invalid Credentials" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Invalid Credentials" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• An authenticated session is created for the User
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.2-E2 — Verify system behavior for exception flow: Suspended Account (Authenticate User (Log In))

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.2-E2
Related Use Case	UC-6.2 — Authenticate User (Log In)
Requirement Traceability	PR-ACC-02
Test Objective	Confirm the system correctly handles the exception scenario "Suspended Account" defined in Exception Flow E2 of "Authenticate User (Log In)", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online

TEST CASE ELEMENT	SPECIFICATION
	• User has a registered, active account • Condition under test: The condition "Suspended Account" occurs for the login session during the "Authenticate User (Log In)" workflow.
Test Data	User (Registered Reader, Staff) account (as required by the scenario); a login session configured so the "Suspended Account" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a login session / environment state that will trigger the "Suspended Account" condition. 2. As User (Registered Reader, Staff), perform the relevant Basic Flow action(s) on the login session so that the "Suspended Account" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E2: it detects the "Suspended Account" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• An authenticated session is created for the User
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-6.3: Subscribe — Test Suite

Use case purpose: Allow a Registered Reader to purchase a premium subscription plan, upgrading their access to Subscriber status.

Actors: Registered Reader, Payment Gateway (secondary, via include) **Requirement:** PR-ACC-03 **Priority:** High

This suite contains 6 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-6.3.

Test Case ID	Scenario	Type	Priority
TC-6.3-BF	Verify successful subscribe (Basic Flow)	Positive	High
TC-6.3-A1	Verify system behavior for alternate flow: Switch Plan Selection (Subscribe)	Negative / Alternate	High
TC-6.3-A2	Verify system behavior for alternate flow: Existing Active Subscription (Subscribe)	Negative / Alternate	High

Test Case ID	Scenario	Type	Priority
TC-6.3-A3	Verify system behavior for alternate flow: Checkout Cancellation (Subscribe)	Negative / Alternate	High
TC-6.3-E1	Verify system behavior for exception flow: Payment Failure (Subscribe)	Negative / Exception	High
TC-6.3-E-AUTH	Verify access is denied when session/authorization is invalid (Subscribe)	Negative / Exception	High

TC-6.3-BF — Verify successful subscribe (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.3-BF
Related Use Case	UC-6.3 — Subscribe
Requirement Traceability	PR-ACC-03
Test Objective	Confirm that Registered Reader / Payment Gateway (secondary, via include) can successfully complete the "Subscribe" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow a Registered Reader to purchase a premium subscription plan, upgrading their access to Subscriber status.
Priority	High
Test Type	Positive
Preconditions	• System is online • Registered Reader is authenticated and logged in
Test Data	Valid, active Registered Reader / Payment Gateway (secondary, via include) account with the required role/permission; a subscription plan selection in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Registered Reader selects "Subscribe." 2. System displays the available subscription plans. 3. Reader selects a plan. 4. System includes the Manage Payment module to process the transaction (UC-9.1). 5. Upon successful payment, system upgrades the Reader's account to Subscriber status. 6. System confirms the upgrade to the user. 7. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - On successful payment, the Reader's account is upgraded to Subscriber status
Postconditions (Verification)	• On successful payment, the Reader's account is upgraded to Subscriber status
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.3-A1 — Verify system behavior for alternate flow: Switch Plan Selection (Subscribe)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.3-A1
Related Use Case	UC-6.3 — Subscribe
Requirement Traceability	PR-ACC-03
Test Objective	Confirm the system correctly handles the alternate scenario "Switch Plan Selection" defined in Alternate Flow A1 of "Subscribe", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Registered Reader is authenticated and logged in • Condition under test: The condition "Switch Plan Selection" occurs for the subscription plan selection during the "Subscribe" workflow.
Test Data	Registered Reader / Payment Gateway (secondary, via include) account (as required by the scenario); a subscription plan selection configured so the "Switch Plan Selection" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a subscription plan selection / environment state that will trigger the "Switch Plan Selection" condition. 2. As Registered Reader / Payment Gateway (secondary, via include), perform the relevant Basic Flow action(s) on the subscription plan selection so that the "Switch Plan Selection" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Switch Plan Selection" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• On successful payment, the Reader's account is upgraded to Subscriber status
Actual Result	<i>To be recorded during test execution.</i>

TEST CASE ELEMENT	SPECIFICATION
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.3-A2 — Verify system behavior for alternate flow: Existing Active Subscription (Subscribe)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.3-A2
Related Use Case	UC-6.3 — Subscribe
Requirement Traceability	PR-ACC-03
Test Objective	Confirm the system correctly handles the alternate scenario "Existing Active Subscription" defined in Alternate Flow A2 of "Subscribe", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Registered Reader is authenticated and logged in • Condition under test: The condition "Existing Active Subscription" occurs for the subscription plan selection during the "Subscribe" workflow.
Test Data	Registered Reader / Payment Gateway (secondary, via include) account (as required by the scenario); a subscription plan selection configured so the "Existing Active Subscription" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a subscription plan selection / environment state that will trigger the "Existing Active Subscription" condition. 2. As Registered Reader / Payment Gateway (secondary, via include), perform the relevant Basic Flow action(s) on the subscription plan selection so that the "Existing Active Subscription" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A2: it detects and validates the "Existing Active Subscription" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.

TEST CASE ELEMENT	SPECIFICATION
Postconditions (Verification)	On successful payment, the Reader's account is upgraded to Subscriber status
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.3-A3 — Verify system behavior for alternate flow: Checkout Cancellation (Subscribe)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.3-A3
Related Use Case	UC-6.3 — Subscribe
Requirement Traceability	PR-ACC-03
Test Objective	Confirm the system correctly handles the alternate scenario "Checkout Cancellation" defined in Alternate Flow A3 of "Subscribe", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	- System is online - Registered Reader is authenticated and logged in - Condition under test: The condition "Checkout Cancellation" occurs for the subscription plan selection during the "Subscribe" workflow.
Test Data	Registered Reader / Payment Gateway (secondary, via include) account (as required by the scenario); a subscription plan selection configured so the "Checkout Cancellation" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	- Prepare a subscription plan selection / environment state that will trigger the "Checkout Cancellation" condition. - As Registered Reader / Payment Gateway (secondary, via include), perform the relevant Basic Flow action(s) on the subscription plan selection so that the "Checkout Cancellation" condition is encountered. - Verify the system detects the condition at the correct point in the Basic Flow. - Verify the system validates the current record, permission, and applicable business rules. - Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. - Verify the system clearly explains the resulting state and the required next action to the user. - Verify the event is recorded in the audit log where audit/security rules require it. - Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.

TEST CASE ELEMENT	SPECIFICATION
Expected Result	The system correctly follows Alternate Flow A3: it detects and validates the "Checkout Cancellation" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	On successful payment, the Reader's account is upgraded to Subscriber status
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.3-E1 — Verify system behavior for exception flow: Payment Failure (Subscribe)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.3-E1
Related Use Case	UC-6.3 — Subscribe
Requirement Traceability	PR-ACC-03
Test Objective	Confirm the system correctly handles the exception scenario "Payment Failure" defined in Exception Flow E1 of "Subscribe", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	- System is online - Registered Reader is authenticated and logged in - Condition under test: The condition "Payment Failure" occurs for the subscription plan selection during the "Subscribe" workflow.
Test Data	Registered Reader / Payment Gateway (secondary, via include) account (as required by the scenario); a subscription plan selection configured so the "Payment Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a subscription plan selection / environment state that will trigger the "Payment Failure" condition. 2. As Registered Reader / Payment Gateway (secondary, via include), perform the relevant Basic Flow action(s) on the subscription plan selection so that the "Payment Failure" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state.

TEST CASE ELEMENT	SPECIFICATION
	6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Payment Failure" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	On successful payment, the Reader's account is upgraded to Subscriber status
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.3-E-AUTH — Verify access is denied when session/authorization is invalid (Subscribe)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.3-E-AUTH
Related Use Case	UC-6.3 — Subscribe
Requirement Traceability	PR-ACC-03
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Subscribe", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	- System is online - Registered Reader is authenticated and logged in - Condition under test: Registered Reader / Payment Gateway (secondary, via include)'s session has expired, or Registered Reader / Payment Gateway (secondary, via include) lacks the role/ownership permission required to perform "Subscribe", immediately before the state-changing step.
Test Data	Registered Reader / Payment Gateway (secondary, via include) account (as required by the scenario); a subscription plan selection configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	- Simulate an expired session (or log in as a user without the required role/ownership permission) for Registered Reader / Payment Gateway (secondary, via include). - Attempt to initiate the protected "Subscribe" action (subscription plan selection).

TEST CASE ELEMENT	SPECIFICATION
	3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the subscription plan selection. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	On successful payment, the Reader's account is upgraded to Subscriber status
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-6.4: Update Profile Information — Test Suite

Use case purpose: Allow a User to update their personal profile details, such as name, avatar, bio, and preferences.

Actors: User (Registered Reader, Staff) **Requirement:** PR-ACC-04 **Priority:** Low

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-6.4.

Test Case ID	Scenario	Type	Priority
TC-6.4-BF	Verify successful update profile information (Basic Flow)	Positive	Low
TC-6.4-A1	Verify system behavior for alternate flow: Photo Upload (Update Profile Information)	Negative / Alternate	Low
TC-6.4-E1	Verify system behavior for exception flow: Invalid Input (Update Profile Information)	Negative / Exception	Low
TC-6.4-E-AUTH	Verify access is denied when session/authorization is invalid (Update Profile Information)	Negative / Exception	Low

TC-6.4-BF — Verify successful update profile information (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.4-BF
Related Use Case	UC-6.4 — Update Profile Information
Requirement Traceability	PR-ACC-04

TEST CASE ELEMENT	SPECIFICATION
Test Objective	Confirm that User (Registered Reader, Staff) can successfully complete the "Update Profile Information" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow a User to update their personal profile details, such as name, avatar, bio, and preferences.
Priority	Low
Test Type	Positive
Preconditions	• System is online • User is authenticated and logged in
Test Data	Valid, active User (Registered Reader, Staff) account with the required role/permission; a profile information in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. User navigates to Account Settings. 2. User selects "Update Profile Information." 3. System displays the current profile form. 4. User edits the desired fields. 5. User saves. 6. System validates and stores the updated profile. 7. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Profile information is updated
Postconditions (Verification)	• Profile information is updated
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.4-A1 — Verify system behavior for alternate flow: Photo Upload (Update Profile Information)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.4-A1
Related Use Case	UC-6.4 — Update Profile Information
Requirement Traceability	PR-ACC-04
Test Objective	Confirm the system correctly handles the alternate scenario "Photo Upload" defined in Alternate Flow A1 of "Update Profile Information", protecting data integrity and informing the user appropriately.
Priority	Low
Test Type	Negative / Alternate

TEST CASE ELEMENT	SPECIFICATION
Preconditions	• System is online • User is authenticated and logged in • Condition under test: The condition "Photo Upload" occurs for the profile information during the "Update Profile Information" workflow.
Test Data	User (Registered Reader, Staff) account (as required by the scenario); a profile information configured so the "Photo Upload" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a profile information / environment state that will trigger the "Photo Upload" condition. 2. As User (Registered Reader, Staff), perform the relevant Basic Flow action(s) on the profile information so that the "Photo Upload" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Photo Upload" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Profile information is updated
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.4-E1 — Verify system behavior for exception flow: Invalid Input (Update Profile Information)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.4-E1
Related Use Case	UC-6.4 — Update Profile Information
Requirement Traceability	PR-ACC-04
Test Objective	Confirm the system correctly handles the exception scenario "Invalid Input" defined in Exception Flow E1 of "Update Profile Information", protecting data integrity and informing the user appropriately.

TEST CASE ELEMENT	SPECIFICATION
Priority	Low
Test Type	Negative / Exception
Preconditions	• System is online • User is authenticated and logged in • Condition under test: The condition "Invalid Input" occurs for the profile information during the "Update Profile Information" workflow.
Test Data	User (Registered Reader, Staff) account (as required by the scenario); a profile information configured so the "Invalid Input" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a profile information / environment state that will trigger the "Invalid Input" condition. 2. As User (Registered Reader, Staff), perform the relevant Basic Flow action(s) on the profile information so that the "Invalid Input" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Invalid Input" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Profile information is updated
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.4-E-AUTH — Verify access is denied when session/authorization is invalid (Update Profile Information)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.4-E-AUTH
Related Use Case	UC-6.4 — Update Profile Information
Requirement Traceability	PR-ACC-04

TEST CASE ELEMENT	SPECIFICATION
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Update Profile Information", protecting data integrity and informing the user appropriately.
Priority	Low
Test Type	Negative / Exception
Preconditions	• System is online • User is authenticated and logged in • Condition under test: User (Registered Reader, Staff)'s session has expired, or User (Registered Reader, Staff) lacks the role/ownership permission required to perform "Update Profile Information", immediately before the state-changing step.
Test Data	User (Registered Reader, Staff) account (as required by the scenario); a profile information configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for User (Registered Reader, Staff). 2. Attempt to initiate the protected "Update Profile Information" action (profile information). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the profile information. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Profile information is updated
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-6.5: Manage Account Security — Test Suite

Use case purpose: Allow a User to change their password and manage security settings such as two-factor authentication.

Actors: User (Registered Reader, Staff) **Requirement:** PR-ACC-05 **Priority:** High

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-6.5.

Test Case ID	Scenario	Type	Priority
TC-6.5-BF	Verify successful manage account security (Basic Flow)	Positive	High
TC-6.5-A1	Verify system behavior for alternate flow: Enable 2Fa (Manage Account Security)	Negative / Alternate	High
TC-6.5-E1	Verify system behavior for exception flow: Incorrect Current Password (Manage Account Security)	Negative / Exception	High
TC-6.5-E-AUTH	Verify access is denied when session/authorization is invalid (Manage Account Security)	Negative / Exception	High

TC-6.5-BF — Verify successful manage account security (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.5-BF
Related Use Case	UC-6.5 — Manage Account Security
Requirement Traceability	PR-ACC-05
Test Objective	Confirm that User (Registered Reader, Staff) can successfully complete the "Manage Account Security" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow a User to change their password and manage security settings such as two-factor authentication.
Priority	High
Test Type	Positive
Preconditions	• System is online • User is authenticated and logged in
Test Data	Valid, active User (Registered Reader, Staff) account with the required role/permission; a account security settings in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. User navigates to Account Settings. 2. User selects "Manage Account Security." 3. System displays security options: change password, enable two-factor authentication, view active sessions. 4. User updates the desired settings. 5. User confirms the changes. 6. System applies the changes and, if the password changed, invalidates other active sessions. 7. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Security settings are updated; other active sessions are invalidated if the password changed
Postconditions (Verification)	• Security settings are updated; other active sessions are invalidated if the password changed

TEST CASE ELEMENT	SPECIFICATION
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.5-A1 — Verify system behavior for alternate flow: Enable 2Fa (Manage Account Security)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.5-A1
Related Use Case	UC-6.5 — Manage Account Security
Requirement Traceability	PR-ACC-05
Test Objective	Confirm the system correctly handles the alternate scenario "Enable 2Fa" defined in Alternate Flow A1 of "Manage Account Security", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • User is authenticated and logged in • Condition under test: The condition "Enable 2Fa" occurs for the account security settings during the "Manage Account Security" workflow.
Test Data	User (Registered Reader, Staff) account (as required by the scenario); a account security settings configured so the "Enable 2Fa" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a account security settings / environment state that will trigger the "Enable 2Fa" condition. 2. As User (Registered Reader, Staff), perform the relevant Basic Flow action(s) on the account security settings so that the "Enable 2Fa" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Enable 2Fa" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.

TEST CASE ELEMENT	SPECIFICATION
Postconditions (Verification)	Security settings are updated; other active sessions are invalidated if the password changed
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.5-E1 — Verify system behavior for exception flow: Incorrect Current Password (Manage Account Security)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.5-E1
Related Use Case	UC-6.5 — Manage Account Security
Requirement Traceability	PR-ACC-05
Test Objective	Confirm the system correctly handles the exception scenario "Incorrect Current Password" defined in Exception Flow E1 of "Manage Account Security", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	- System is online - User is authenticated and logged in - Condition under test: The condition "Incorrect Current Password" occurs for the account security settings during the "Manage Account Security" workflow.
Test Data	User (Registered Reader, Staff) account (as required by the scenario); a account security settings configured so the "Incorrect Current Password" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a account security settings / environment state that will trigger the "Incorrect Current Password" condition. 2. As User (Registered Reader, Staff), perform the relevant Basic Flow action(s) on the account security settings so that the "Incorrect Current Password" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it.

TEST CASE ELEMENT	SPECIFICATION
	8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Incorrect Current Password" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Security settings are updated; other active sessions are invalidated if the password changed
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-6.5-E-AUTH — Verify access is denied when session/authorization is invalid (Manage Account Security)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-6.5-E-AUTH
Related Use Case	UC-6.5 — Manage Account Security
Requirement Traceability	PR-ACC-05
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Manage Account Security", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • User is authenticated and logged in • Condition under test: User (Registered Reader, Staff)'s session has expired, or User (Registered Reader, Staff) lacks the role/ownership permission required to perform "Manage Account Security", immediately before the state-changing step.
Test Data	User (Registered Reader, Staff) account (as required by the scenario); a account security settings configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for User (Registered Reader, Staff). 2. Attempt to initiate the protected "Manage Account Security" action (account security settings). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes.

TEST CASE ELEMENT	SPECIFICATION
	4. Verify the system blocks the action and does NOT partially commit any change to the account security settings. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Security settings are updated; other active sessions are invalidated if the password changed
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

8. Module 7: Manage Content Interaction

UC-7.1: Browse Featured Content — Test Suite

Use case purpose: Allow a Reader to browse curated and featured articles and issues on the platform.

Actors: Reader **Requirement:** PR-INT-01 **Priority:** Medium

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-7.1.

Test Case ID	Scenario	Type	Priority
TC-7.1-BF	Verify successful browse featured content (Basic Flow)	Positive	Medium
TC-7.1-A1	Verify system behavior for alternate flow: Category Filter (Browse Featured Content)	Negative / Alternate	Medium
TC-7.1-E1	Verify system behavior for exception flow: No Featured Content (Browse Featured Content)	Negative / Exception	Medium

TC-7.1-BF — Verify successful browse featured content (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-7.1-BF
Related Use Case	UC-7.1 — Browse Featured Content
Requirement Traceability	PR-INT-01
Test Objective	Confirm that Reader can successfully complete the "Browse Featured Content" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow a Reader to browse curated and featured articles and issues on the platform.
Priority	Medium
Test Type	Positive
Preconditions	• System is online • Reader has access to the homepage or featured section
Test Data	Valid, active Reader account with the required role/permission; a featured content listing in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Reader navigates to the homepage or featured section. 2. System retrieves and displays featured content. 3. Reader scrolls or browses through the featured items. 4. Reader selects an article to read; while viewing it, the Reader may optionally trigger Bookmark Content (UC-7.1.1) or Share Content (UC-7.1.2). 5. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Featured content is displayed to the Reader

TEST CASE ELEMENT	SPECIFICATION
Postconditions (Verification)	Featured content is displayed to the Reader
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-7.1-A1 — Verify system behavior for alternate flow: Category Filter (Browse Featured Content)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-7.1-A1
Related Use Case	UC-7.1 — Browse Featured Content
Requirement Traceability	PR-INT-01
Test Objective	Confirm the system correctly handles the alternate scenario "Category Filter" defined in Alternate Flow A1 of "Browse Featured Content", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	- System is online - Reader has access to the homepage or featured section - Condition under test: The condition "Category Filter" occurs for the featured content listing during the "Browse Featured Content" workflow.
Test Data	Reader account (as required by the scenario); a featured content listing configured so the "Category Filter" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a featured content listing / environment state that will trigger the "Category Filter" condition. 2. As Reader, perform the relevant Basic Flow action(s) on the featured content listing so that the "Category Filter" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.

TEST CASE ELEMENT	SPECIFICATION
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Category Filter" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	Featured content is displayed to the Reader
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-7.1-E1 — Verify system behavior for exception flow: No Featured Content (Browse Featured Content)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-7.1-E1
Related Use Case	UC-7.1 — Browse Featured Content
Requirement Traceability	PR-INT-01
Test Objective	Confirm the system correctly handles the exception scenario "No Featured Content" defined in Exception Flow E1 of "Browse Featured Content", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	- System is online - Reader has access to the homepage or featured section - Condition under test: The condition "No Featured Content" occurs for the featured content listing during the "Browse Featured Content" workflow.
Test Data	Reader account (as required by the scenario); a featured content listing configured so the "No Featured Content" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a featured content listing / environment state that will trigger the "No Featured Content" condition. 2. As Reader, perform the relevant Basic Flow action(s) on the featured content listing so that the "No Featured Content" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user.

TEST CASE ELEMENT	SPECIFICATION
	7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "No Featured Content" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	Featured content is displayed to the Reader
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-7.1.1: Bookmark Content — Test Suite

Use case purpose: Allow a Reader to save an article for later reading.

Actors: Reader **Requirement:** PR-INT-02 **Priority:** Low

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-7.1.1.

Test Case ID	Scenario	Type	Priority
TC-7.1.1-BF	Verify successful bookmark content (Basic Flow)	Positive	Low
TC-7.1.1-A1	Verify system behavior for alternate flow: Remove Bookmark (Bookmark Content)	Negative / Alternate	Low
TC-7.1.1-E1	Verify system behavior for exception flow: Not Logged In (Bookmark Content)	Negative / Exception	Low
TC-7.1.1-E-AUTH	Verify access is denied when session/authorization is invalid (Bookmark Content)	Negative / Exception	Low

TC-7.1.1-BF — Verify successful bookmark content (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-7.1.1-BF
Related Use Case	UC-7.1.1 — Bookmark Content
Requirement Traceability	PR-INT-02
Test Objective	Confirm that Reader can successfully complete the "Bookmark Content" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow a Reader to save an article for later reading.
Priority	Low
Test Type	Positive

TEST CASE ELEMENT	SPECIFICATION
Preconditions	• System is online • Reader is authenticated and logged in • Reader is viewing an article reached through Browse Featured Content (UC-7.1)
Test Data	Valid, active Reader account with the required role/permission; a content bookmark in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. While viewing an article reached through Browse Featured Content (UC-7.1), Reader selects “Bookmark.” 2. System adds the article to the Reader’s bookmark list. 3. System confirms with a visual indicator and returns control to Browse Featured Content (UC-7.1). 4. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Article is added to the Reader’s bookmark list
Postconditions (Verification)	• Article is added to the Reader’s bookmark list
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-7.1.1-A1 — Verify system behavior for alternate flow: Remove Bookmark (Bookmark Content)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-7.1.1-A1
Related Use Case	UC-7.1.1 — Bookmark Content
Requirement Traceability	PR-INT-02
Test Objective	Confirm the system correctly handles the alternate scenario "Remove Bookmark" defined in Alternate Flow A1 of "Bookmark Content", protecting data integrity and informing the user appropriately.
Priority	Low
Test Type	Negative / Alternate
Preconditions	• System is online • Reader is authenticated and logged in • Reader is viewing an article reached through Browse Featured Content (UC-7.1) • Condition under test: The condition "Remove Bookmark" occurs for the content bookmark during the "Bookmark Content" workflow.
Test Data	Reader account (as required by the scenario); a content bookmark configured so the "Remove Bookmark" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).

TEST CASE ELEMENT	SPECIFICATION
Test Steps	1. Prepare a content bookmark / environment state that will trigger the "Remove Bookmark" condition. 2. As Reader, perform the relevant Basic Flow action(s) on the content bookmark so that the "Remove Bookmark" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Remove Bookmark" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Article is added to the Reader's bookmark list
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-7.1.1-E1 — Verify system behavior for exception flow: Not Logged In (Bookmark Content)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-7.1.1-E1
Related Use Case	UC-7.1.1 — Bookmark Content
Requirement Traceability	PR-INT-02
Test Objective	Confirm the system correctly handles the exception scenario "Not Logged In" defined in Exception Flow E1 of "Bookmark Content", protecting data integrity and informing the user appropriately.
Priority	Low
Test Type	Negative / Exception
Preconditions	• System is online • Reader is authenticated and logged in • Reader is viewing an article reached through Browse Featured Content (UC-7.1) • Condition under test: The condition "Not Logged In" occurs for the content bookmark during the "Bookmark Content" workflow.

TEST CASE ELEMENT	SPECIFICATION
Test Data	Reader account (as required by the scenario); a content bookmark configured so the "Not Logged In" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a content bookmark / environment state that will trigger the "Not Logged In" condition. 2. As Reader, perform the relevant Basic Flow action(s) on the content bookmark so that the "Not Logged In" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Not Logged In" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Article is added to the Reader's bookmark list
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-7.1.1-E-AUTH — Verify access is denied when session/authorization is invalid (Bookmark Content)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-7.1.1-E-AUTH
Related Use Case	UC-7.1.1 — Bookmark Content
Requirement Traceability	PR-INT-02
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Bookmark Content", protecting data integrity and informing the user appropriately.
Priority	Low
Test Type	Negative / Exception
Preconditions	• System is online

TEST CASE ELEMENT	SPECIFICATION
	• Reader is authenticated and logged in • Reader is viewing an article reached through Browse Featured Content (UC-7.1) • Condition under test: Reader's session has expired, or Reader lacks the role/ownership permission required to perform "Bookmark Content", immediately before the state-changing step.
Test Data	Reader account (as required by the scenario); a content bookmark configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Reader. 2. Attempt to initiate the protected "Bookmark Content" action (content bookmark). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the content bookmark. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Article is added to the Reader's bookmark list
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-7.1.2: Share Content — Test Suite

Use case purpose: Allow a Reader to share an article link via social media or a direct link copy.

Actors: Reader **Requirement:** PR-INT-03 **Priority:** Low

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-7.1.2.

Test Case ID	Scenario	Type	Priority
TC-7.1.2-BF	Verify successful share content (Basic Flow)	Positive	Low
TC-7.1.2-A1	Verify system behavior for alternate flow: Copy Link (Share Content)	Negative / Alternate	Low
TC-7.1.2-E1	Verify system behavior for exception flow: Service Unreachable (Share Content)	Negative / Exception	Low

TC-7.1.2-BF — Verify successful share content (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-7.1.2-BF
Related Use Case	UC-7.1.2 — Share Content
Requirement Traceability	PR-INT-03
Test Objective	Confirm that Reader can successfully complete the "Share Content" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow a Reader to share an article link via social media or a direct link copy.
Priority	Low
Test Type	Positive
Preconditions	• System is online • Reader is viewing an article reached through Browse Featured Content (UC-7.1)
Test Data	Valid, active Reader account with the required role/permission; a content share link in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. While viewing an article reached through Browse Featured Content (UC-7.1), Reader selects “Share.” 2. System displays sharing options (social platforms, copy link). 3. Reader selects a channel. 4. System generates the shareable link, completes the share action, and returns control to Browse Featured Content (UC-7.1). 5. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - A shareable link is generated and the share action is completed
Postconditions (Verification)	• A shareable link is generated and the share action is completed
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-7.1.2-A1 — Verify system behavior for alternate flow: Copy Link (Share Content)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-7.1.2-A1
Related Use Case	UC-7.1.2 — Share Content
Requirement Traceability	PR-INT-03

TEST CASE ELEMENT	SPECIFICATION
Test Objective	Confirm the system correctly handles the alternate scenario "Copy Link" defined in Alternate Flow A1 of "Share Content", protecting data integrity and informing the user appropriately.
Priority	Low
Test Type	Negative / Alternate
Preconditions	• System is online • Reader is viewing an article reached through Browse Featured Content (UC-7.1) • Condition under test: The condition "Copy Link" occurs for the content share link during the "Share Content" workflow.
Test Data	Reader account (as required by the scenario); a content share link configured so the "Copy Link" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a content share link / environment state that will trigger the "Copy Link" condition. 2. As Reader, perform the relevant Basic Flow action(s) on the content share link so that the "Copy Link" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Copy Link" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• A shareable link is generated and the share action is completed
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-7.1.2-E1 — Verify system behavior for exception flow: Service Unreachable (Share Content)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-7.1.2-E1
Related Use Case	UC-7.1.2 — Share Content

TEST CASE ELEMENT	SPECIFICATION
Requirement Traceability	PR-INT-03
Test Objective	Confirm the system correctly handles the exception scenario "Service Unreachable" defined in Exception Flow E1 of "Share Content", protecting data integrity and informing the user appropriately.
Priority	Low
Test Type	Negative / Exception
Preconditions	• System is online • Reader is viewing an article reached through Browse Featured Content (UC-7.1) • Condition under test: The condition "Service Unreachable" occurs for the content share link during the "Share Content" workflow.
Test Data	Reader account (as required by the scenario); a content share link configured so the "Service Unreachable" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a content share link / environment state that will trigger the "Service Unreachable" condition. 2. As Reader, perform the relevant Basic Flow action(s) on the content share link so that the "Service Unreachable" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Service Unreachable" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• A shareable link is generated and the share action is completed
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

9. Module 8: Manage Premium Content

UC-8.1: Download Digital Issues — Test Suite

Use case purpose: Allow a Subscriber to download a full digital issue for offline reading.

Actors: Subscriber **Requirement:** PR-PRM-01 **Priority:** Medium

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-8.1.

Test Case ID	Scenario	Type	Priority
TC-8.1-BF	Verify successful download digital issues (Basic Flow)	Positive	Medium
TC-8.1-A1	Verify system behavior for alternate flow: Format Choice (Download Digital Issues)	Negative / Alternate	Medium
TC-8.1-E1	Verify system behavior for exception flow: Expired Subscription (Download Digital Issues)	Negative / Exception	Medium

TC-8.1-BF — Verify successful download digital issues (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-8.1-BF
Related Use Case	UC-8.1 — Download Digital Issues
Requirement Traceability	PR-PRM-01
Test Objective	Confirm that Subscriber can successfully complete the "Download Digital Issues" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow a Subscriber to download a full digital issue for offline reading.
Priority	Medium
Test Type	Positive
Preconditions	• System is online • Subscriber is authenticated and logged in • Subscriber has an active premium subscription
Test Data	Valid, active Subscriber account with the required role/permission; a digital issue download in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Subscriber navigates to an available issue. 2. Subscriber selects "Download Digital Issue." 3. System verifies the Subscriber's active premium status. 4. System generates or serves the downloadable file. 5. Subscriber's device saves the file. 6. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - The digital issue file is downloaded to the Subscriber's device

TEST CASE ELEMENT	SPECIFICATION
Postconditions (Verification)	The digital issue file is downloaded to the Subscriber's device
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-8.1-A1 — Verify system behavior for alternate flow: Format Choice (Download Digital Issues)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-8.1-A1
Related Use Case	UC-8.1 — Download Digital Issues
Requirement Traceability	PR-PRM-01
Test Objective	Confirm the system correctly handles the alternate scenario "Format Choice" defined in Alternate Flow A1 of "Download Digital Issues", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	- System is online - Subscriber is authenticated and logged in - Subscriber has an active premium subscription - Condition under test: The condition "Format Choice" occurs for the digital issue download during the "Download Digital Issues" workflow.
Test Data	Subscriber account (as required by the scenario); a digital issue download configured so the "Format Choice" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a digital issue download / environment state that will trigger the "Format Choice" condition. 2. As Subscriber, perform the relevant Basic Flow action(s) on the digital issue download so that the "Format Choice" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.

TEST CASE ELEMENT	SPECIFICATION
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Format Choice" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	The digital issue file is downloaded to the Subscriber's device
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-8.1-E1 — Verify system behavior for exception flow: Expired Subscription (Download Digital Issues)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-8.1-E1
Related Use Case	UC-8.1 — Download Digital Issues
Requirement Traceability	PR-PRM-01
Test Objective	Confirm the system correctly handles the exception scenario "Expired Subscription" defined in Exception Flow E1 of "Download Digital Issues", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	- System is online - Subscriber is authenticated and logged in - Subscriber has an active premium subscription - Condition under test: The condition "Expired Subscription" occurs for the digital issue download during the "Download Digital Issues" workflow.
Test Data	Subscriber account (as required by the scenario); a digital issue download configured so the "Expired Subscription" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a digital issue download / environment state that will trigger the "Expired Subscription" condition. 2. As Subscriber, perform the relevant Basic Flow action(s) on the digital issue download so that the "Expired Subscription" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state.

TEST CASE ELEMENT	SPECIFICATION
	6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Expired Subscription" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	The digital issue file is downloaded to the Subscriber's device
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-8.2: Access Premium Articles — Test Suite

Use case purpose: Allow a Subscriber to view the full content of paywalled premium articles.

Actors: Subscriber **Requirement:** PR-PRM-02 **Priority:** High

This suite contains 3 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-8.2.

Test Case ID	Scenario	Type	Priority
TC-8.2-BF	Verify successful access premium articles (Basic Flow)	Positive	High
TC-8.2-A1	Verify system behavior for alternate flow: Single Article Access (Access Premium Articles)	Negative / Alternate	High
TC-8.2-E1	Verify system behavior for exception flow: Inactive Subscription (Access Premium Articles)	Negative / Exception	High

TC-8.2-BF — Verify successful access premium articles (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-8.2-BF
Related Use Case	UC-8.2 — Access Premium Articles
Requirement Traceability	PR-PRM-02
Test Objective	Confirm that Subscriber can successfully complete the "Access Premium Articles" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow a Subscriber to view the full content of paywalled premium articles.
Priority	High
Test Type	Positive

TEST CASE ELEMENT	SPECIFICATION
Preconditions	• System is online • Subscriber is authenticated and logged in
Test Data	Valid, active Subscriber account with the required role/permission; a premium article access in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Subscriber selects a premium-marked article. 2. System verifies the Subscriber's active subscription status. 3. System renders the full article content, bypassing the paywall preview. 4. Subscriber reads the article. 5. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Full premium article content is rendered for the Subscriber
Postconditions (Verification)	• Full premium article content is rendered for the Subscriber
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-8.2-A1 — Verify system behavior for alternate flow: Single Article Access (Access Premium Articles)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-8.2-A1
Related Use Case	UC-8.2 — Access Premium Articles
Requirement Traceability	PR-PRM-02
Test Objective	Confirm the system correctly handles the alternate scenario "Single Article Access" defined in Alternate Flow A1 of "Access Premium Articles", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Subscriber is authenticated and logged in • Condition under test: The condition "Single Article Access" occurs for the premium article access during the "Access Premium Articles" workflow.
Test Data	Subscriber account (as required by the scenario); a premium article access configured so the "Single Article Access" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).

TEST CASE ELEMENT	SPECIFICATION
Test Steps	1. Prepare a premium article access / environment state that will trigger the "Single Article Access" condition. 2. As Subscriber, perform the relevant Basic Flow action(s) on the premium article access so that the "Single Article Access" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Single Article Access" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Full premium article content is rendered for the Subscriber
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-8.2-E1 — Verify system behavior for exception flow: Inactive Subscription (Access Premium Articles)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-8.2-E1
Related Use Case	UC-8.2 — Access Premium Articles
Requirement Traceability	PR-PRM-02
Test Objective	Confirm the system correctly handles the exception scenario "Inactive Subscription" defined in Exception Flow E1 of "Access Premium Articles", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Subscriber is authenticated and logged in • Condition under test: The condition "Inactive Subscription" occurs for the premium article access during the "Access Premium Articles" workflow.

TEST CASE ELEMENT	SPECIFICATION
Test Data	Subscriber account (as required by the scenario); a premium article access configured so the "Inactive Subscription" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a premium article access / environment state that will trigger the "Inactive Subscription" condition. 2. As Subscriber, perform the relevant Basic Flow action(s) on the premium article access so that the "Inactive Subscription" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Inactive Subscription" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Full premium article content is rendered for the Subscriber
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

10. Module 9: Manage Payment

UC-9.1: Subscribe to Premium Plan — Test Suite

Use case purpose: Process a Registered Reader's purchase of a recurring premium subscription plan via the Payment Gateway.

Actors: Registered Reader, Payment Gateway (secondary, via include: UC-9.1.1) **Requirement:** PR-PAY-01

Priority: High

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-9.1.

Test Case ID	Scenario	Type	Priority
TC-9.1-BF	Verify successful subscribe to premium plan (Basic Flow)	Positive	High
TC-9.1-A1	Verify system behavior for alternate flow: Saved Payment Method (Subscribe to Premium Plan)	Negative / Alternate	High
TC-9.1-E1	Verify system behavior for exception flow: Transaction Declined (Subscribe to Premium Plan)	Negative / Exception	High
TC-9.1-E-AUTH	Verify access is denied when session/authorization is invalid (Subscribe to Premium Plan)	Negative / Exception	High

TC-9.1-BF — Verify successful subscribe to premium plan (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.1-BF
Related Use Case	UC-9.1 — Subscribe to Premium Plan
Requirement Traceability	PR-PAY-01
Test Objective	Confirm that Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) can successfully complete the "Subscribe to Premium Plan" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Process a Registered Reader's purchase of a recurring premium subscription plan via the Payment Gateway.
Priority	High
Test Type	Positive
Preconditions	• System is online • Registered Reader is authenticated and logged in • Payment Gateway service is reachable
Test Data	Valid, active Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) account with the required role/permission; a premium plan subscription order in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Registered Reader selects a subscription plan and confirms intent to subscribe. 2. System includes Process Payment Transaction (UC-9.1.1) to capture and process the payment.

TEST CASE ELEMENT	SPECIFICATION
	3. On receiving a success token from UC-9.1.1, System activates the Subscriber tier on the Reader's account and records the billing cycle. 4. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Subscriber tier is activated on the Reader's account and the billing cycle is recorded
Postconditions (Verification)	Subscriber tier is activated on the Reader's account and the billing cycle is recorded
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.1-A1 — Verify system behavior for alternate flow: Saved Payment Method (Subscribe to Premium Plan)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.1-A1
Related Use Case	UC-9.1 — Subscribe to Premium Plan
Requirement Traceability	PR-PAY-01
Test Objective	Confirm the system correctly handles the alternate scenario "Saved Payment Method" defined in Alternate Flow A1 of "Subscribe to Premium Plan", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	- System is online - Registered Reader is authenticated and logged in - Payment Gateway service is reachable - Condition under test: The condition "Saved Payment Method" occurs for the premium plan subscription order during the "Subscribe to Premium Plan" workflow.
Test Data	Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) account (as required by the scenario); a premium plan subscription order configured so the "Saved Payment Method" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	- Prepare a premium plan subscription order / environment state that will trigger the "Saved Payment Method" condition. - As Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1), perform the relevant Basic Flow action(s) on the premium plan subscription order so that the "Saved Payment Method" condition is encountered. - Verify the system detects the condition at the correct point in the Basic Flow.

TEST CASE ELEMENT	SPECIFICATION
	4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Saved Payment Method" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	Subscriber tier is activated on the Reader's account and the billing cycle is recorded
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.1-E1 — Verify system behavior for exception flow: Transaction Declined (Subscribe to Premium Plan)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.1-E1
Related Use Case	UC-9.1 — Subscribe to Premium Plan
Requirement Traceability	PR-PAY-01
Test Objective	Confirm the system correctly handles the exception scenario "Transaction Declined" defined in Exception Flow E1 of "Subscribe to Premium Plan", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	- System is online - Registered Reader is authenticated and logged in - Payment Gateway service is reachable - Condition under test: The condition "Transaction Declined" occurs for the premium plan subscription order during the "Subscribe to Premium Plan" workflow.
Test Data	Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) account (as required by the scenario); a premium plan subscription order configured so the

TEST CASE ELEMENT	SPECIFICATION
	"Transaction Declined" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a premium plan subscription order / environment state that will trigger the "Transaction Declined" condition. 2. As Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1), perform the relevant Basic Flow action(s) on the premium plan subscription order so that the "Transaction Declined" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Transaction Declined" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Subscriber tier is activated on the Reader's account and the billing cycle is recorded
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.1-E-AUTH — Verify access is denied when session/authorization is invalid (Subscribe to Premium Plan)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.1-E-AUTH
Related Use Case	UC-9.1 — Subscribe to Premium Plan
Requirement Traceability	PR-PAY-01
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Subscribe to Premium Plan", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception

TEST CASE ELEMENT	SPECIFICATION
Preconditions	• System is online • Registered Reader is authenticated and logged in • Payment Gateway service is reachable • Condition under test: Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1)'s session has expired, or Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) lacks the role/ownership permission required to perform "Subscribe to Premium Plan", immediately before the state-changing step.
Test Data	Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) account (as required by the scenario); a premium plan subscription order configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1). 2. Attempt to initiate the protected "Subscribe to Premium Plan" action (premium plan subscription order). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the premium plan subscription order. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Subscriber tier is activated on the Reader's account and the billing cycle is recorded
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-9.1.1: Process Payment Transaction — Test Suite

Use case purpose: Serve as the central payment-processing engine that connects to the Payment Gateway, transmits transaction details, and returns a success or fail token to the invoking use case.

Actors: Registered Reader (indirect, via invoking use case), Payment Gateway **Requirement:** PR-PAY-06 **Priority:** High

This suite contains 5 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-9.1.1.

Test Case ID	Scenario	Type	Priority
TC-9.1.1-BF	Verify successful process payment transaction (Basic Flow)	Positive	High
TC-9.1.1-A1	Verify system behavior for alternate flow: Saved Payment Method (Process Payment Transaction)	Negative / Alternate	High
TC-9.1.1-E1	Verify system behavior for exception flow: Transaction Declined (Process Payment Transaction)	Negative / Exception	High
TC-9.1.1-E2	Verify system behavior for exception flow: Gateway Unreachable (Process Payment Transaction)	Negative / Exception	High
TC-9.1.1-E-AUTH	Verify access is denied when session/authorization is invalid (Process Payment Transaction)	Negative / Exception	High

TC-9.1.1-BF — Verify successful process payment transaction (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.1.1-BF
Related Use Case	UC-9.1.1 — Process Payment Transaction
Requirement Traceability	PR-PAY-06
Test Objective	Confirm that Registered Reader (indirect, via invoking use case) / Payment Gateway can successfully complete the "Process Payment Transaction" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Serve as the central payment-processing engine that connects to the Payment Gateway, transmits transaction details, and returns a success or fail token to the invoking use case.
Priority	High
Test Type	Positive
Preconditions	• System is online • Payment Gateway service is reachable • The invoking use case has supplied the transaction amount and context
Test Data	Valid, active Registered Reader (indirect, via invoking use case) / Payment Gateway account with the required role/permission; a payment transaction in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Invoking use case (UC-9.1, UC-9.2, or UC-9.3) triggers Process Payment Transaction and passes the transaction amount and context. 2. System connects to the Payment Gateway. 3. Reader enters or confirms payment details. 4. Payment Gateway processes the transaction. 5. Payment Gateway returns a success or fail token to the System. 6. System returns the token and transaction reference to the invoking use case. 7. Verify each postcondition listed below is met.

TEST CASE ELEMENT	SPECIFICATION
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Transaction is processed by the Payment Gateway and a success or fail token, along with a transaction reference, is returned to the invoking use case
Postconditions (Verification)	Transaction is processed by the Payment Gateway and a success or fail token, along with a transaction reference, is returned to the invoking use case
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.1.1-A1 — Verify system behavior for alternate flow: Saved Payment Method (Process Payment Transaction)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.1.1-A1
Related Use Case	UC-9.1.1 — Process Payment Transaction
Requirement Traceability	PR-PAY-06
Test Objective	Confirm the system correctly handles the alternate scenario "Saved Payment Method" defined in Alternate Flow A1 of "Process Payment Transaction", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	- System is online - Payment Gateway service is reachable - The invoking use case has supplied the transaction amount and context - Condition under test: The condition "Saved Payment Method" occurs for the payment transaction during the "Process Payment Transaction" workflow.
Test Data	Registered Reader (indirect, via invoking use case) / Payment Gateway account (as required by the scenario); a payment transaction configured so the "Saved Payment Method" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a payment transaction / environment state that will trigger the "Saved Payment Method" condition. 2. As Registered Reader (indirect, via invoking use case) / Payment Gateway, perform the relevant Basic Flow action(s) on the payment transaction so that the "Saved Payment Method" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules.

TEST CASE ELEMENT	SPECIFICATION
	5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Saved Payment Method" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	Transaction is processed by the Payment Gateway and a success or fail token, along with a transaction reference, is returned to the invoking use case
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.1.1-E1 — Verify system behavior for exception flow: Transaction Declined (Process Payment Transaction)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.1.1-E1
Related Use Case	UC-9.1.1 — Process Payment Transaction
Requirement Traceability	PR-PAY-06
Test Objective	Confirm the system correctly handles the exception scenario "Transaction Declined" defined in Exception Flow E1 of "Process Payment Transaction", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	- System is online - Payment Gateway service is reachable - The invoking use case has supplied the transaction amount and context - Condition under test: The condition "Transaction Declined" occurs for the payment transaction during the "Process Payment Transaction" workflow.
Test Data	Registered Reader (indirect, via invoking use case) / Payment Gateway account (as required by the scenario); a payment transaction configured so the "Transaction Declined" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).

TEST CASE ELEMENT	SPECIFICATION
Test Steps	1. Prepare a payment transaction / environment state that will trigger the "Transaction Declined" condition. 2. As Registered Reader (indirect, via invoking use case) / Payment Gateway, perform the relevant Basic Flow action(s) on the payment transaction so that the "Transaction Declined" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Transaction Declined" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Transaction is processed by the Payment Gateway and a success or fail token, along with a transaction reference, is returned to the invoking use case
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.1.1-E2 — Verify system behavior for exception flow: Gateway Unreachable (Process Payment Transaction)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.1.1-E2
Related Use Case	UC-9.1.1 — Process Payment Transaction
Requirement Traceability	PR-PAY-06
Test Objective	Confirm the system correctly handles the exception scenario "Gateway Unreachable" defined in Exception Flow E2 of "Process Payment Transaction", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Payment Gateway service is reachable • The invoking use case has supplied the transaction amount and context

TEST CASE ELEMENT	SPECIFICATION
	Condition under test: The condition "Gateway Unreachable" occurs for the payment transaction during the "Process Payment Transaction" workflow.
Test Data	Registered Reader (indirect, via invoking use case) / Payment Gateway account (as required by the scenario); a payment transaction configured so the "Gateway Unreachable" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a payment transaction / environment state that will trigger the "Gateway Unreachable" condition. 2. As Registered Reader (indirect, via invoking use case) / Payment Gateway, perform the relevant Basic Flow action(s) on the payment transaction so that the "Gateway Unreachable" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E2: it detects the "Gateway Unreachable" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	Transaction is processed by the Payment Gateway and a success or fail token, along with a transaction reference, is returned to the invoking use case
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.1.1-E-AUTH — Verify access is denied when session/authorization is invalid (Process Payment Transaction)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.1.1-E-AUTH
Related Use Case	UC-9.1.1 — Process Payment Transaction
Requirement Traceability	PR-PAY-06
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Process Payment Transaction", protecting data integrity and informing the user appropriately.

TEST CASE ELEMENT	SPECIFICATION
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Payment Gateway service is reachable • The invoking use case has supplied the transaction amount and context • Condition under test: Registered Reader (indirect, via invoking use case) / Payment Gateway's session has expired, or Registered Reader (indirect, via invoking use case) / Payment Gateway lacks the role/ownership permission required to perform "Process Payment Transaction", immediately before the state-changing step.
Test Data	Registered Reader (indirect, via invoking use case) / Payment Gateway account (as required by the scenario); a payment transaction configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Registered Reader (indirect, via invoking use case) / Payment Gateway. 2. Attempt to initiate the protected "Process Payment Transaction" action (payment transaction). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the payment transaction. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Transaction is processed by the Payment Gateway and a success or fail token, along with a transaction reference, is returned to the invoking use case
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-9.2: Purchase Single Article — Test Suite

Use case purpose: Process a Registered Reader's one-time purchase of a single premium article without a full subscription.

Actors: Registered Reader, Payment Gateway (secondary, via include: UC-9.1.1) **Requirement:** PR-PAY-02

Priority: Medium

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-9.2.

Test Case ID	Scenario	Type	Priority
TC-9.2-BF	Verify successful purchase single article (Basic Flow)	Positive	Medium
TC-9.2-A1	Verify system behavior for alternate flow: Bundle Purchase (Purchase Single Article)	Negative / Alternate	Medium
TC-9.2-E1	Verify system behavior for exception flow: Transaction Declined (Purchase Single Article)	Negative / Exception	Medium
TC-9.2-E-AUTH	Verify access is denied when session/authorization is invalid (Purchase Single Article)	Negative / Exception	Medium

TC-9.2-BF — Verify successful purchase single article (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.2-BF
Related Use Case	UC-9.2 — Purchase Single Article
Requirement Traceability	PR-PAY-02
Test Objective	Confirm that Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) can successfully complete the "Purchase Single Article" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Process a Registered Reader's one-time purchase of a single premium article without a full subscription.
Priority	Medium
Test Type	Positive
Preconditions	• System is online • Registered Reader is authenticated and logged in
Test Data	Valid, active Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) account with the required role/permission; a single-article purchase in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Registered Reader selects a premium article and chooses "Purchase Single Article." 2. System displays the price and includes Process Payment Transaction (UC-9.1.1) to capture and process the payment. 3. On receiving a success token from UC-9.1.1, System grants the Reader permanent access to that specific article. 4. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Reader is granted permanent access to the purchased article
Postconditions (Verification)	• Reader is granted permanent access to the purchased article
Actual Result	<i>To be recorded during test execution.</i>

TEST CASE ELEMENT	SPECIFICATION
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.2-A1 — Verify system behavior for alternate flow: Bundle Purchase (Purchase Single Article)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.2-A1
Related Use Case	UC-9.2 — Purchase Single Article
Requirement Traceability	PR-PAY-02
Test Objective	Confirm the system correctly handles the alternate scenario "Bundle Purchase" defined in Alternate Flow A1 of "Purchase Single Article", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	• System is online • Registered Reader is authenticated and logged in • Condition under test: The condition "Bundle Purchase" occurs for the single-article purchase during the "Purchase Single Article" workflow.
Test Data	Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) account (as required by the scenario); a single-article purchase configured so the "Bundle Purchase" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a single-article purchase / environment state that will trigger the "Bundle Purchase" condition. 2. As Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1), perform the relevant Basic Flow action(s) on the single-article purchase so that the "Bundle Purchase" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Bundle Purchase" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.

TEST CASE ELEMENT	SPECIFICATION
Postconditions (Verification)	• Reader is granted permanent access to the purchased article
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.2-E1 — Verify system behavior for exception flow: Transaction Declined (Purchase Single Article)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.2-E1
Related Use Case	UC-9.2 — Purchase Single Article
Requirement Traceability	PR-PAY-02
Test Objective	Confirm the system correctly handles the exception scenario "Transaction Declined" defined in Exception Flow E1 of "Purchase Single Article", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • Registered Reader is authenticated and logged in • Condition under test: The condition "Transaction Declined" occurs for the single-article purchase during the "Purchase Single Article" workflow.
Test Data	Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) account (as required by the scenario); a single-article purchase configured so the "Transaction Declined" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a single-article purchase / environment state that will trigger the "Transaction Declined" condition. 2. As Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1), perform the relevant Basic Flow action(s) on the single-article purchase so that the "Transaction Declined" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it.

TEST CASE ELEMENT	SPECIFICATION
	8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Transaction Declined" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Reader is granted permanent access to the purchased article
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.2-E-AUTH — Verify access is denied when session/authorization is invalid (Purchase Single Article)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.2-E-AUTH
Related Use Case	UC-9.2 — Purchase Single Article
Requirement Traceability	PR-PAY-02
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Purchase Single Article", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • Registered Reader is authenticated and logged in • Condition under test: Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1)'s session has expired, or Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) lacks the role/ownership permission required to perform "Purchase Single Article", immediately before the state-changing step.
Test Data	Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) account (as required by the scenario); a single-article purchase configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1). 2. Attempt to initiate the protected "Purchase Single Article" action (single-article purchase).

TEST CASE ELEMENT	SPECIFICATION
	3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the single-article purchase. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Reader is granted permanent access to the purchased article
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-9.3: Purchase Single Digital Issue — Test Suite

Use case purpose: Process a Registered Reader's one-time purchase of a single digital issue without a full subscription.

Actors: Registered Reader, Payment Gateway (secondary, via include: UC-9.1.1) **Requirement:** PR-PAY-03

Priority: Medium

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-9.3.

Test Case ID	Scenario	Type	Priority
TC-9.3-BF	Verify successful purchase single digital issue (Basic Flow)	Positive	Medium
TC-9.3-A1	Verify system behavior for alternate flow: Gift Issue (Purchase Single Digital Issue)	Negative / Alternate	Medium
TC-9.3-E1	Verify system behavior for exception flow: Transaction Declined (Purchase Single Digital Issue)	Negative / Exception	Medium
TC-9.3-E-AUTH	Verify access is denied when session/authorization is invalid (Purchase Single Digital Issue)	Negative / Exception	Medium

TC-9.3-BF — Verify successful purchase single digital issue (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.3-BF
Related Use Case	UC-9.3 — Purchase Single Digital Issue

TEST CASE ELEMENT	SPECIFICATION
Requirement Traceability	PR-PAY-03
Test Objective	Confirm that Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) can successfully complete the "Purchase Single Digital Issue" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Process a Registered Reader's one-time purchase of a single digital issue without a full subscription.
Priority	Medium
Test Type	Positive
Preconditions	• System is online • Registered Reader is authenticated and logged in
Test Data	Valid, active Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) account with the required role/permission; a single digital issue purchase in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Registered Reader selects an issue and chooses "Purchase Single Digital Issue." 2. System displays the price and includes Process Payment Transaction (UC-9.1.1) to capture and process the payment. 3. On receiving a success token from UC-9.1.1, System grants the Reader access and download rights to that issue. 4. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Reader is granted access and download rights to the purchased issue
Postconditions (Verification)	• Reader is granted access and download rights to the purchased issue
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.3-A1 — Verify system behavior for alternate flow: Gift Issue (Purchase Single Digital Issue)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.3-A1
Related Use Case	UC-9.3 — Purchase Single Digital Issue
Requirement Traceability	PR-PAY-03
Test Objective	Confirm the system correctly handles the alternate scenario "Gift Issue" defined in Alternate Flow A1 of "Purchase Single Digital Issue", protecting data integrity and informing the user appropriately.
Priority	Medium

TEST CASE ELEMENT	SPECIFICATION
Test Type	Negative / Alternate
Preconditions	• System is online • Registered Reader is authenticated and logged in • Condition under test: The condition "Gift Issue" occurs for the single digital issue purchase during the "Purchase Single Digital Issue" workflow.
Test Data	Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) account (as required by the scenario); a single digital issue purchase configured so the "Gift Issue" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a single digital issue purchase / environment state that will trigger the "Gift Issue" condition. 2. As Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1), perform the relevant Basic Flow action(s) on the single digital issue purchase so that the "Gift Issue" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Gift Issue" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Reader is granted access and download rights to the purchased issue
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.3-E1 — Verify system behavior for exception flow: Transaction Declined (Purchase Single Digital Issue)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.3-E1
Related Use Case	UC-9.3 — Purchase Single Digital Issue
Requirement Traceability	PR-PAY-03

TEST CASE ELEMENT	SPECIFICATION
Test Objective	Confirm the system correctly handles the exception scenario "Transaction Declined" defined in Exception Flow E1 of "Purchase Single Digital Issue", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • Registered Reader is authenticated and logged in • Condition under test: The condition "Transaction Declined" occurs for the single digital issue purchase during the "Purchase Single Digital Issue" workflow.
Test Data	Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) account (as required by the scenario); a single digital issue purchase configured so the "Transaction Declined" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a single digital issue purchase / environment state that will trigger the "Transaction Declined" condition. 2. As Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1), perform the relevant Basic Flow action(s) on the single digital issue purchase so that the "Transaction Declined" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Transaction Declined" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Reader is granted access and download rights to the purchased issue
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.3-E-AUTH — Verify access is denied when session/authorization is invalid (Purchase Single Digital Issue)

TEST CASE SPECIFICATION	
Test Case ID	TC-9.3-E-AUTH
Related Use Case	UC-9.3 — Purchase Single Digital Issue
Requirement Traceability	PR-PAY-03
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Purchase Single Digital Issue", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • Registered Reader is authenticated and logged in • Condition under test: Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1)'s session has expired, or Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) lacks the role/ownership permission required to perform "Purchase Single Digital Issue", immediately before the state-changing step.
Test Data	Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1) account (as required by the scenario); a single digital issue purchase configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Registered Reader / Payment Gateway (secondary, via include: UC-9.1.1). 2. Attempt to initiate the protected "Purchase Single Digital Issue" action (single digital issue purchase). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the single digital issue purchase. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Reader is granted access and download rights to the purchased issue
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-9.4: Manage Billing Information — Test Suite

Use case purpose: Allow a Subscriber to view and update their stored billing and payment details.

Actors: Subscriber, Payment Gateway **Requirement:** PR-PAY-04 **Priority:** Medium

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-9.4.

Test Case ID	Scenario	Type	Priority
TC-9.4-BF	Verify successful manage billing information (Basic Flow)	Positive	Medium
TC-9.4-A1	Verify system behavior for alternate flow: Backup Payment Method (Manage Billing Information)	Negative / Alternate	Medium
TC-9.4-E1	Verify system behavior for exception flow: Method Rejected (Manage Billing Information)	Negative / Exception	Medium
TC-9.4-E-AUTH	Verify access is denied when session/authorization is invalid (Manage Billing Information)	Negative / Exception	Medium

TC-9.4-BF — Verify successful manage billing information (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.4-BF
Related Use Case	UC-9.4 — Manage Billing Information
Requirement Traceability	PR-PAY-04
Test Objective	Confirm that Subscriber / Payment Gateway can successfully complete the "Manage Billing Information" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow a Subscriber to view and update their stored billing and payment details.
Priority	Medium
Test Type	Positive
Preconditions	• System is online • Subscriber is authenticated and logged in
Test Data	Valid, active Subscriber / Payment Gateway account with the required role/permission; a billing information record in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Subscriber navigates to Billing Settings. 2. Subscriber selects “Manage Billing Information.” 3. System retrieves current billing details via the Payment Gateway (masked card info). 4. Subscriber updates the payment method or billing address. 5. System validates the update with the Payment Gateway. 6. System confirms the update.

TEST CASE ELEMENT	SPECIFICATION
	7. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Billing information is updated and validated with the Payment Gateway
Postconditions (Verification)	• Billing information is updated and validated with the Payment Gateway
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.4-A1 — Verify system behavior for alternate flow: Backup Payment Method (Manage Billing Information)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.4-A1
Related Use Case	UC-9.4 — Manage Billing Information
Requirement Traceability	PR-PAY-04
Test Objective	Confirm the system correctly handles the alternate scenario "Backup Payment Method" defined in Alternate Flow A1 of "Manage Billing Information", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Alternate
Preconditions	• System is online • Subscriber is authenticated and logged in • Condition under test: The condition "Backup Payment Method" occurs for the billing information record during the "Manage Billing Information" workflow.
Test Data	Subscriber / Payment Gateway account (as required by the scenario); a billing information record configured so the "Backup Payment Method" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a billing information record / environment state that will trigger the "Backup Payment Method" condition. 2. As Subscriber / Payment Gateway, perform the relevant Basic Flow action(s) on the billing information record so that the "Backup Payment Method" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state.

TEST CASE ELEMENT	SPECIFICATION
	6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Backup Payment Method" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Billing information is updated and validated with the Payment Gateway
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.4-E1 — Verify system behavior for exception flow: Method Rejected (Manage Billing Information)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.4-E1
Related Use Case	UC-9.4 — Manage Billing Information
Requirement Traceability	PR-PAY-04
Test Objective	Confirm the system correctly handles the exception scenario "Method Rejected" defined in Exception Flow E1 of "Manage Billing Information", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • Subscriber is authenticated and logged in • Condition under test: The condition "Method Rejected" occurs for the billing information record during the "Manage Billing Information" workflow.
Test Data	Subscriber / Payment Gateway account (as required by the scenario); a billing information record configured so the "Method Rejected" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a billing information record / environment state that will trigger the "Method Rejected" condition. 2. As Subscriber / Payment Gateway, perform the relevant Basic Flow action(s) on the billing information record so that the "Method Rejected" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow.

TEST CASE ELEMENT	SPECIFICATION
	4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Method Rejected" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Billing information is updated and validated with the Payment Gateway
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.4-E-AUTH — Verify access is denied when session/authorization is invalid (Manage Billing Information)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.4-E-AUTH
Related Use Case	UC-9.4 — Manage Billing Information
Requirement Traceability	PR-PAY-04
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Manage Billing Information", protecting data integrity and informing the user appropriately.
Priority	Medium
Test Type	Negative / Exception
Preconditions	• System is online • Subscriber is authenticated and logged in • Condition under test: Subscriber / Payment Gateway's session has expired, or Subscriber / Payment Gateway lacks the role/ownership permission required to perform "Manage Billing Information", immediately before the state-changing step.
Test Data	Subscriber / Payment Gateway account (as required by the scenario); a billing information record configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).

TEST CASE ELEMENT	SPECIFICATION
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Subscriber / Payment Gateway. 2. Attempt to initiate the protected "Manage Billing Information" action (billing information record). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the billing information record. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Billing information is updated and validated with the Payment Gateway
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

UC-9.5: Cancel Subscription — Test Suite

Use case purpose: Allow a Subscriber to cancel their recurring premium subscription.

Actors: Subscriber, Payment Gateway **Requirement:** PR-PAY-05 **Priority:** High

This suite contains 4 test case(s) derived from the Basic Flow and every documented Alternate and Exception Flow of UC-9.5.

Test Case ID	Scenario	Type	Priority
TC-9.5-BF	Verify successful cancel subscription (Basic Flow)	Positive	High
TC-9.5-A1	Verify system behavior for alternate flow: Rescind Cancellation (Cancel Subscription)	Negative / Alternate	High
TC-9.5-E1	Verify system behavior for exception flow: Gateway Acknowledgment Failure (Cancel Subscription)	Negative / Exception	High
TC-9.5-E-AUTH	Verify access is denied when session/authorization is invalid (Cancel Subscription)	Negative / Exception	High

TC-9.5-BF — Verify successful cancel subscription (Basic Flow)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.5-BF

TEST CASE ELEMENT	SPECIFICATION
Related Use Case	UC-9.5 — Cancel Subscription
Requirement Traceability	PR-PAY-05
Test Objective	Confirm that Subscriber / Payment Gateway can successfully complete the "Cancel Subscription" use case end-to-end via the documented Basic Flow, and that the system reaches all specified postconditions. Allow a Subscriber to cancel their recurring premium subscription.
Priority	High
Test Type	Positive
Preconditions	• System is online • Subscriber is authenticated and logged in • Subscriber has an active subscription
Test Data	Valid, active Subscriber / Payment Gateway account with the required role/permission; a subscription cancellation request in the exact starting state described in the Preconditions above, populated with valid, complete data for every field referenced in the Basic Flow.
Test Steps	1. Subscriber navigates to Billing Settings. 2. Subscriber selects “Cancel Subscription.” 3. System prompts for confirmation and an optional cancellation reason. 4. Subscriber confirms. 5. System notifies the Payment Gateway to stop future recurring charges. 6. System schedules the downgrade to Registered Reader tier at the end of the current billing cycle. 7. Verify each postcondition listed below is met.
Expected Result	The system completes every Basic Flow step without error and reaches the following state: - Recurring charges are stopped and the account is scheduled to downgrade at the end of the billing cycle
Postconditions (Verification)	• Recurring charges are stopped and the account is scheduled to downgrade at the end of the billing cycle
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.5-A1 — Verify system behavior for alternate flow: Rescind Cancellation (Cancel Subscription)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.5-A1
Related Use Case	UC-9.5 — Cancel Subscription

TEST CASE SPECIFICATION	
Requirement Traceability	PR-PAY-05
Test Objective	Confirm the system correctly handles the alternate scenario "Rescind Cancellation" defined in Alternate Flow A1 of "Cancel Subscription", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Alternate
Preconditions	• System is online • Subscriber is authenticated and logged in • Subscriber has an active subscription • Condition under test: The condition "Rescind Cancellation" occurs for the subscription cancellation request during the "Cancel Subscription" workflow.
Test Data	Subscriber / Payment Gateway account (as required by the scenario); a subscription cancellation request configured so the "Rescind Cancellation" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a subscription cancellation request / environment state that will trigger the "Rescind Cancellation" condition. 2. As Subscriber / Payment Gateway, perform the relevant Basic Flow action(s) on the subscription cancellation request so that the "Rescind Cancellation" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Alternate Flow A1: it detects and validates the "Rescind Cancellation" condition, performs only the permitted handling, clearly communicates the resulting state, and (where applicable) allows the user to continue to a valid end state.
Postconditions (Verification)	• Recurring charges are stopped and the account is scheduled to downgrade at the end of the billing cycle
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.5-E1 — Verify system behavior for exception flow: Gateway Acknowledgment Failure (Cancel Subscription)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.5-E1
Related Use Case	UC-9.5 — Cancel Subscription
Requirement Traceability	PR-PAY-05
Test Objective	Confirm the system correctly handles the exception scenario "Gateway Acknowledgment Failure" defined in Exception Flow E1 of "Cancel Subscription", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Subscriber is authenticated and logged in • Subscriber has an active subscription • Condition under test: The condition "Gateway Acknowledgment Failure" occurs for the subscription cancellation request during the "Cancel Subscription" workflow.
Test Data	Subscriber / Payment Gateway account (as required by the scenario); a subscription cancellation request configured so the "Gateway Acknowledgment Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Prepare a subscription cancellation request / environment state that will trigger the "Gateway Acknowledgment Failure" condition. 2. As Subscriber / Payment Gateway, perform the relevant Basic Flow action(s) on the subscription cancellation request so that the "Gateway Acknowledgment Failure" condition is encountered. 3. Verify the system detects the condition at the correct point in the Basic Flow. 4. Verify the system validates the current record, permission, and applicable business rules. 5. Verify the system performs only the permitted handling (completes the allowed alternate action, or blocks the invalid operation) and does not silently allow an inconsistent state. 6. Verify the system clearly explains the resulting state and the required next action to the user. 7. Verify the event is recorded in the audit log where audit/security rules require it. 8. Resolve the condition (correct the data / retry after recovery) and verify the user can continue from the appropriate point, reaching the normal outcome.
Expected Result	The system correctly follows Exception Flow E1: it detects the "Gateway Acknowledgment Failure" condition, prevents any invalid or partial state change, clearly communicates the failure/resulting state, logs the event where required, and allows a successful retry once the condition is resolved.
Postconditions (Verification)	• Recurring charges are stopped and the account is scheduled to downgrade at the end of the billing cycle

TEST CASE ELEMENT	SPECIFICATION
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

TC-9.5-E-AUTH — Verify access is denied when session/authorization is invalid (Cancel Subscription)

TEST CASE ELEMENT	SPECIFICATION
Test Case ID	TC-9.5-E-AUTH
Related Use Case	UC-9.5 — Cancel Subscription
Requirement Traceability	PR-PAY-05
Test Objective	Confirm the system correctly handles the exception scenario "Authorization Or Session Failure" defined in Exception Flow E-AUTH of "Cancel Subscription", protecting data integrity and informing the user appropriately.
Priority	High
Test Type	Negative / Exception
Preconditions	• System is online • Subscriber is authenticated and logged in • Subscriber has an active subscription • Condition under test: Subscriber / Payment Gateway's session has expired, or Subscriber / Payment Gateway lacks the role/ownership permission required to perform "Cancel Subscription", immediately before the state-changing step.
Test Data	Subscriber / Payment Gateway account (as required by the scenario); a subscription cancellation request configured so the "Authorization Or Session Failure" condition can be reproduced on demand (e.g., via test fixtures, mocked service failure, or intentionally invalid/incomplete input).
Test Steps	1. Simulate an expired session (or log in as a user without the required role/ownership permission) for Subscriber / Payment Gateway. 2. Attempt to initiate the protected "Cancel Subscription" action (subscription cancellation request). 3. Verify the system detects the expired session or missing permission immediately before the state-changing action executes. 4. Verify the system blocks the action and does NOT partially commit any change to the subscription cancellation request. 5. Verify the system records the denied attempt in the audit/security log, where required by policy. 6. Verify the system redirects the user to authentication, or displays an "Access Denied" message. 7. Re-authenticate / obtain the required permission, retry the action, and verify it now proceeds via the Basic Flow.

TEST CASE ELEMENT	SPECIFICATION
Expected Result	The action is blocked with no partial state change; the denial is logged where policy requires it; the user is redirected to authentication or shown an access-denied message; the action succeeds normally once authorization is restored.
Postconditions (Verification)	• Recurring charges are stopped and the account is scheduled to downgrade at the end of the billing cycle
Actual Result	<i>To be recorded during test execution.</i>
Status (Pass/Fail)	<i>Not yet executed.</i>

10. Requirement-to-Test Traceability Matrix

Maps each use case and its inherited requirement ID to the test cases that provide coverage.

Use Case	Requirement ID	Test Case IDs
UC-1.1 Submit Bundle	PR-EDT-01	TC-1.1-BF, TC-1.1-A1, TC-1.1-E1, TC-1.1-E-AUTH
UC-1.2 Draft Article	PR-EDT-02	TC-1.2-BF, TC-1.2-A1, TC-1.2-E1
UC-1.3 Edit Article Content	PR-EDT-03	TC-1.3-BF, TC-1.3-A1, TC-1.3-E1
UC-1.4 Request Revision	PR-EDT-04	TC-1.4-BF, TC-1.4-A1, TC-1.4-A2, TC-1.4-E1, TC-1.4-E2
UC-1.5 Override AI	PR-EDT-05	TC-1.5-BF, TC-1.5-A1, TC-1.5-E1
UC-1.6 Update Article Workflow Status	PR-EDT-06	TC-1.6-BF, TC-1.6-A1, TC-1.6-E1, TC-1.6-E-AUTH
UC-1.7 Withdraw Article	PR-EDT-07	TC-1.7-BF, TC-1.7-A1, TC-1.7-E1, TC-1.7-E-AUTH
UC-1.8 Upload Magazine Design	PR-EDT-08	TC-1.8-BF, TC-1.8-A1, TC-1.8-A2, TC-1.8-E1, TC-1.8-E2, TC-1.8-E3, TC-1.8-E4
UC-1.9 View Pipeline Dashboard	PR-EDT-09	TC-1.9-BF, TC-1.9-A1, TC-1.9-E1
UC-1.10 View Bundle	PR-EDT-10	TC-1.10-BF, TC-1.10-A1, TC-1.10-E1
UC-1.11 Generate Article Evaluation	PR-EDT-11	TC-1.11-BF, TC-1.11-A1, TC-1.11-E1
UC-2.1 Configure Issue Formats	PR-PUB-01	TC-2.1-BF, TC-2.1-A1, TC-2.1-E1
UC-2.2 Schedule Issue Release	PR-PUB-02	TC-2.2-BF, TC-2.2-A1, TC-2.2-E1, TC-2.2-E-AUTH
UC-2.3 Execute Live Publishing	PR-PUB-03	TC-2.3-BF, TC-2.3-A1, TC-2.3-E1, TC-2.3-E-AUTH
UC-2.3.1 Trigger Subscriber Notifications	PR-PUB-04	TC-2.3.1-BF, TC-2.3.1-A1, TC-2.3.1-E1, TC-2.3.1-E2
UC-2.5 Archive Previous Issue	PR-PUB-05	TC-2.5-BF, TC-2.5-A1, TC-2.5-E1, TC-2.5-E-AUTH
UC-3.1 Generate Revenue Report	PR-RPT-01	TC-3.1-BF, TC-3.1-A1, TC-3.1-E1
UC-3.1.1 Export Report Data	PR-RPT-04	TC-3.1.1-BF, TC-3.1.1-A1, TC-3.1.1-E1
UC-3.2 Generate Editorial Pipeline Report	PR-RPT-02	TC-3.2-BF, TC-3.2-A1, TC-3.2-E1
UC-3.3 Generate AI Evaluation Summary	PR-RPT-03	TC-3.3-BF, TC-3.3-A1, TC-3.3-E1
UC-4.1 Update Site Theme and Layout	PR-SET-01	TC-4.1-BF, TC-4.1-A1, TC-4.1-E1, TC-4.1-E-AUTH
UC-4.2 Configure System Integrations	PR-SET-02	TC-4.2-BF, TC-4.2-A1, TC-4.2-E1, TC-4.2-E-AUTH
UC-4.3 Toggle Maintenance Mode	PR-SET-03	TC-4.3-BF, TC-4.3-A1, TC-4.3-E1, TC-4.3-E-AUTH
UC-5.1 Add New User	PR-USR-01	TC-5.1-BF, TC-5.1-A1, TC-5.1-E1, TC-5.1-E-AUTH
UC-5.2 View User Directory	PR-USR-02	TC-5.2-BF, TC-5.2-A1, TC-5.2-E1
UC-5.2.1 Update User Role	PR-USR-03	TC-5.2.1-BF, TC-5.2.1-A1, TC-5.2.1-E1
UC-5.2.2 Suspend User Account	PR-USR-04	TC-5.2.2-BF, TC-5.2.2-A1, TC-5.2.2-E1
UC-5.2.3 Force Password Reset	PR-USR-05	TC-5.2.3-BF, TC-5.2.3-A1, TC-5.2.3-E1
UC-6.1 Register Account	PR-ACC-01	TC-6.1-BF, TC-6.1-A1, TC-6.1-E1
UC-6.2 Authenticate User (Log In)	PR-ACC-02	TC-6.2-BF, TC-6.2-A1, TC-6.2-E1, TC-6.2-E2

Use Case	Requirement ID	Test Case IDs
UC-6.3 Subscribe	PR-ACC-03	TC-6.3-BF, TC-6.3-A1, TC-6.3-A2, TC-6.3-A3, TC-6.3-E1, TC-6.3-E-AUTH
UC-6.4 Update Profile Information	PR-ACC-04	TC-6.4-BF, TC-6.4-A1, TC-6.4-E1, TC-6.4-E-AUTH
UC-6.5 Manage Account Security	PR-ACC-05	TC-6.5-BF, TC-6.5-A1, TC-6.5-E1, TC-6.5-E-AUTH
UC-7.1 Browse Featured Content	PR-INT-01	TC-7.1-BF, TC-7.1-A1, TC-7.1-E1
UC-7.1.1 Bookmark Content	PR-INT-02	TC-7.1.1-BF, TC-7.1.1-A1, TC-7.1.1-E1, TC-7.1.1-E-AUTH
UC-7.1.2 Share Content	PR-INT-03	TC-7.1.2-BF, TC-7.1.2-A1, TC-7.1.2-E1
UC-8.1 Download Digital Issues	PR-PRM-01	TC-8.1-BF, TC-8.1-A1, TC-8.1-E1
UC-8.2 Access Premium Articles	PR-PRM-02	TC-8.2-BF, TC-8.2-A1, TC-8.2-E1
UC-9.1 Subscribe to Premium Plan	PR-PAY-01	TC-9.1-BF, TC-9.1-A1, TC-9.1-E1, TC-9.1-E-AUTH
UC-9.1.1 Process Payment Transaction	PR-PAY-06	TC-9.1.1-BF, TC-9.1.1-A1, TC-9.1.1-E1, TC-9.1.1-E2, TC-9.1.1-E-AUTH
UC-9.2 Purchase Single Article	PR-PAY-02	TC-9.2-BF, TC-9.2-A1, TC-9.2-E1, TC-9.2-E-AUTH
UC-9.3 Purchase Single Digital Issue	PR-PAY-03	TC-9.3-BF, TC-9.3-A1, TC-9.3-E1, TC-9.3-E-AUTH
UC-9.4 Manage Billing Information	PR-PAY-04	TC-9.4-BF, TC-9.4-A1, TC-9.4-E1, TC-9.4-E-AUTH
UC-9.5 Cancel Subscription	PR-PAY-05	TC-9.5-BF, TC-9.5-A1, TC-9.5-E1, TC-9.5-E-AUTH